House Price Predictor using Machine Learning

Contents

Analysis **6**
Identifying the problem 6
A computational approach 6
Features that make the problem solvable by computational methods 6
Computational methods that the solution lends itself to 7
Stakeholders 9
Identifying and describing stakeholders 9
How they will use proposed solution 9
Interview questions 10
Interview responses 11
How solution is appropriate 12
Research 13
Similar solutions 13
Key features 18
Explanation of features 18
Limitations 19
Potential limitations due to time restraints 20
Requirements 21
Stakeholder program requirements 21
Software requirements 22
Hardware requirements 22
Success Criteria 23
Justification 24
Design **25**
Problem decomposition 25
Explanation and justification 25
Justification as a whole 27
Diagrams 28
File structure 30
Algorithms 31
Description of algorithms 31
Justification 37

Program Flowchart 38
Subroutines 39
UI design 43
Concept 43
Visualisation 44
Justification 45
Breakdown of UI design 45
Outlining method of development 46
Stakeholder input 47
Replies 47
Inputs and outputs 49
Key variables 50
Other variables 51
Validation 52
Iterative testing method 53
Post development testing 63
Development and testing 66
Phase 1: Web scraping development 66
Overview 66
Development 66
Validation and testing 88
Failures 89
Review 90
Phase 2: Machine learning model 91
Overview 91
Development 91
Validation and Testing 95
Review 96
Phase 3: User Interface 97
Overview 97
Development 97
Testing 109
Correcting Errors 109
Testing again 110

Review 111
Phase 4: Calculating a prediction 112
Overview 112
Development 112
Testing 116
Review 116
Phase 5: Presenting similar properties 117
Overview 117
Development 117
Testing 127
Correcting Errors and Improvements 127
Testing again 129
Review 130
Phase 6: Range of estimates 131
Overview 131
Development 131
Testing 132
Review 132
Phase 7: Interactive Map 134
Overview 134
Development 134
Testing 139
Correcting errors 139
Testing again 143
Review 143
Additional developments 145
Evaluation 147
Validation testing for usability and robustness 147
Testing 147
Review 148
Post development testing 149
Test 1 149
Test 2 150
Feedback 151

Review 151
Have the success criteria been met? 152
Requirement 1 152
Requirement 2 155
Requirement 3 157
Requirement 4 159
Requirement 5 161
Requirement 6 162
Requirement 7 163
Requirement 8 164
Further development to meet all success criteria 165
Usability features 166
Evidence 166
Evaluating success 170
Further development of usability features 171
Maintenance issues and limitations of the solution 172
Improvements to the solution 174
Final code 175
webScraper.py 175
trainModel.py 180
ui.py 183
predictPrice.py 192
similarProperties.py 194
rangeEstimates.py 198
map.py 199
Bibliography 201

Analysis

Identifying the problem

The housing market has become even more volatile with high interest rates and the effects of the cost-of-living crisis. As a result, it has become much harder for potential buyers to know the right cost to pay for a property, or for sellers to know what price to sell for. Hence, many people opt to use an estate agent as they have expertise in this area. However, they may have limited availability and would not be able to provide an instant evaluation - something buyers seek to give reassurance.

There are also some other evaluation websites online, but they are often full of advertisements and require registration to their websites before use. They also currently provide vague predictions, which are on a separate page to property listings, meaning it is difficult to quickly see both the predicted price and the other property listings. Hence, there is no way of seeing the predicted price in context of similar properties in the real market.

My proposed solution would provide an instant evaluation of the property, for the client to use in conjunction with estate agents. The evaluation would also come with additional information such as the lowest and highest prices that a similar property is priced at, giving the client all the information they would need in one place.

A computational approach

Features that make the problem solvable by computational methods

For this specific problem, a computational approach would be most appropriate for a number of reasons. The main focus of this project is the use of machine learning and artificial intelligence to predict the price of a property, which requires a computational approach as there is simply no other way of using machine learning. Furthermore, to collect sufficient data to train an accurate model, a computational approach is also needed, because of the speed and volume of data required – a physical collection and extraction of data would be too slow and would be vulnerable to human error. Instead, software can be used to scrape the data from websites and organize into a file. In addition, another aspect of the project is providing the information in a clear and easy-to-understand manner. This is best suited to a computational approach as the formatting can be adjusted and changed until perfect and would provide a guaranteed output as desired. Therefore, this solution is best suited to a computational approach.

Computational methods that the solution lends itself to

Decomposition

Decomposition is the modularisation of a problem into smaller more manageable components, which can then be individually focussed on and solved. This problem and hence solution can be decomposed into a number of smaller steps, including:

- Using web scraping to create a dataset
- Training the machine learning model based on the dataset
- Making a prediction using the trained model
- Outputting the prediction along with other relevant data

Algorithmic methods

This is the use of step-by-step instructions for solving a specific problem. For this problem, multiple algorithms can be used, for example:

- Machine learning algorithms
- Searching algorithms
- Sorting algorithms
- Web scraping algorithms

Data structures

The use of data structures is imperative for this solution due to the focus on machine learning. Some of the data structures that may be used can include:

- Arrays
- Dictionaries
- CSV files – for permanent storage

Machine learning

This is the use of algorithms and statistical models to teach and train a model to make predictions. This is the main focus of my solution due to the adaptability of machine learning models, which are suited to data analysis and pattern recognition. This makes it perfect for my problem, as it will be able to use the vast amount of data to create its own models and make predictions based on historical data.

Data mining

This is the extraction of patterns and insights from large datasets. This is heavily related to machine learning, which will create its own patters from the large dataset that it will be trained on.

Web scraping

Web scraping is the automatic collection of data from the web. This will be used in my solution to collect information regarding houses that are on the market. Web scraping allows my machine learning model to be trained on relevant and up-to-date data, which is essential for producing an accurate prediction.

Abstraction

Abstraction is the focus on relevant data and removal of unnecessary information. This will be used in my solution throughout, for example only choosing relevant data from websites when training the machine learning model and ignoring excess information. Furthermore, when presenting the information to the client at the end, abstraction will be used to provide the user with only information that is relevant.

Stakeholders

Identifying and describing stakeholders

There are multiple different groups of stakeholders – those with an interest in my solution. As my solution is related to the property market, the stakeholders are all limited to this area, and are as follows:

- Property buyers
- Property sellers
- Estate agents
- Developers

How they will use proposed solution

All stakeholders will use my program for the main purpose of obtaining a predicted price for a property, though for varying reasons:

Group 1 - Property buyers

This group of people are potential buyers of property and are looking for properties to buy. They are wary of overpaying in their purchase and want to ensure that they are paying a fair and good price for their property. They will use my program to predict the price of a property they want to buy.

Group 2 - Property sellers

This group of people want to sell their home. However, with a variable property market, they are keen to sell it for the price that it is worth so that they would not lose out on any money they could have earned. They will use my program to help them set the selling price.

Group 3 - Estate agents

Estate agents have a lot of experience in the property market. However, with rising interest rates and the cost-of-living having a large impact on the market, prices are now ever-changing, and the market is changing at a dramatic rate. They want to ensure that they are up to date with property prices and are pitching their client's property at the right price. They will use my program to help them understand the market prices better.

Group 4 - Developers

This group of people are looking to build new property and repurpose areas of land to put onto the housing market. They want to know how much money they would be able to generate from selling the property once they have developed it, and hence the profitability of developing the land. They will use my product to predict the price of the development.

Interview questions

I will ask some questions to each stakeholder and record their responses. These questions should help me to understand their needs and help me identify any potential problems to solve.

Questions for Group 1 – Property buyers

1. What do you currently use to help you buy property on the market?
2. Are you satisfied with the options currently available?
3. If no, what do you think you are missing?
4. How confident would you be in paying the right price?
5. What could help improve your confidence level?
6. What information would you like to see along with the price?
7. Do you have anything else to add?

Questions for Group 2 – Property sellers

1. What do you currently use to help you put property on the market?
2. Are you satisfied with the options currently available?
3. If no, what do you think you are missing?
4. How confident are you in selling at the right price?
5. What could help improve your confidence level?
6. What information would you like to see along with the price?
7. Do you have anything else to add?

Questions for Group 3 – Estate agents

1. How would you describe the housing market currently?
2. What factors go into the price of a property?
3. How difficult is it to set the price of a property?
4. Would you be open to using additional information to help set a price?
5. What information would you like to see along with the price?
6. Do you have anything else to add?

Questions for Group 4 – Developers

1. What do you look for when deciding to develop some land?
2. How important would the price of the property be?
3. How do you come up with the price that you will sell the property for?
4. Would you be open to using additional information to help set a price?
5. What information would you like to see along with the price?
6. Do you have anything else to add?

Commentary on questions

The questions for Group 1 (property buyers) and Group 2 (property sellers) are very similar. This is because they will be looking for very similar things, only one is selling and one is buying. These

questions will also help me to understand what my stakeholders are currently using, and what they would like to see in my solution. This will act as a guide for when I develop my solution. They cover a broad range of topics and allow for the stakeholders to add any extra information they feel would be useful.

Interview responses

Group 1 – Property buyers (Andrew)

1. "I usually use Rightmove and estate agents."
2. "Reasonably so, yes"
3. "Maybe some way to easily compare different properties directly."
4. "I just follow the estate agent's advice and trust that they are right."
5. "If there was another way to obtain an estimate which matched the estate agent's valuation that would give me full confidence."
6. "Other details like location, size, number of bedrooms, potentially how far away from stations."
7. "No, that is all."

Group 2 – Property sellers (Bianca)

1. "Estate agents mainly"
2. "Previously some estate agents are very slow to give valuations, and do not give competitive valuations. However, there are others which are very good."
3. "The estate agents which I have had problems with before do not give prices that are matching with the market."
4. "I usually can trust in the estate agents, though I also conduct my own research to understand what the market is like, especially similar properties in my area."
5. "Knowing that similar properties to mine have been sold at similar prices."
6. "How the price compares with other properties, and how high or low the estimate is."
7. "No."

Group 3 – Estate agents (Colin)

1. "The market is very dynamic. It changes depending on external factors such as stamp duty, interest rates, mortgage rates, just to name a few."
2. "There are so many factors, including location, type of property, number of bedrooms, developments on the property, number of bathrooms, size of property – just to name a few."
3. "You're always trying to set a good price for the client – something that will give them good return, whilst also attractive for the buyers. Nobody wants to a property to sit on the market for too long."
4. "Personally, I wouldn't mind. It will do no harm to be given extra information."
5. "I would like to see how it compares in the local area, and historical data."
6. "No that would be it."

Group 4 – Developers (Dave)

1. “Location is very important. It determines pretty much the potential of the property and the price bracket it will be sold at.”
2. “The price bracket is highly pivotal, it determines the budget of the whole project, and to what level we build to. If we are building a house with the aim to be sold at a certain price, we want to ensure we are not over building for that price.”
3. “We look at property in the area, especially similar to what we are looking at to make our decision.”
4. “I mean additional information can’t hurt, so yeah sure.”
5. “I want to see location, and other similar properties in the local area.”
6. “No.”

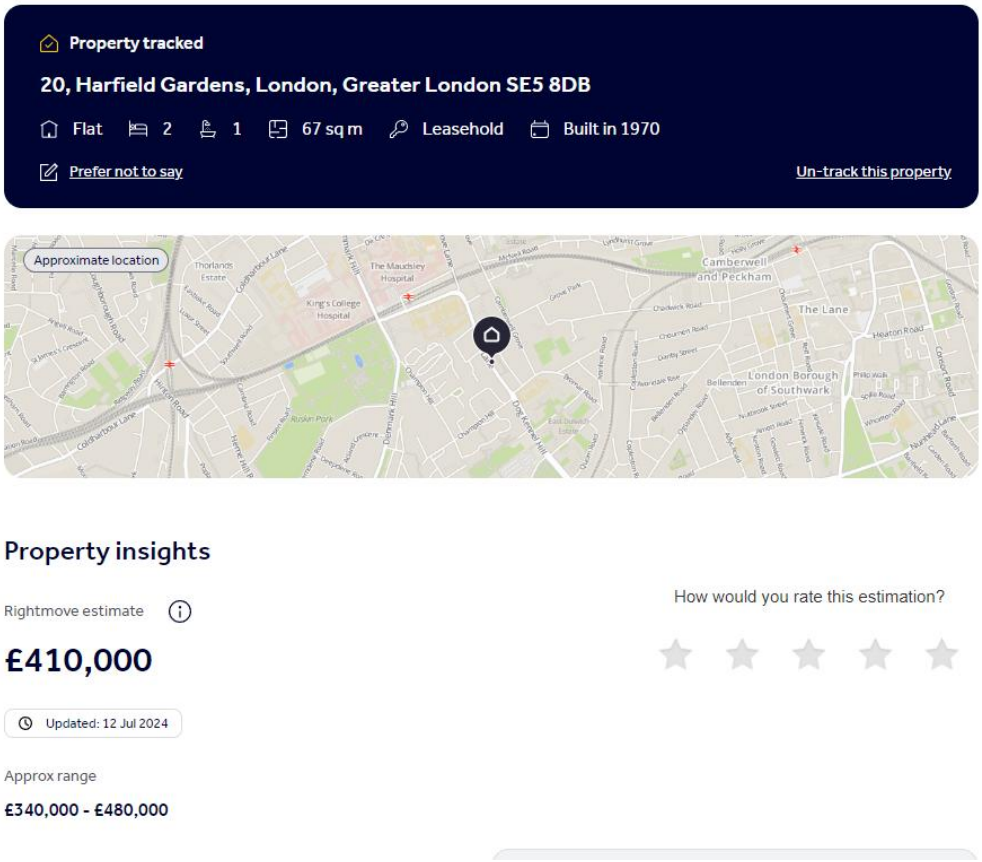
How solution is appropriate

As demonstrated in the stakeholder interviews, all stakeholders have shown some level of interest towards this solution. They all require a product to help inform them when purchasing or selling property. As of right now, there doesn’t seem to be a product which provides everything that all stakeholders need. There are some requirements outlined by the stakeholders, which my solution will look to meet, such as showing similar properties in the local area, which was mentioned by all four stakeholders. My solution should also provide a more accurate prediction, which is what the stakeholders are looking for.

Research

Similar solutions

Rightmove 'Instant online valuation'



Overview

Rightmove has an instant online valuation page on their website. After providing the address, it will provide an estimate value for the property. It gives additional information such as the type of property, the number of bedrooms, the area of the property, and shows the location of the property on a map. Furthermore, they have given an approximate range for the value, although the price range is very broad. The information is outputted in a very simple layout, which is quite simple to understand.

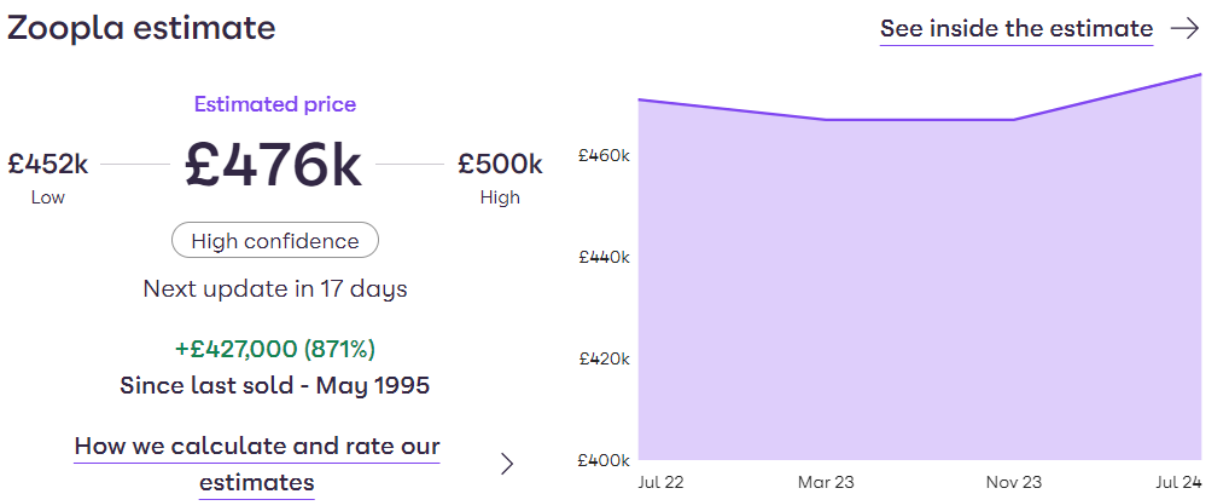
Parts that I can apply to my solution

This concept is very similar to my solution. I will also use an estimate price and display the information regarding the property, such as the address, property type, number of bedrooms etc. However, I will give additional information regarding the pricing, such as the averages for location, and the ranges of prices for similar properties.

Justification of suitable approach

My program will follow a similar approach as the core functionality of this similar solution is close to what the stakeholders have requested. The information about the property must be displayed for usability and to ensure the user knows what property the predicted price is representing. This makes the program clear and easy to use to help benefit the stakeholders.

Zoopla 'instant valuation'



Overview

Zoopla is another property insight company which offers an instant valuation. It gives a smaller range of prices for the specific property – which was the same one used in Rightmove's. It gives a confidence rating, as well as showing previous history of prices, ranging two years.

Parts that I can apply to my solution

I like the way it displays the ranges of price and the estimated price. The information about last sold is also useful, and I would like to use that in my solution. I would also like to include some sort of graph of sold prices if possible, taking inspiration from the graph in Zoopla – though instead of a graph of an estimate I would use the actual sold values.

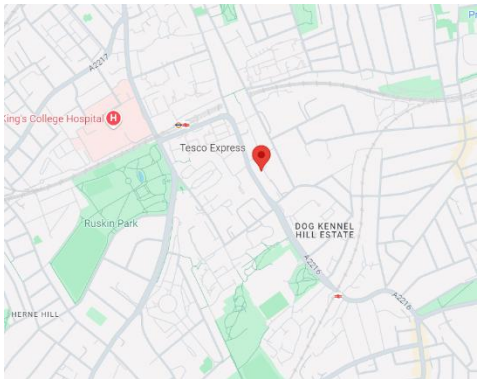
Justification of suitable approach

My solution should employ this use of a range of estimates, as it meets the stakeholders' needs of having context behind the prediction, giving them even more valuable information to help them when purchasing a property, or selling their property.

Google maps

Overview

This provides a pin-point location marker for the address, showing a region of the surrounding area, including any train stations.



Parts that I can apply to my solution

I would like to show the address of the property in question, linking in google maps.

Justification of solution

My solution will be heavily centred around the location of a property, so in order to help give as much information as possible to the user, it is sensible to represent this location in some way, and a map is the simplest method and easiest to understand. This allows stakeholders to clearly obtain the information they need.

On the market 'Instant Online House Valuation'

Current Step: Tell us about your property

Postcode*
SE5 8DB

Select address*
Select address

Property type*
Please choose...

Property description*
Please choose...
Select Conversion property if your home is converted, e.g. barn or church conversion

Property age*
Please choose...

Tenure*
Please choose...

Bedrooms*
Please choose...

Bathrooms*
Please choose...
Include WC, en suite and bathrooms in total bathroom count

Reception rooms*
Please choose...
Include study/office, living room and dining room

Overview

This particular website allows users to enter specific information about the property. They use drop down menus and required fields to ensure users enter all the information required by the program.

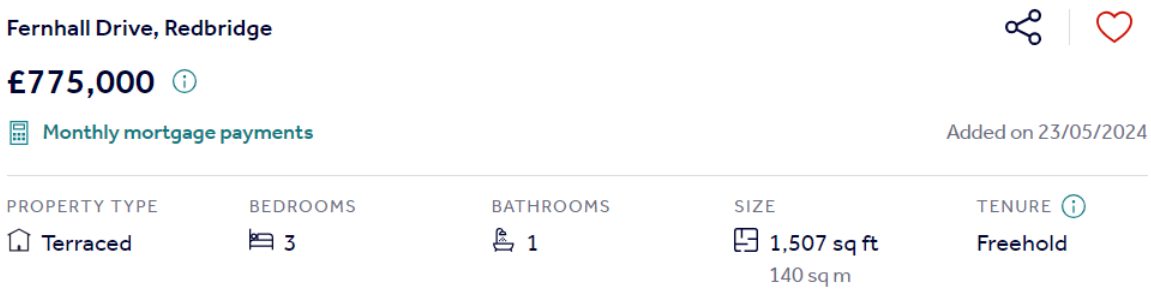
Parts that I can apply to my solution

I would like some sort of interface so that the user can enter some additional information – though maybe not as in depth as the one shown above. This data entry must also match the same format as the data that I will obtain through web scraping.

Justification of solution

This solution seems simple and appropriate, giving the user a drop-down menu to select the option that is the most applicable for them. This means that less validation will be required, as the inputs are already outlined for the user, making the program easier to use and suitable for the stakeholders.

<u>Rightmove property information display</u>



Overview

The essential information is displayed here and is really simple to understand.

Parts that I can apply to my solution

I could use the icons and a similar UI when comparing properties. It may be useful for the user to have a direct comparison between two different properties, where all the information is directly next to each other, so they can compare easily.

Justification of solution

To meet the stakeholder's needs, my program should display the information in a clear format, where the data is clearly matched to the corresponding category. This similar solution allows for this to happen, hence is necessary.

Summary

Overall, my solution will use elements from other similar solutions and integrate them together. It should be simple for the user to understand, and they should only see necessary information. The prediction should be accurate, giving a range of prices, and should be backed up by historical data – potentially displayed using a graphical form. Additionally, there should be some way of displaying the location of the property in context, such as an interactive map embedded into the UI.

Key features

Explanation of features

Machine learning model

My solution will use a machine learning model to generate a prediction. This is necessary, as without the model, a prediction on new data cannot be made, and my solution will not solve the problem.

Web scraping

This is needed to obtain the dataset, that will be used to train the machine learning model. Furthermore, this will be used to provide any further information regarding properties in the local area – one of the stakeholders' main priorities.

Organised Dataset

This is necessary to be able to train the model, clearly labelling the different factors which affect the final price. This is also required when inputting data for a prediction, as the model will only make an accurate prediction, if the data is inputted in the same format as it has been trained on. Furthermore, when displaying the data, I will need to be able to identify what the different data refers to – i.e. number of bedrooms, or postcode etc. Only then will my solution be able to present the data to the user in a clear, understandable format.

Interactive UI

This will be needed when the user wants to obtain a predicted price. They need to be able to adjust certain parameters such as square feet, or number of bedrooms. Hence, there must be some way that they can input the data for the model to make a prediction afterwards.

Map address integration

All of the stakeholders requested to have detailed information on the location of the properties. This can be best achieved through the use of a map, showing the local area of the property. The map needs to be integrated into the program.

Property information

Many of the stakeholders requested that information regarding the property be shown in a clear format, with some requesting some method of comparing two properties side-by-side. Hence their needs to be a way to obtain this information and present this to the user.

Limitations

Limited accuracy in less populated areas

One of the main limitations with my solution is the reliance on previously sold or listed properties. This means that the model will only be trained on this historical data. However, in areas where there is not a lot of activity in the property market, there will not be large amounts of data, potentially making the model less accurate in those areas.

Justification

This is a limitation as it means the program will have a bias towards different geographical areas, and will not be as accurate as possible, limiting the solution.

Time spent training the model

There is a vast amount of data available regarding properties that have been sold or are currently being listed. If I were to use all the data available from web scraping, although the model will be highly accurate, it will take a lot of time to train the model. Hence, I may have to sacrifice some accuracy in order to train the model quicker.

Justification

This is a limitation as it leads to a loss in accuracy of the solution, and hence will be not as effective in its core purpose of predicting a price.

Changes in websites affecting web scraping

Web scraping is highly specific to the particular website you web scrape from. Hence, if any changes are made to the website that I web scrape my data from, this could cause errors and require intervention and changing the code. My solution is heavily reliant on other websites and hence factors I cannot control.

Justification

This is a big limitation, as it means the program could potentially stop working if the source website were to be updated with changes. The model cannot therefore be retrained and hence will be out of date compared to the market. This will again lead to a loss of accuracy and hence is a limitation of the solution.

Potential limitations due to time restraints

These are features which are desirable but may not be included due to time constraints.

Graphical representation of historical data

Historical data such as the previously sold prices have been mentioned by some of the stakeholders and could be useful in providing more insight. This can potentially be presented graphically, with price over time – however, this is subject to time.

Justification

The graphical representation of data was not directly requested by the stakeholders, but it will most likely benefit the user, giving even more information about the prediction. However, as this was not a key point in the interviews, this may be excluded if there is a lack of time.

Direct comparison between two properties

This was mentioned by one of the stakeholders as a desirable feature. However, this strays away from the main purpose of a price predicting solution. Otherwise, this feature would allow the user to have a direct comparison between two properties, seeing the differences between the two, in areas such as size and location.

Justification

This could be potentially excluded from the program if there is a lack of time, as this is simply an additional feature. The solution should be able to look at similar properties, to meet the stakeholder's requirements, but a direct comparison is not strictly necessary, and may be omitted, creating a limitation to the solution.

Requirements

Stakeholder program requirements

Having spoken to the stakeholders, the following requirements have been expressed:

Requirements	**Explanation and justification**
1. Information on property	Shows relevant details regarding the property such as number of bedrooms, size, number of bathrooms etc. Necessary to give enough information for user to make a well-informed decision.
2. Location of property	Provides details on the property's location, including postcode and a map of the local area. Necessary to show user where the property is, and to also allow for similar properties to be shown.
3. Predicted price	This is the program's main purpose, and it should be able to estimate a price for a property given the information about it. Necessary as this is what the user is mainly looking to obtain from the program.
4. Range of estimates	There should be some context behind the prediction, potentially related to the lowest and highest prices of similar properties sold. Necessary, as this differentiates from other non-specific existing solutions
5. Simple UI	The information should be easy for the user to understand, and intuitive for the user to use. Necessary for user to feel they know what to do and use the solution.
6. Similar properties	There should be some way for the user to quickly see the similar properties to the one they have requested. Necessary to differentiate from other solutions.
7. Web Scraping	The program must source the data it uses for prediction using web scraping and formatting the data into a suitable dataset. This is needed to ensure all the information the model is trained on is up to date with the market.
8. Machine learning model	The program must use a machine learning model to calculate a prediction. This model must be as accurate as possible. Necessary to perform the key function of the program which is the prediction.
Functionality and robustness	The program should run without crashing or displaying errors. It should allow the user to enter inputs and validate these appropriately. Necessary to ensure the program is easy to use and runs as expected by the stakeholders.

Software requirements

Requirement	Description	Justification
Google Colab	This is a service that allows for Jupyter notebooks to be created, which allows for cell by cell running of the program.	This is required as the development of the program will be simpler and easier as code can be run line by line to check for any potential errors.
Visual Studio Code	This is an IDE that enables the running and execution of code, whilst providing essential libraries and access to debugging tools.	This is required as the code must be written in an IDE that allows for the editing and writing of code for this program.

Essential libraries

Requirement	Description	Justification
Beautiful Soup Library	This is a library which allows for the use of web scraping and extraction of data from HTML.	This is required as it means that the web scraping function will be able to return useful clean data, which can then be plugged straight into the machine learning model without any further processing.
Scikit Learn library	This is a library which contains the machine learning model, and the associated code to predict and train the model.	This is required as the program relies on a machine learning prediction, hence an instance of the model is required through the use of this library.
Pandas library	This is a library which allows for the use of panda data frames, which are a type of data structure. This is especially beneficial and suited to machine learning programs.	This is required to manipulate and process the data from the machine learning program, as well as allowing for easy cleaning of the data.

Hardware requirements

Requirement	Description	Justification
Standard peripherals	This is the use of basic input and output systems such as mouse, keyboard and monitor.	This is required as the user must be able to interact with the program in some way and select the options they require, whilst also being able to view the predicted price and other information the program outputs.

Success Criteria

There are several success criteria for this project. They will be referred to throughout the documentation by the number e.g. *Success criteria 7* or in reference to the requirement e.g. *Success criteria 2.3 (3rd criteria of Requirement 2)*

Functionality / Usability

1. Runs without crashing
2. Allows user to input parameters
3. Validates all inputs

Requirement 1: Information of property

4. Displays number of bedrooms
5. Displays size and type of property
6. Displays number of bathrooms
7. Displays the tenure
8. Displays the address

Requirement 2: Location of property

9. Displays postcode
10. Displays map of local area
11. Map is navigable

Requirement 3: Predicted price

12. Calculates and displays a predicted price using a machine learning model
13. The predicted price should be as accurate as possible

Requirement 4: Range of estimates

14. Displays lowest price of similar properties
15. Displays highest price of similar properties

Requirement 5: Simple UI

16. UI is displayed when program starts
17. UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties
18. User can clearly see which buttons to press in the UI

Requirement 6: Similar properties

19. Program identifies similar properties
20. The properties share similar characteristics

Requirement 7: Web scraping

21. Uses web scraping to extract information from property website
22. Extracts only necessary information
23. Stores necessary data into a suitable data structure

Requirement 8: Machine learning model

24. Trains a machine learning model
25. Tests a machine learning model
26. Compares multiple machine learning models

Justification

These particular success criteria refer to each of the requirements outlined in the Requirements section, and are therefore necessary for the program to fulfil the stakeholder's requirements. Each individual requirement has a number of success criteria referring to each requirement. These success criteria can be evidenced to measure success through any screenshots or videos of the program when running. If all success criteria are completed, then all requirements shall be met, hence justifying these particular success criteria. Furthermore, they also cover all core features and functions of the program as well as robustness, making them a good measure of success.

Design

Problem decomposition

The problem can be decomposed into a series of smaller problems that are suitable for computational solutions. These smaller problems include:

1. Retrieving data from website (Cleaning data, Processing data)
2. Create machine learning model
3. Prediction based on user input
4. Finding similar properties
5. Display information to user
6. Range of estimates
7. Location

Explanation and justification

Retrieving data from website

This involves using web scraping to extract information from websites such as 'rightmove.co.uk'. The data should be of suitable length and depth. This is required to train the model so that it can predict – the core aim of this program.

Links to success criteria 12 – 'Calculates and displays a predicted price using a machine learning model'.

Cleaning data

This involves removing unnecessary information and only keeping the data that we need. To do this, the relevant data must be identified and kept. This is an example of abstraction. This is necessary to ensure that the model is properly trained.

Links to success criteria 22 – 'Extracts only necessary information'.

Processing data

Once the data has been cleaned, it needs to be organised into some sort of array, with the columns and categories clearly labelled. The data can then be encoded so that data in text format can be converted into numbers, which could be better for the machine learning model to understand.

Create machine learning model

The dataset must then be fed into the machine learning model, which can be obtained through a library, incorporating a supervised learning model, where the training data is labelled. A model will then be generated, with potentially multiple different types of models, which will be evaluated to determine the most accurate. This is required to meet the key requirement of a price prediction.

Links to success criteria 13, 24 and 26– 'The predicted price should be as accurate as possible' and 'Trains a machine learning model' and 'Compares multiple machine learning models.'

Prediction based on user input

The program will allow users to enter specific details about the property in question and feed the data into the model so that a prediction can then be outputted. This is necessary for the solution to meet the requirement of an accurate prediction.

Links to success criteria 2 – 'Allows user to input parameters.'

Finding similar properties

This involves taking the user's input and finding similar properties. This will be done by searching algorithms to search through the dataset, finding similar properties in the same area as the one requested by the user. This is required to give the user more information about the prediction, including context behind the prediction, which is a key requirement outlined by the stakeholders.

Links to success criteria 19 and 20 – 'Program identifies similar properties' and 'The properties share similar characteristics.'

Display information to user

This involves creating a user interface that is interactable, allowing the user to modify the parameters of the property, whilst also seeing the predicted price. On top of this, they can navigate to find other similar properties and investigate them. This is required for the solution to be able to predict the price for a property that the user intends. The prediction must be specific to the property the user enters.

Links to success criteria 17 and 18 – 'UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties' and 'User can clearly see which buttons to press in the UI'

Range of estimates

This involves finding the range of estimates in which similar properties can be found and displaying the highest and lowest values. This is required to again provide real-world context behind the prediction. This will give the user additional information regarding the accuracy of the calculation.

Links to success criteria 14 and 15 – 'Displays lowest price of similar properties' and 'Displays highest price of similar properties'

Location

This involves presenting the location of the house in some manner to the user, potentially through a navigable map. This is necessary to show where the property is located, giving the user more information as required by the stakeholders.

Links to success criteria 10 and 11 'Displays map of local area' and 'Map is navigable'

Justification as a whole

I have broken my problem down into these seven stages for several reasons:

Order that program runs

The seven stages outline the main functions of the program, from the beginning to the end. The end stages cannot be completed and tested fully before the early stages. For example, the model cannot be trained if the data has not been obtained in the first place.

Order of development

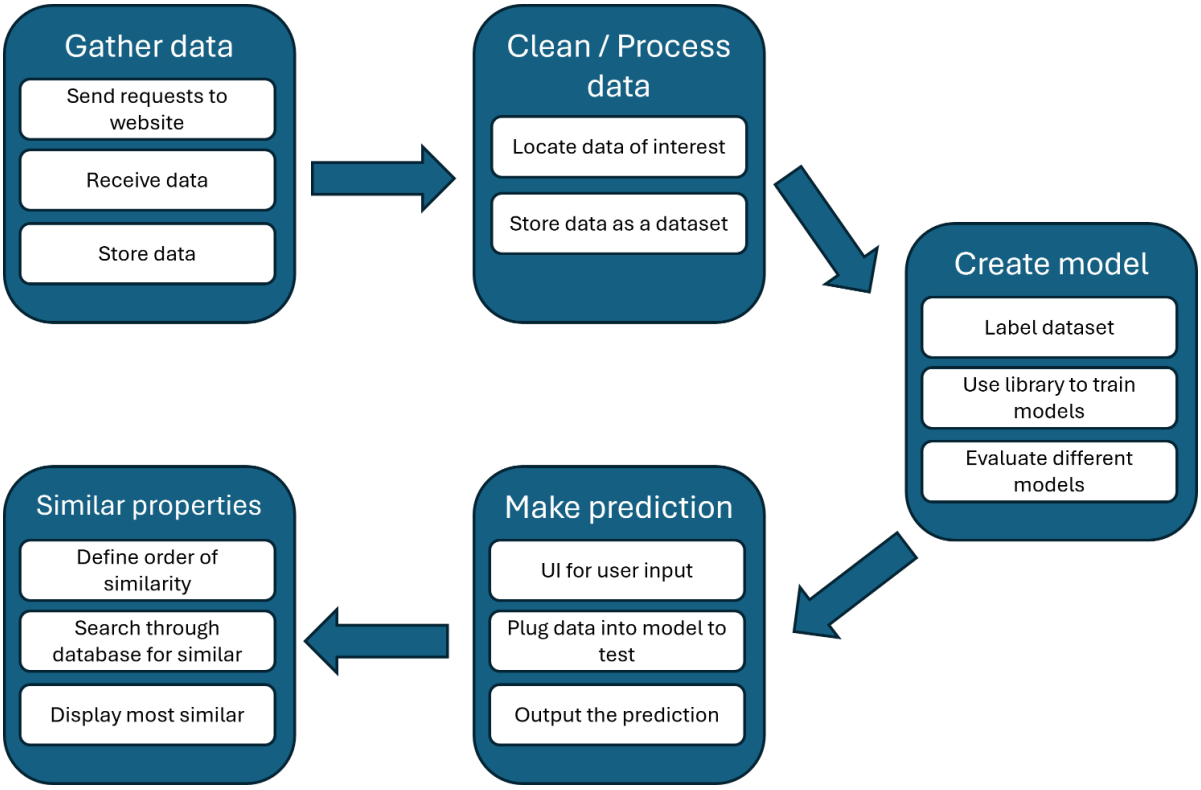
These seven stages will also most likely be the phases of development. The stages will allow me to work on each stage, finding all the potential problems before the next stage of development. This will allow me to develop the code ensuring that all potential problems have been tested and fixed.

Link to success criteria

These stages are all relevant to the program and link to specific success criteria and requirements. The sub-problems however cover enough scope to complete the entire problem.

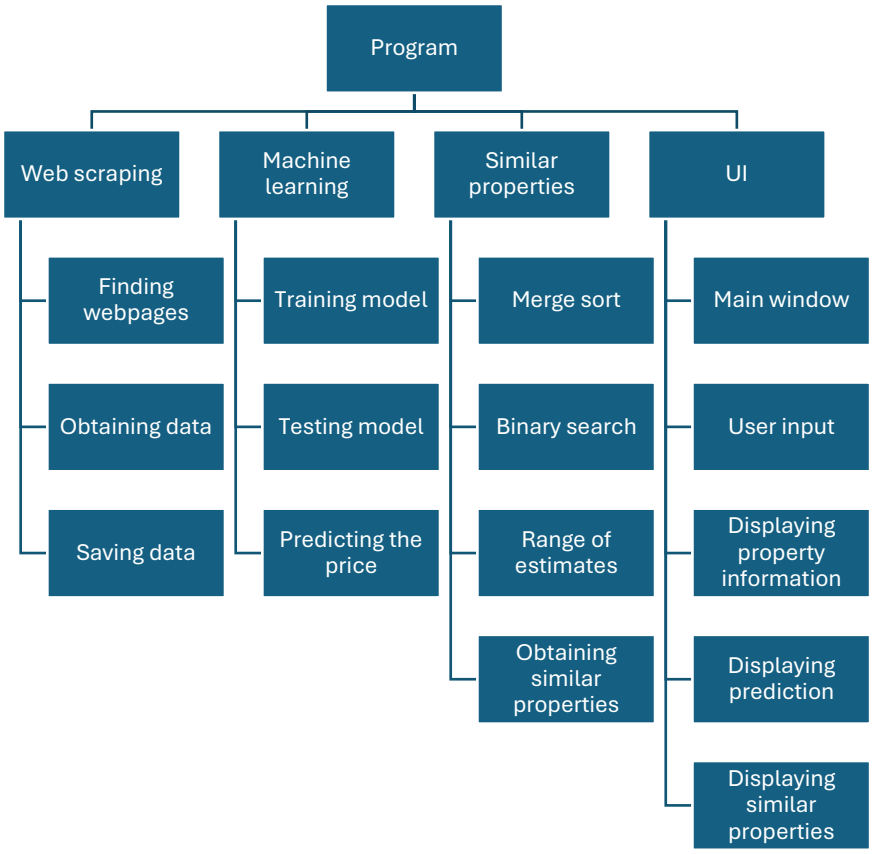
Diagrams

The decomposition of my program can be seen in these diagrams:



The above diagram shows the different overall functions that are required for this program. Within each overall function, there are a couple of subroutines that are required to achieve the overall aim. This diagram also shows the direction of the program.

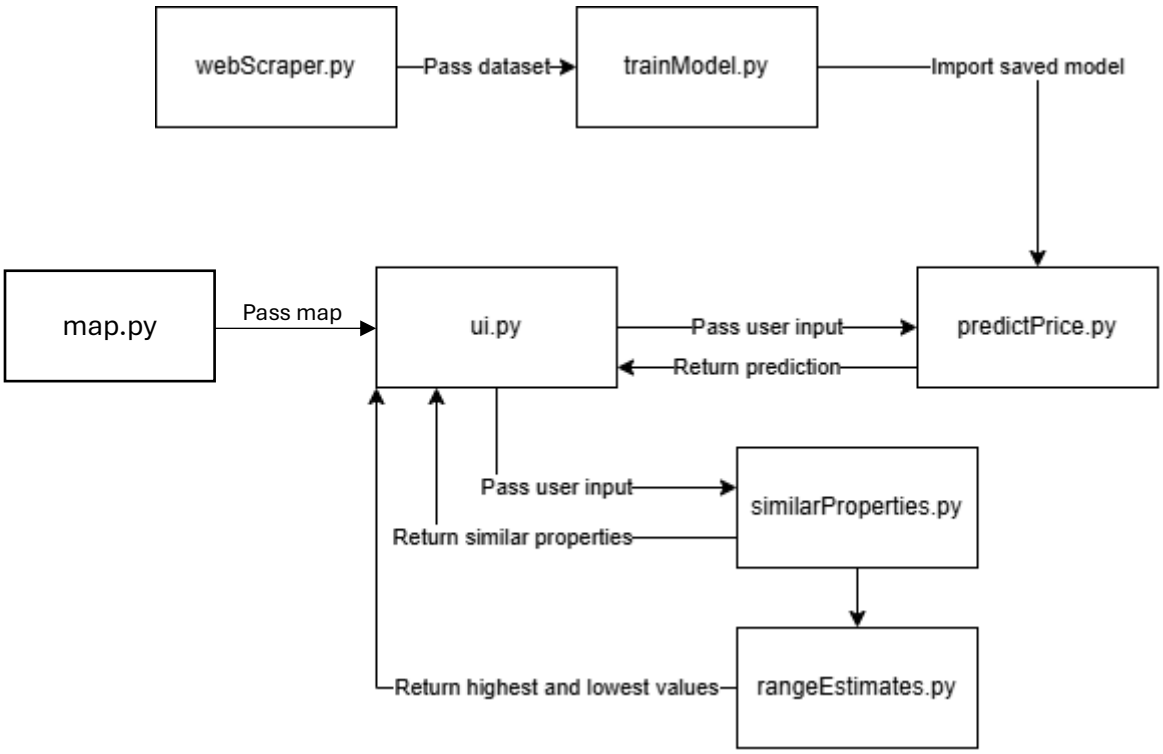
Problem decomposition diagram:



This diagram shows the different aspects of the code, and the hierarchy of the problem in more detail. This particular diagram highlights the sub-functions mentioned earlier in greater depth, to give clarity on the structure of the solution.

File structure

As my decomposition, I will use the following file structure. The program has been abstracted into the necessary files, with each file achieving a specific and dedicated purpose within the program.



Algorithms

Description of algorithms

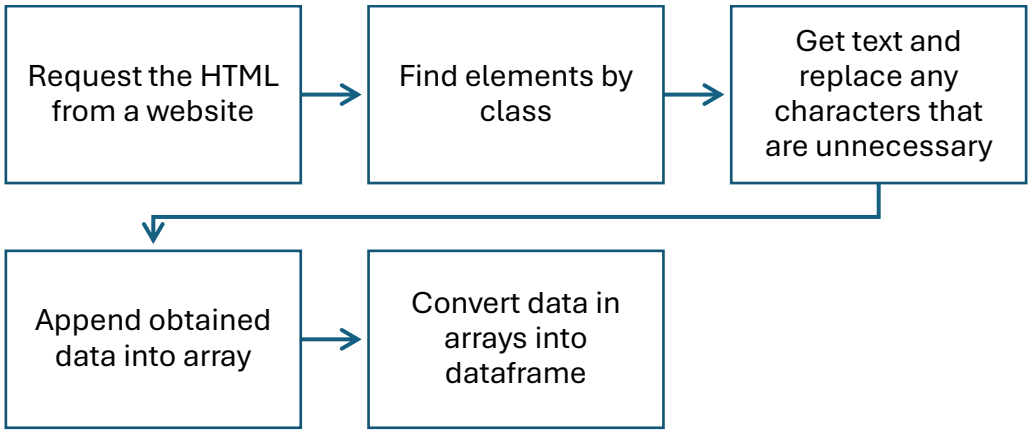
There are multiple algorithms that my solution will require, including:

- Web scraping algorithm – including algorithm to isolate and abstract data
- Machine Learning algorithm – including linear regression and random forest
- Similar properties search algorithm – including merge sort and binary search

Web scraping algorithm:

The aim of this algorithm is to scrape data from a website, keeping only the necessary information.

This algorithm can be broken into the following steps:



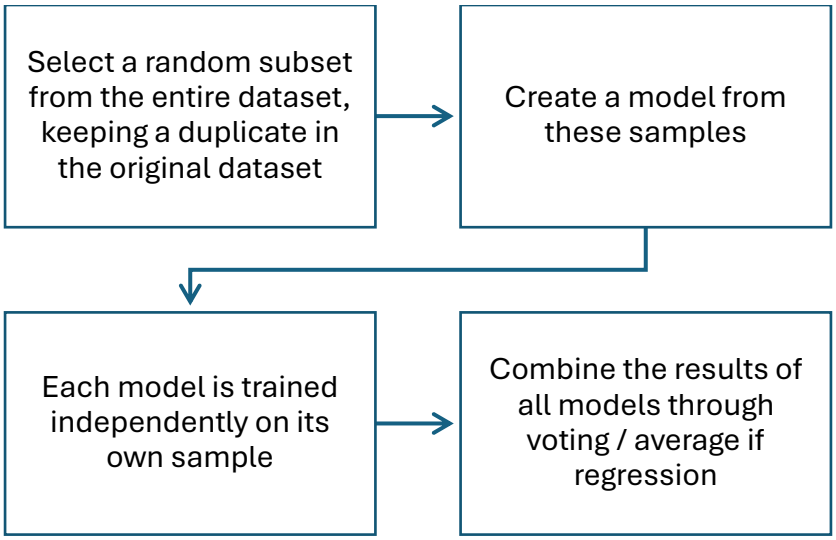
Links to Success criteria 21 and 22 – 'Uses web scraping to extract information from property website' and 'Extracts only necessary information'.

Machine learning algorithm 1: Random Forest:

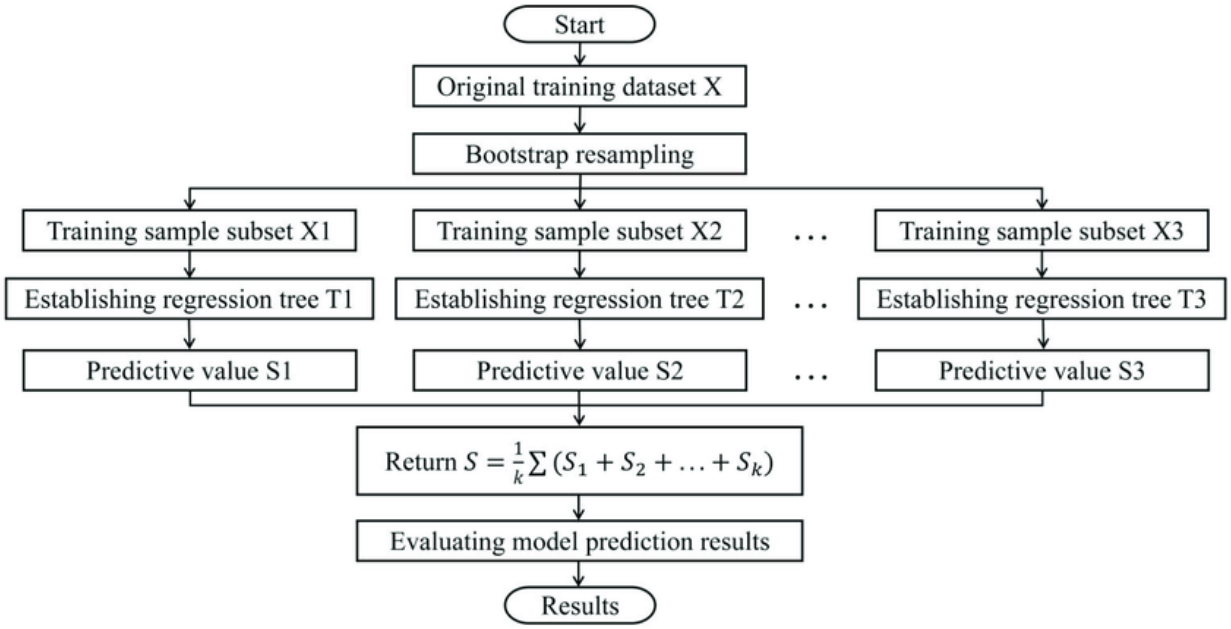
This algorithm will train machine learning models, using the data obtained using the web scraping algorithms.

One model is called Random Forest and is an ensemble learning method for both classification and regression. It uses multiple decision trees and takes an average of the outputs to reach a prediction. The algorithm uses an ensemble method called bagging, where a random sample of data is selected and replaced and trained independently. A parameter to adjust would be the number of different decision trees we use to train our model.

An overview of how the model works can be seen in this diagram:



Here is a flowchart of the algorithm in more detail:



Forecast of Hourly Airport Visibility Based on Artificial Intelligence Methods 2022 (Ding, Zhang, Wang, Xue)

Links to success criteria 24 and 25 – 'Trains a machine learning model' and 'Tests a machine learning model.'

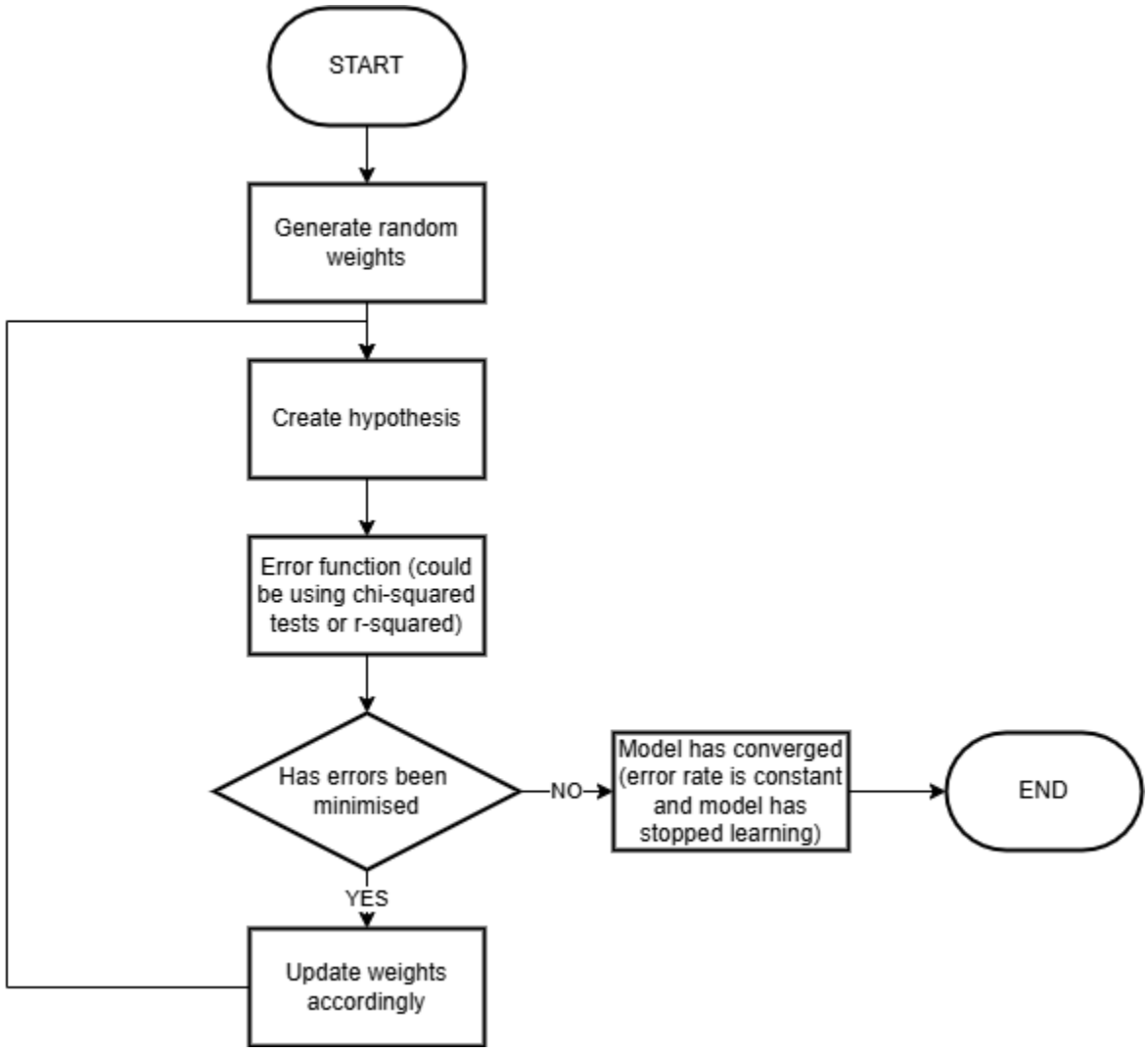
Machine learning model 2: Linear Regression

For this program, I will also use a linear regression model. Machine learning models can be split into either regression or classification models, and for this project, a regression model will be used as a price needs to be calculated rather than a classification.

I have chosen linear regression as it is one of the simplest types of regression models. The model calculates a mathematical function using a linear model in the form y = ax + b. The model also uses and error function and attempts to minimise the error of the model by changing the parameters and relationship between the different factors.

Furthermore, a linear regression model is a type of supervised machine learning. Supervised learning uses labelled datasets to train algorithms that predict outcomes. In linear regression, the relationship between the independent variables (number of bedrooms, bathrooms, location etc.) and the dependent variable (price of the house), is calculated to form a model. This model can then be used to predict the price of a new house. However, as the regression is strictly linear, accuracy can be lost.

I made a flowchart for linear regression as can be seen here:



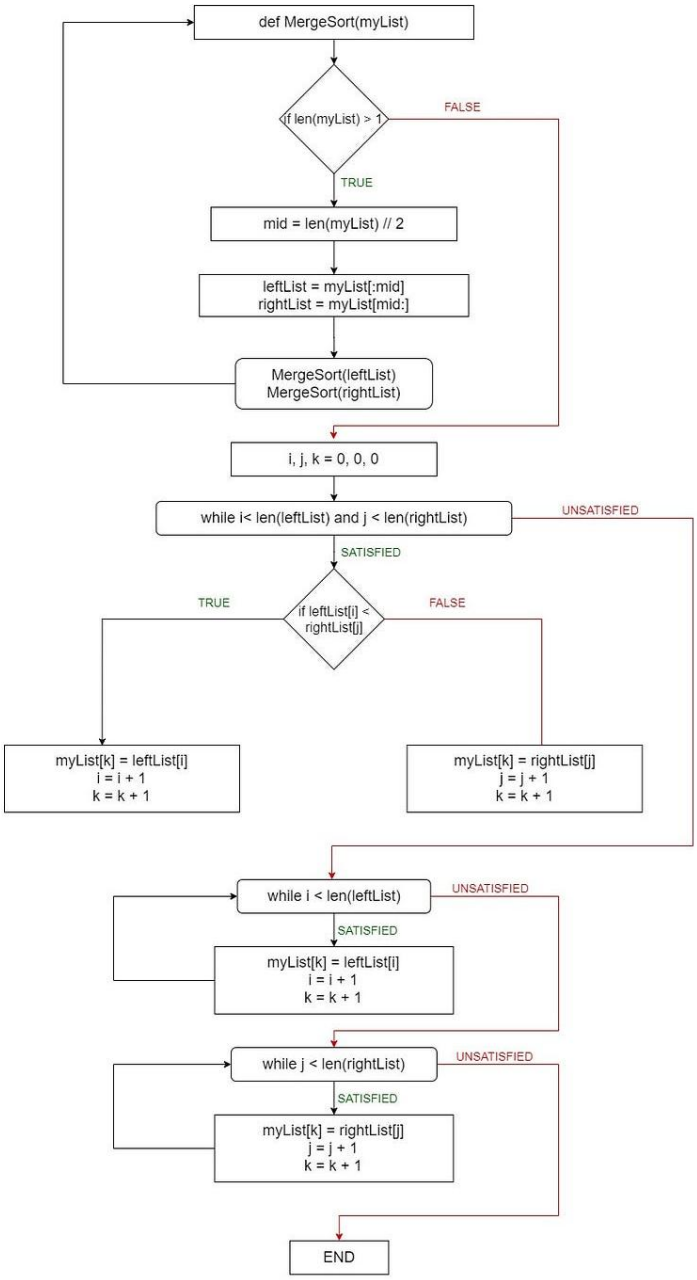
Links to success criteria 24 and 25 – 'Trains a machine learning model' and 'Tests a machine learning model.'

Sorting algorithm for similar properties:

To be able to quickly search for a specific location within the dataset I will use a merge sort algorithm to ensure that the dataset is sorted by postcode. This sorting algorithm may also be needed for searching for house type and other factors.

I will use merge sort for this sorting algorithm, as it has a time complexity of O(n logn), hence is more efficient when dealing with large datasets, such as the one in this project.

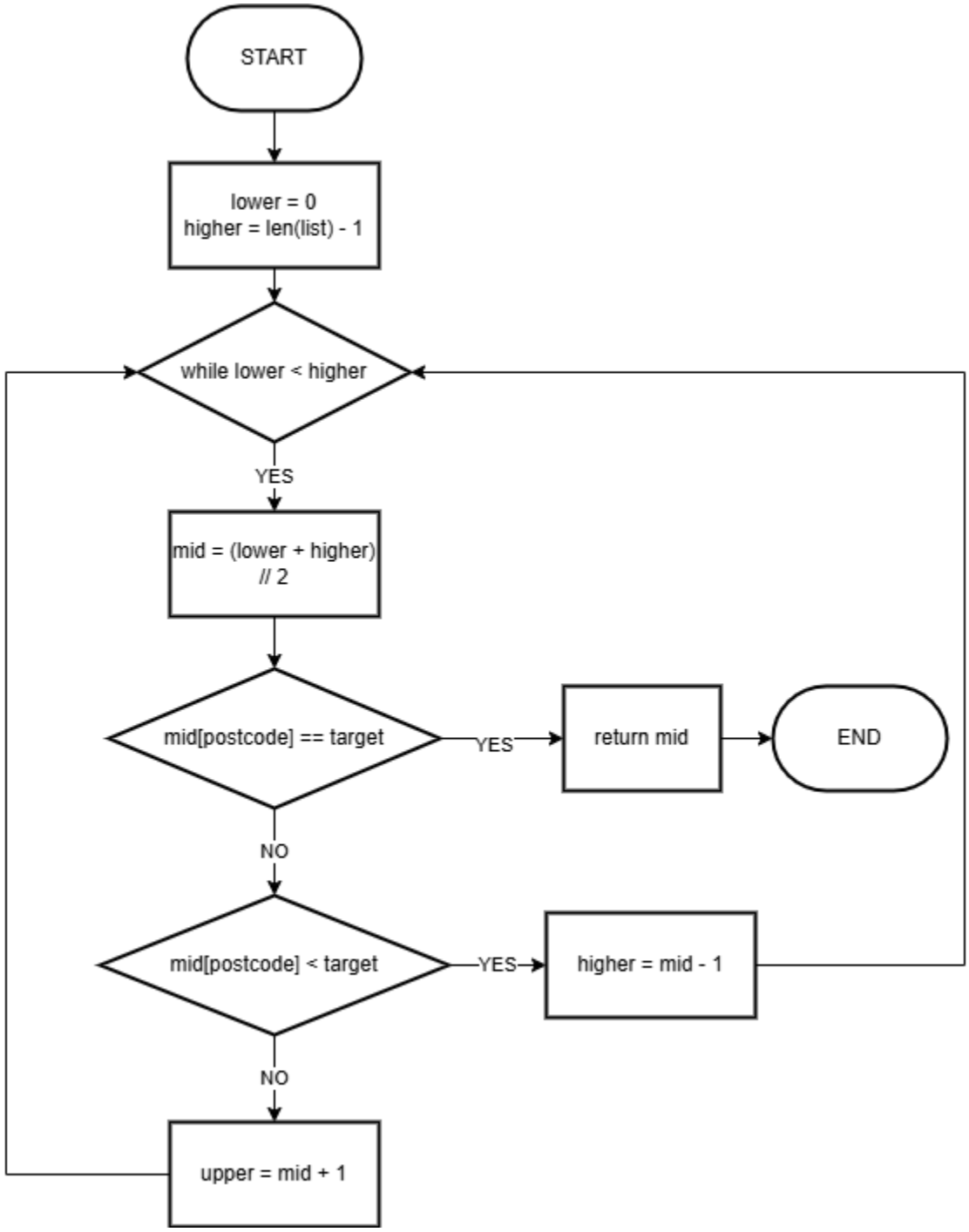
Here is the flowchart for a standard merge sort:



5 Sorting algorithms every programmer should know (2020, Sathiyabal)

Binary Search for similar properties:

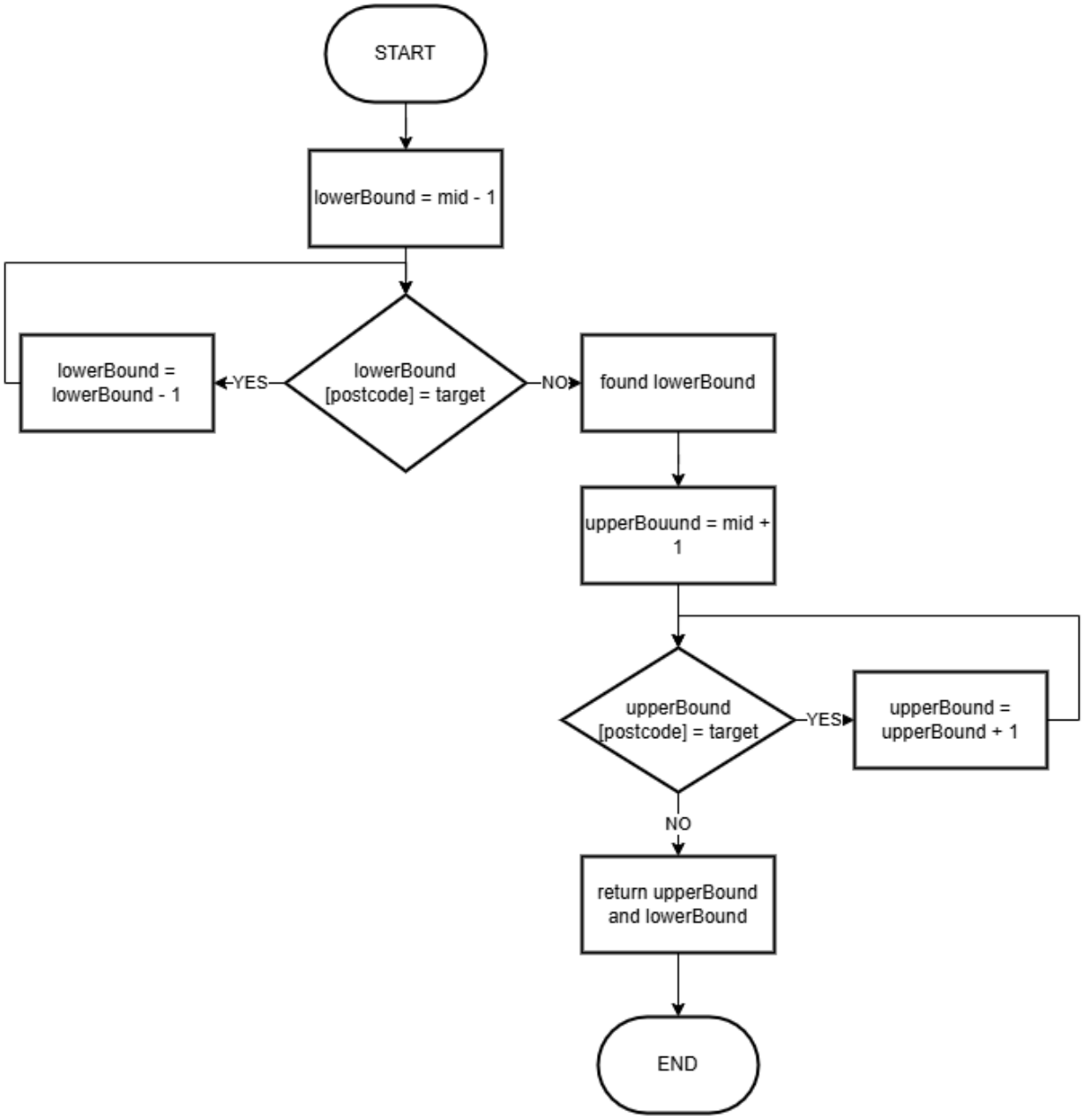
This algorithm is needed to search for through the sorted list of properties. By sorting and then searching the binary search will be able to search in a quicker time compared to linear search, in a time complexity of O(log n). The flowchart for binary search can be seen here:



Linear search Contiguous postcode search:

When using binary search to find a property with the same postcode, only one postcode will be returned. However, my program requires all postcodes that are the same sector to be returned, rather than just one property. Hence an algorithm needs to find the lower and upper bound in which the postcodes match, for the whole range to be returned. I will use linear search for this.

The flowchart for this algorithm can be seen here:



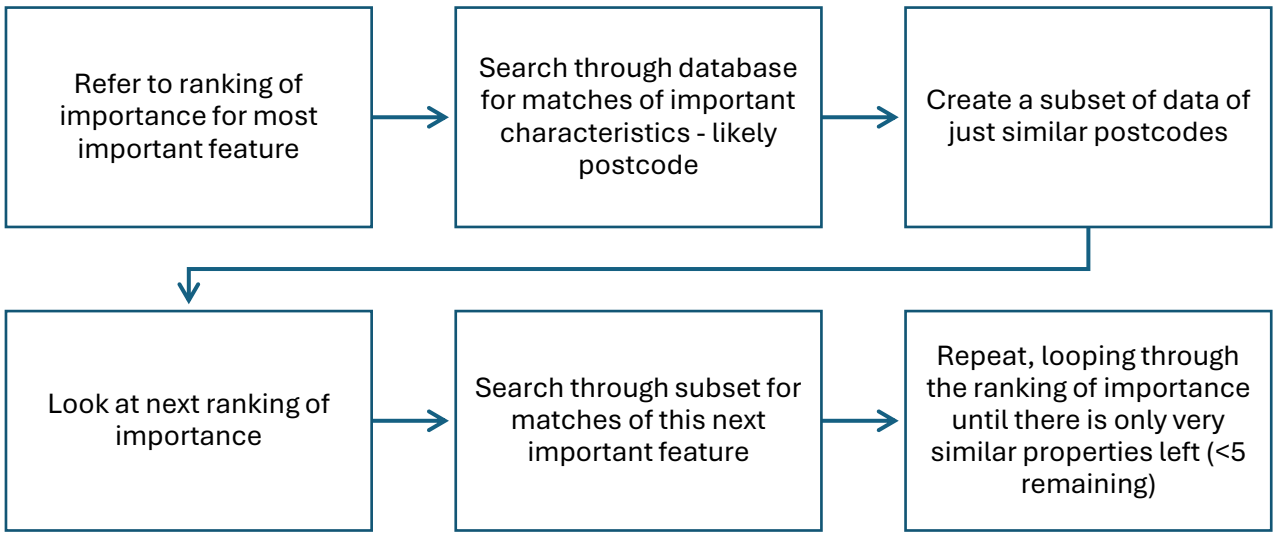
Similar properties search:

This algorithm is needed to search for similar properties to the one inputted by the user. To rank the similarity of properties, I will use a hierarchy of importance when determining how similar properties are. Then, using this, the algorithm will be able to determine if there is a similar property and how similar they are.

Alternatively, an algorithm could be used which gives different weighting to the different properties, and the property with the 'highest total score of similarity' can be inferred to be the most similar.

Links to success criteria 19 and 20 – 'Program identifies similar properties' and 'The properties share similar characteristics.'

An overview of the first algorithm can be seen in this diagram:



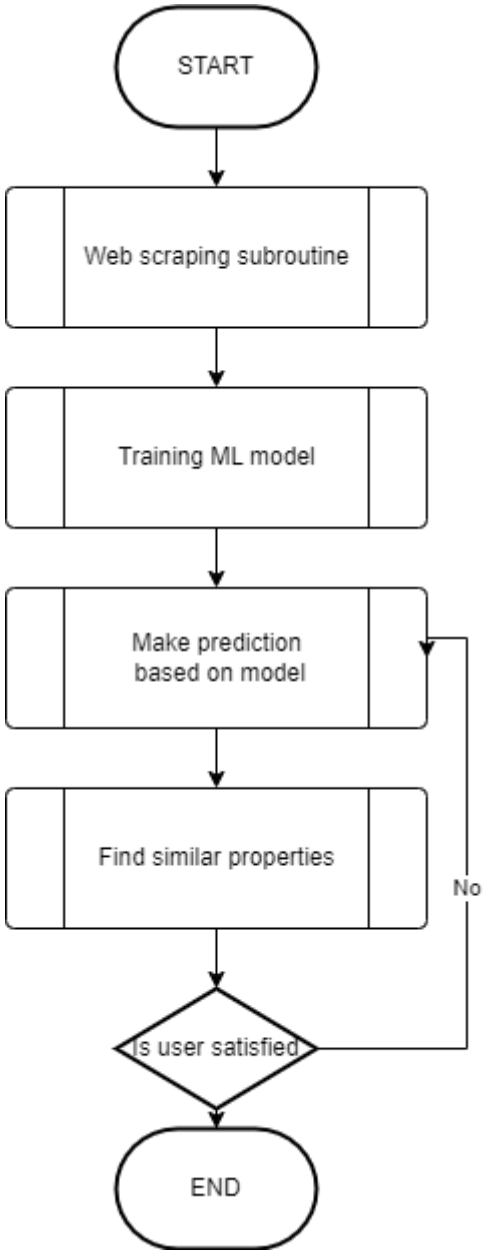
Justification

These are the core algorithms that this solution requires. Each of these are algorithms because they are a step-by-step instruction detailing how to achieve certain parts of the decomposed problem. Without these algorithms, the program will not meet any of the requirements set out by the stakeholders, and to an extent will not solve the problem. The searching and sorting algorithms are needed to reduce the time spent processing the data when searching for similar properties. I will use a binary search method as well as linear search to extract the contiguous values and hence will require the use of a sorting algorithm.

The combination of these algorithms complete all necessary requirements, and hence form a complete solution to the problem.

Program Flowchart

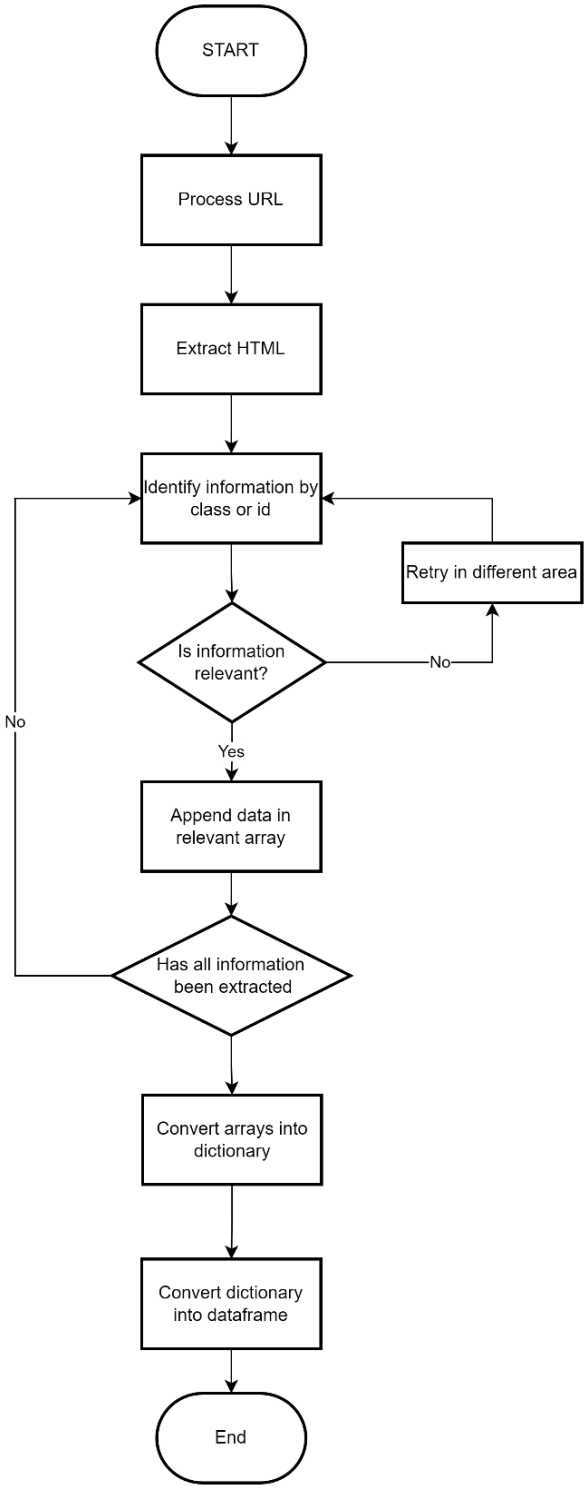
Here is the flowchart of the overall program.



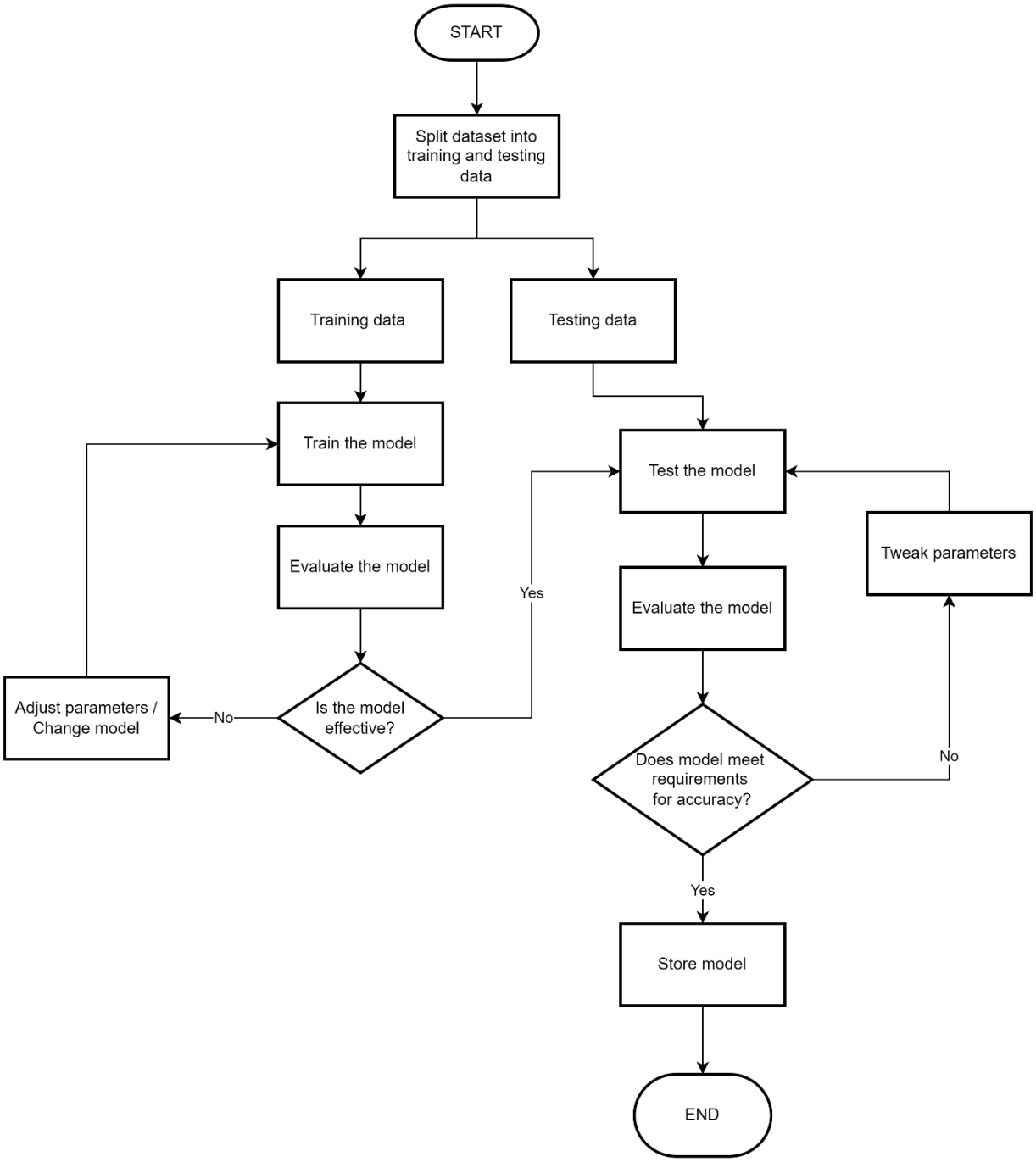
Subroutines

Within the overall program flowchart, each subroutine can be seen as follows:

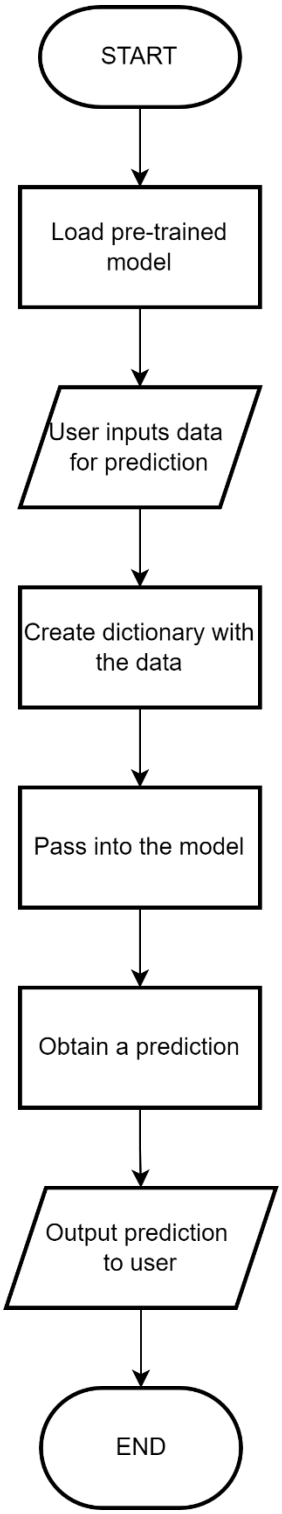
Web scraping subroutine:



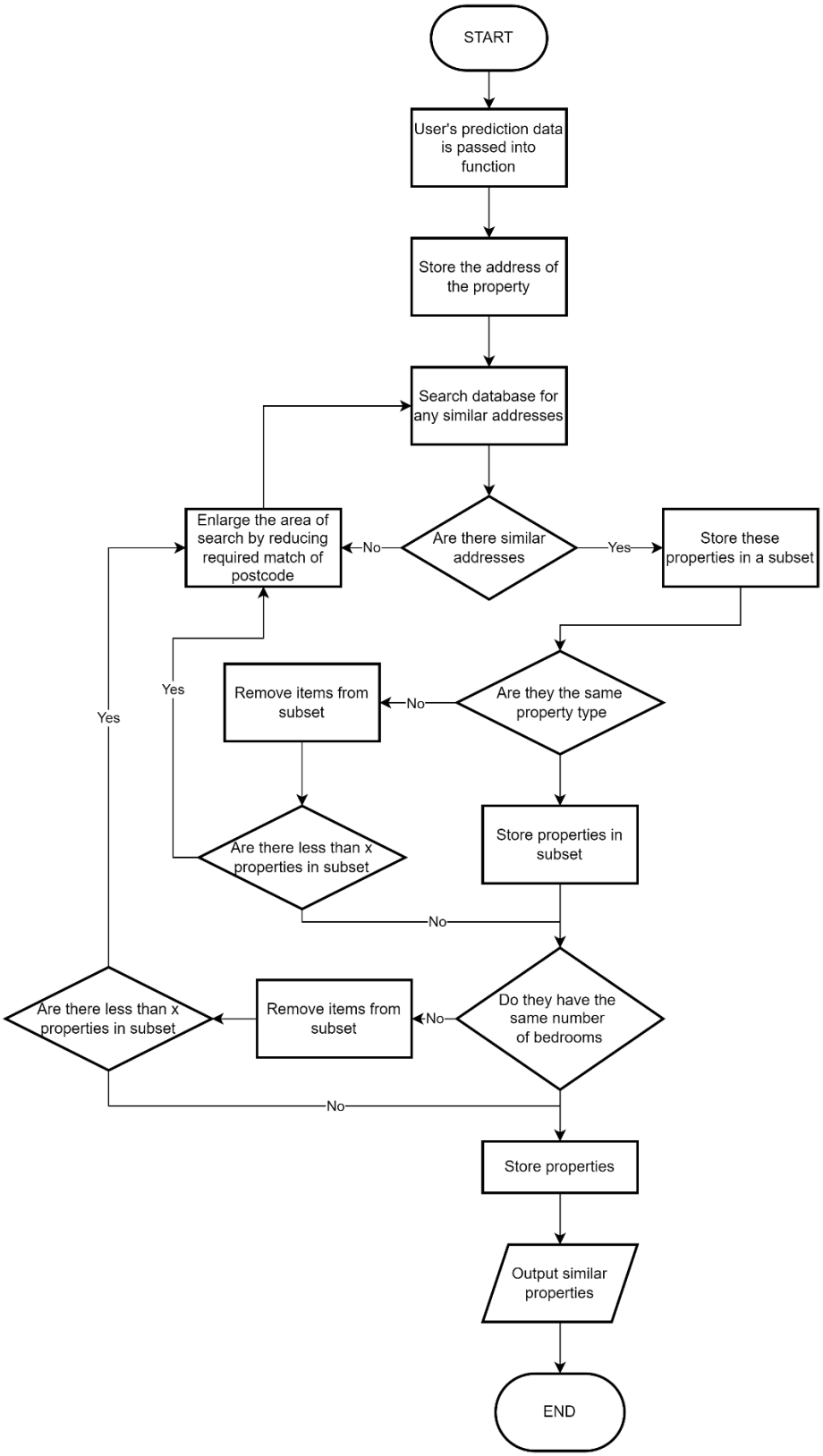
Machine learning subroutine:



Prediction algorithm:



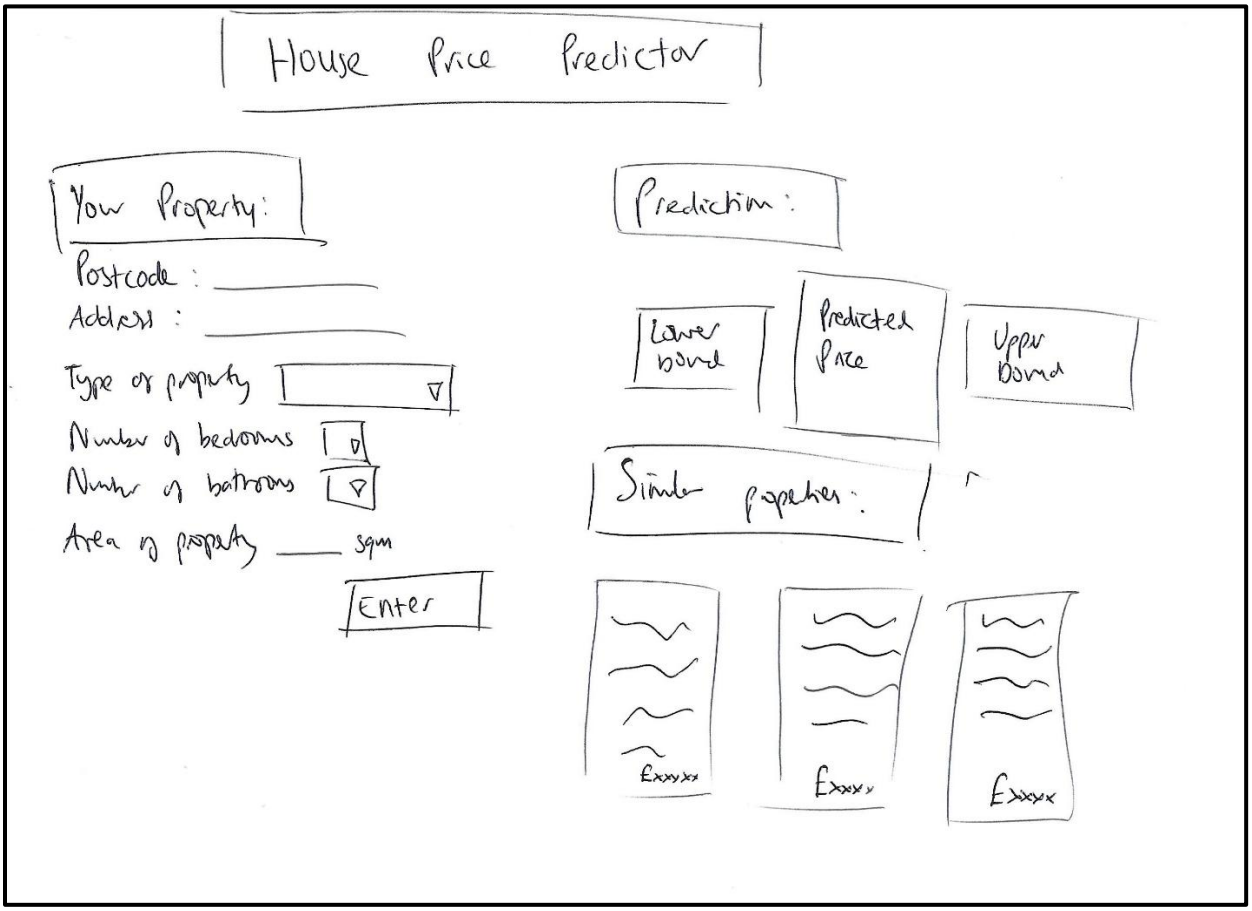
Similar properties algorithm:



UI design

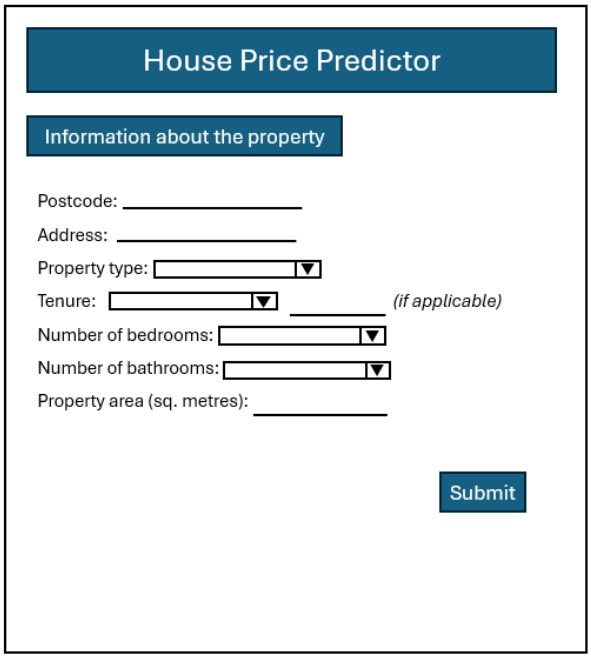
Concept

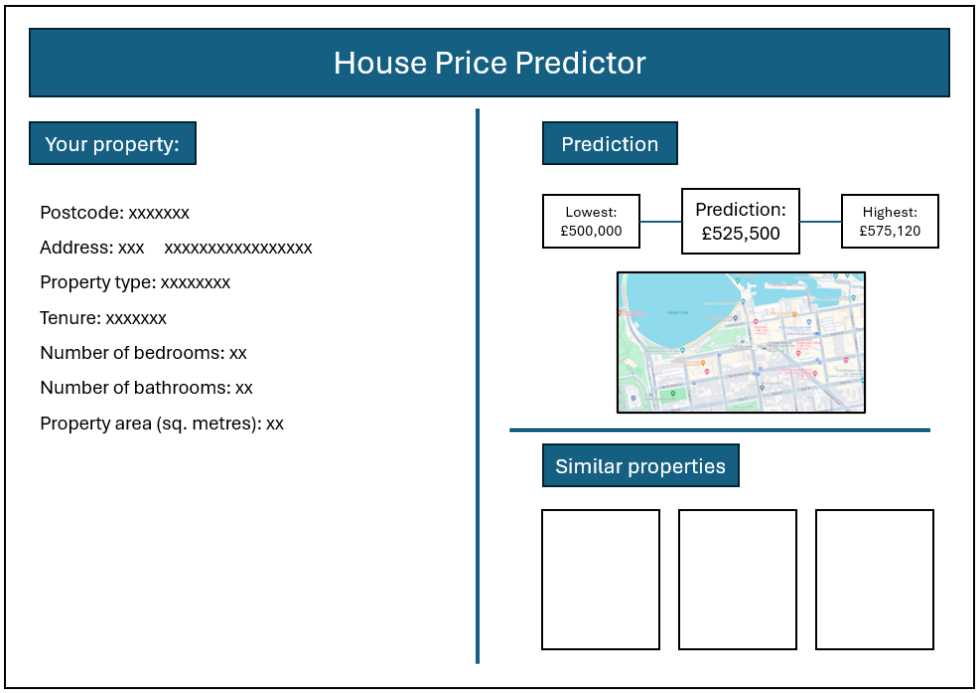
Here I have shown the initial concept for the UI. I wanted everything to be clearly laid out so that the user will be able to see everything that they need to see. It should be very intuitive for them to use.



Visualisation

Here is the visualisation of the UI. It displays all the functionality required of the solution in the screen and allows the user to enter their information and see the prediction side by side. This could allow them to make adjustments and see the update of the price. The similar properties is also displayed on the screen allowing the user to make quick comparisons.





Justification

This UI involves a simple colour scheme, where the user can also read and understand the data with ease. The UI also allows the user to enter their data, with text boxes for when text is required, and a drop-down menu when there are limited options, for example property type. This means validation will not be needed for the drop-down menu inputs and gives the user clarity on what to submit.

The prediction values are displayed with the actual prediction, and the ranges of prices in the real world. This can give the user more context behind their prediction, and overall, more information to help them make their decisions.

Links to success criteria 17 and 18 – 'UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties' and 'User can clearly see which buttons to press in the UI'.

Breakdown of UI design

Title box

This is a coloured box which clearly tells the user the function of the program and should be clearly visible. The text size is larger here to clearly show it is the title.

Subtitles of sections

Each section is titled in a coloured box which defines each section, making it clear which section the user should be looking at. *Links to success criteria 15*.

Entry of data

The section on the left is for the user to enter the data about the property. It clearly defines all the information required for the program, and the type of data required by the program. This is all the user needs to fill in for the program to make the prediction. Some changes to this particular UI design may be made, such as increasing/reducing parameters for user to enter. However, the concept of the design will remain the same.

Map

This UI allows space for the map of the location of the property to be shown, so that the user can see easily where the property is located, and potentially any similar properties.

Box to submit

This clearly allows the user to signify to the program when they are happy with the data they have entered and want the program to make the prediction. This should be very intuitive and allow the user to achieve functionality when using the program.

Size

The application should fill the size of the complete desktop window. This should make it large enough for the user to read all the information clearly without needing to enlarge or struggle with seeing which buttons to press.

Links to success criteria 17 and 18 – 'UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties' and 'User can clearly see which buttons to press in the UI'.

Outlining method of development

As this program is coded in python, upon research there are two main options for creating a UI:

- Tkinter
- PyQt

Researching the two, I found that there were disadvantages and advantages to both, and they are more suitable to certain applications:

Aspect / Feature	Tkinter	PyQt5
UI appearance	Basic, slightly outdated	Modern and professional
Widgets & Controls	Limited and simple (buttons, labels, etc)	Extensive (advanced tables, web views, multimedia etc)
Layout Management	Simple with functions to align different sections	Very powerful layout management with dedicated functions
Threading	Limited and needs to be imported	Built-in support
Cross-platform support	Works on Windows, macOS and Linux	Works on Windows, macOS and Linux
Ease of use	Easier and built into Python	Quite complex and hard to learn
Suited to	Small lightweight GUI apps	Large-scale professional applications

For this particular program, I would like to use both libraries to create the UI, due to their strengths in different areas. The Tkinter library can be used to create the simple UI needed to output a prediction and take the user's inputs, whilst the PyQt5 library can be used for the more complex tasks such as generating the interactive map.

Justification

The UI needs to be developed in this manner to allow for a simple development and UI to be created, whilst also maintaining all key parts of the program, for example displaying the map.

Stakeholder input

Having completed UI designs, I have sent them to the stakeholders, and checked for feedback with them, to ensure they are all completely happy with the designs. Any changes can be implemented now before development.

Here is the email I sent them:

"Hi,

We are at the point of designing the application and would very much like your input at this stage. Please find attached to this email the diagrams of how the software will look. All the buttons should be clearly visible, and all functionalities should be achieved through this UI. Please let me know your thoughts and any changes that could be made to improve it.

Thanks,

Victor"

Replies

Andrew - Stakeholder 1: Property buyers

"Hi Victor,

It all looks pretty decent. I can clearly see where to enter the information and where the machine outputs the predicted value. As long as I will be able to see information in the similar properties section, I'm happy.

Andrew"

I then addressed his concern and replied:

"Hi Andrew,

Yes, there will be information about the similar properties, such as location, type of property etc. Many thanks for your feedback.

Victor"

Bianca – Stakeholder 2: Property sellers

"Hi Victor,

Thanks for reaching out. I'm pretty pleased with the way the application looks so far. It is very easy to understand and readable."

Colin – Stakeholder 3: Estate agents

"Hi Victor,

Thanks for letting me see the designs you have created so far. They look pretty good, and I can clearly see all the information that I will need to see. This will help form part of my judgements on certain properties, and will be insightful nonetheless. The colour scheme looks and feels professional and clean. We look forward to seeing it.

Kind regards,

Colin"

Dave – Stakeholder 4: Developers

"Hi Victor,

This design looks pretty easy to understand and simple to use. I think it'll be good enough for me to use and navigate, to research any potential prices I will need when looking for land to develop and potential within an area."

Conclusions:

All of the stakeholders seem satisfied with the UI design and have not raised any potential problems they have with it. Hence, I will continue with this design into the development.

Inputs and outputs

There are limited inputs and outputs for my solution, as the problem does not require it.

Input/output chain of predicted price:

Section	Input	Process	Output
Predicted price	Information about the property	Plug user input into machine learning model to make a prediction	Predicted price is outputted to user
Similar properties	Information about the property	Search through database to find similar properties based on features	Similar properties are shown to the user
Navigable map	User uses cursor to move map	Uses Google maps API to process the change and show relevant images	The map moves accordingly to the user's request

Key variables

Variable name	Data type	How it is to be used and justification
userPostcode	String	Stores the user's input of the property postcode that they want to predict from. Important for prediction as it gives the location
userStreet	String	Stores the user's input of the street of the property. This will not be used for prediction but rather to display the user's full address when outputting the result
userPropertyType	String	This will be a drop down menu, which will allow the user to select the option that they require.
userBedrooms	Integer	Stores the number of bedrooms in the user's property. This will be used for the prediction, as it is a key factor.
userBathrooms	Integer	Stores the number of bathrooms in the user's property. This will also be used for the prediction, as it is a key factor.
userArea	Integer	Stores the size of the property, and will be used for the prediction.
userInfo	Dictionary	This is the dictionary that will store all the information that the user wants to predict from. A dictionary can easily be used for the prediction
df	Data Frame	This will be the variable that holds the array for all the data that the model will be trained on. Using the panda library, the data frame is the most effective for machine learning.
X_train	Data Frame	This variable holds all the factors that lead to the price for the training dataset. The factors are separated from the prices to allow the model to use the factors to train for the corresponding price.
X_test	Data Frame	This variable holds all the factors that lead to the price for the testing dataset.
y_train	Data Frame	This variable holds the corresponding prices for each property in the training dataset.
y_test	Data Frame	This variable holds the corresponding prices for each property in the testing dataset.

Other variables

Variable name	Data type	How it is to be used and justification
y_lr_train_pred	Array	This is an array that holds the predicted prices for the training dataset, using the linear regression model. This is necessary to evaluate the model's prediction against the true values.
y_lr_test_pred	Array	Similarly, this is an array that holds the predicted prices for the testing dataset using the linear regression model.
lr_train_rmse	Float	This is the value of the root mean squared error statistic. It shows the average error distance for each prediction. The lower this value, the closer the prediction to the actual value.
lr_train_rsquared	Float	This is the statistic which shows how good a best fit the training predicted prices are compared to the actual prices. Again used for evaluation and comparisons between different models.
lr_test_rmse	Float	Similar to the training root mean squared error, this is the same statistic for the testing dataset and will be used for evaluation.
lr_test_rsquared	Float	Similar to the training r squared, this is the same statistic but for the testing dataset and will be used for evaluation of models.

These variables would be repeated for any other models that will be used.

Validation

To ensure my solution will be robust, the inputs will all be validated to ensure that they are the correct data type.

Text inputs

A core function of the program is for the user to input their property information such as postcode and address. This has to be done via a text box input. The postcode must be a string and hence it is hard to validate via data type. Instead, we could validate that it follows the same form as a normal postcode. If there are no matches to a suitable level of accuracy, I can reject the postcode and ask the user to type it again.

The address input is not as critical to the solution, hence this may not need to be validated, and instead may just be displayed back to the user. This can be changed however if the stakeholders prefer a valid address as well as the postcode.

Drop-down list

The drop-down list will be used for the user to select specific options such as property type or number of bedrooms. It is easier for this to be a drop-down list, so validation is not necessarily required for these inputs. However, it may be the case that the user's intended inputs are not an option, hence they may need to type in their inputs. This needs to be checked with stakeholders to see what their requirements for the program will be.

User confirmation

Once the user has entered their details, a confirmation box will pop up, where the user would be able to choose to either edit their details or confirm them. These options will be given to the user in buttons, giving robustness to the program. The confirmation button means that the user is sure that they want to enter the details they have given.

Buttons

The program will rely heavily on the use of buttons for the user to choose and select what they want. This adds robustness to the solution, as it means there can be no accidental error by the user, and requires no further validation.

Iterative testing method

To ensure the program works as is required, the inputs should be tested with normal, boundary and erroneous data. The test table below has been constructed so that the **order of development matches the order of testing**.

When developing the functions to handle the user inputs, all data inputs should be tested, firstly with normal data to ensure that the function works as intended. Any error messages should be opened via the program console, and if needed for debug purposes, output the variables and their contents.

When development is finished, the function can be tested again with normal, boundary and erroneous data.

Function test table

This table will be used to test the main functions of the program, to ensure that it runs as intended.

Phase 1: Web scraping

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Web scraping (Requirement 7)	Website address	Fetch html from specified website using web scraping	Returns full html code for specified website	This is needed to ensure that my program is able to connect with the web server, and request and extract the HTML. *Links to success criteria 21 – 'Uses web scraping to extract information from property website'*
2	Web scraping (Requirement 7)	Class/id in given html	Fetch html from a specific class or id from given website using web scraping	Returns html code for specific section of the website	This is needed to prove that the web scraping can isolate certain parts of the html structure and return only this intended information. This is an example of abstraction. *Links to success criteria 22 - 'Extracts only necessary information'*
3	Web scraping (Requirement 7)	Specific class / id found in html	Fetch specific data from within the html	Returns the actual data values for the specified item	This is needed to prove that the web scraping can return usable data values, without the other html code, allowing this to be added into a database. *Links to Success criteria 22 – 'Extracts only necessary information'*

4	Web scraping (Requirement 7)	Multiple classes / id's found in html	Fetch multiple specified data from within the html	Returns the data values for all the items specified	This is needed to test that I can web scrape multiple items, as this will be needed in the full solution to obtain all the properties. *Links to Success criteria 22 – 'Extracts only necessary information'*
5	Web scraping (Requirement 7)	Singular property found in multiple web pages	Fetch a singular characteristic / property of multiple web pages. For example, address in 3 different example pages	Returns the specified data value found in all different web pages	This is needed to test that I can web scrape from multiple web pages in the same site, as in the final solution I will need to scrape from many different properties in a short space of time. *Links to success criteria 21 – 'Uses web scraping to extract information from property website'*
6	Web scraping (Requirement 7)	Multiple properties found in multiple instances on the web site	Fetch multiple sets of data from multiple web pages. For example, address and postcode in 3 different example pages	Returns all the specified data found in all the different web pages	This is needed to test that the program is able to web scrape multiple sets of information from multiple different properties. This is because the final solution will need to obtain many different sets of information, such as postcode, address, number of bedrooms etc. *Links to success criteria 21 – 'Uses web scraping to extract information from property website.'*
7	Web scraping (Requirement 7)	Multiple arrays of data	Store this information into a dataset, using panda library	Creates a dataset with all the information in the correct format	This is needed to test that once the program has web scraped information, it is able to store this all into a dataset, allowing it to be fed into a machine learning model. *Links to success criteria 23 – 'Stores necessary data into a suitable data structure.'*

Phase 2: Machine learning model

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Machine learning model (Requirement 8)	Dataset, with the column for price identified	Split the data into the x_values and the y_value	The program should be able to split the dataset into two different variables: the price, and all the other components	This is needed to test that we can split the dataset into the two components that will then be fed into the machine learning model. *Links to success criteria 24 – 'Trains a machine learning model.'*
2	Machine learning model (Requirement 8)	Dataset, with the given percentage split	Split the data into training and testing data	The program should split this data into the training dataset and the testing dataset at a given percentage split.	This is needed to test that we can split the dataset into training and testing data. This will be needed to evaluate the performance of the model, to ensure that we get the highest performance. *Links to success criteria 24 and 25 – 'Trains a machine learning model' and 'Tests a machine learning model.'*
3	Machine learning model (Requirement 8)	Training dataset	Training the first model with the training dataset	The model should be able to be trained based on this data and return some test statistics	This is needed to show that the model has been trained, and we can see how accurately it has been trained using the statistics. *Links to success criteria 24 – 'Trains a machine learning model.'*
4	Machine learning model (Requirement 8)	Testing dataset	Testing the first model with the testing dataset	The program should produce some statistics based on the performance for this testing data.	This is needed to evaluate the performance of this first model, so we can see how well it deals with no data. *Links to success criteria 25 – 'Tests a machine learning model'.*
5	Machine learning model (Requirement 8)	Training dataset	Training another model with	The program should train another type of machine learning	This is needed to show that another model has been trained, so we can

			the training dataset	model using the training data and obtain the performance statistics	compare results between different models. *Links to success criteria 26 – 'Compares multiple machine learning models.'*
6	Machine learning model (Requirement 8)	Testing dataset	Testing another model with the testing dataset	The program should test the second trained model with the unseen testing dataset, and evaluate the performance and return the statistics	This is needed to test the newly trained model by the same metric as the initial model. This allows for comparisons between the two models, to choose the most effective model. *Links to success criteria 26 – 'Compares multiple machine learning models.'*
7	Machine learning model (Requirement 8)	Full dataset	Comparing both models	The program should look at the test statistic of both models and select the model with a more accurate score.	This allows for comparisons between the two models, to choose the most effective model. *Links to success criteria 26 – 'Compares multiple machine learning models.'*

Phase 3: Developing a GUI

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Simple UI (Requirement 5)	N/A	Does UI pop up when program runs	When program runs the UI should be displayed	This is needed to test that the user is immediately allowed to access the program once the model has been trained. *Links to success criteria 16 – 'UI is displayed when program starts'*
2	Simple UI (Requirement 5)	Sample data to enter for prediction	Does program allow for data to be entered	The UI should allow the user to enter all the information required for the program to run as required.	This is needed to make sure that the user can enter their prediction – an important part of the program, and its main purpose. *Links to success criteria 17 – 'UI has all the options the user needs, such as entering their data and obtaining the prediction'*

3	Simple UI (Requirement 5)	Sample data to enter for prediction	Is information visible and easily accessible for user	UI should display large enough text for user to clearly see options	This is needed to test that the UI is easy to use and accessible. *Links to success criteria 18 – 'User can clearly see which buttons to press in the UI'*

Phase 4: Calculating a prediction

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Predicted price (Requirement 3)	Sample data for prediction	Does the program output a predicted price	The program should output a valid predicted price based on the inputs	This is needed to ensure that the program achieves its main function, by outputting a numerical price based on the inputs. *Links to success criteria 12 – 'Calculates and displays a predicted price using a machine learning model.'*
2	Predicted price (Requirement 3)	Data already in the dataset (or valid existing data on market)	Does the program predict an accurate price that matches the market	The program should output a price that is comparable to the price listed in the real-world market	This is needed to test how accurate the predictions are in real-world cases and gives clear evidence of accuracy in predictions. *Links to success criteria 13 – 'The predicted price should be as accurate as possible.'*

Phase 5: Presenting similar properties

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Similar properties (Requirement 6)	Sample data for prediction	Is the program able to find properties in the same geographical area	Should return properties that are in the same geographical area	This is needed to ensure that the algorithm to find similar properties functions, first for geographical location.

					Links to success criteria 20 – 'The properties share similar characteristics.'
2	Similar properties (Requirement 6)	Sample data for prediction	Is the program able to find similar properties which share multiple similarities	Should return a list of properties that are similar to the inputted data	This is needed to test that the algorithm to identify similar properties works, which is another key part of the program as specified by the stakeholders. *Links to success criteria 19 and 20 'Program identifies similar properties' and 'The properties share similar characteristics.'*
3	Information of property (Requirement 1)	Sample data to obtain similar properties	Is the program able to output the information about a similar property	Should display number of bedrooms/ bathrooms, size, address, tenure (if applicable)	This is needed to test that the user can clearly see all the information they will need to decide about a property. This was requested by the stakeholders again. *Links to success criteria 4-9*

Phase 6: Range of estimates

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Range of estimates (Requirement 4)	Sample data for prediction	Is the program able to output the lowest sold price seen for a similar property	Should return a minimum price for similar properties	This is needed to test that the algorithm is able to produce the minimum real-world values for properties that are similar to the user's inputs. This was requested by the stakeholders. *Links to success criteria 14 'Displays lowest price of similar properties'*
2	Range of estimates (Requirement 4)	Sample data for prediction	Is the program able to output the highest sold price seen	Should return a maximum price for similar properties	This is needed to test that the algorithm is able to produce the maximum real-world values for properties that are similar to the user's

			for a similar property		inputs. This was requested by the stakeholders. *Links to success criteria 15 'Displays highest price of similar properties'*

Phase 7: Creating a map to show location

Test	Requirement	Input	Description of test	Expected outcome	Justification
1	Location of property (Requirement 2)	Sample data for input (focussed on address)	Is the program able to connect to maps using the API	Should display a map with the specified address	This is needed to test that we can display the location of the property in context to the user, which was a key request made by the stakeholders. *Links to success criteria 10 – 'Displays map of local area.'*
2	Location of property (Requirement 2)	Sample data for input (focussed on address)	Is the map easy to understand and locate the property	Should make it clear to the user where the specific location is	This is needed so that the map is useful and beneficial to the user, rather than making it more confusing. *Links to success criteria 10 – 'Displays map of local area.'*
3	Location of property (Requirement 2)	Sample data for input (focussed on address)	Can the map be closed and opened properly	When open button clicked, map should display, and when closed clicked it should disappear	This is needed to ensure that the map is usable and only displayed when the user wants it to be. *Links to success criteria 18 – 'User can clearly see which buttons to press in the UI.'*
4	Location of property (Requirement 2)	Sample data for input (focussed on address)	Is the map interactive	The map should be movable and respond to movements by the cursor	This is needed to test that the user can use the map to see around the property. *Links to success criteria 11 – 'Map is navigable'*

Validity usability test table

This table will be used to check all inputs are validated, and will use normal, erroneous and boundary data. Some inputs are not necessary for validation as they will either not be used for the prediction model (user address), or a drop-down menu will be used (e.g. number of bedrooms)

Links to success criteria 3 – 'Validates all inputs'

Test number	Component	Input	Type of data	Expected outcome	Justification
1	Postcode	Valid postcode e.g. "SE6 2QE"	Normal	Accepts postcode	Validates postcode to ensure user input is a legitimate postcode *Links to success criteria 9 – 'Displays postcode'*
2	Postcode	Another postcode e.g. "W12 7GN"	Normal	Accepts postcode	Validates postcode to ensure user input is a legitimate postcode
3	Postcode	"IGK36 3NM"	Erroneous (exceeds maximum length)	Rejects postcode	Must ensure illegitimate postcodes are rejected, otherwise it will lead to an incorrect prediction as there is no data in that area.
4	Postcode	"1 6"	Erroneous (below minimum length)	Rejects postcode	Must ensure illegitimate postcodes are rejected, otherwise it will lead to an incorrect prediction as there is no data in that area.
5	Postcode	"SW99 8GP"	Erroneous (not a real postcode)	Rejects postcode	Must ensure illegitimate postcodes are rejected, otherwise it will lead to an incorrect prediction as there is no data in that area.
6	Property area	"100"	Normal	Accepts area	Must ensure that a valid property area size is given so that the model can then make accurate predictions *Links to success criteria 5 – 'Displays size and type of property'*

7	Property area	“100a”	Erroneous (only integer values allowed)	Rejects area	Same as above.
8	Property area	“0”	Erroneous (no value given)	Rejects area	Same as above.
9	Address	“21 St John’s Road”	Normal	Displays given address when outputting price	Ensure that the address is being passed through and displayed as entered by the user *Links to success criteria 8 – ‘Displays the address’*
10	Number of bathrooms	Doesn’t select	Erroneous	Rejects as no input	Ensures that program rejects any non-inputs
11	Number of bathrooms	Selects “2”	Normal	Uses given number for prediction and displays this	Ensures that the correct values are being passed through into the program for prediction *Links to success criteria 6 – ‘Displays the number of bathrooms’*
12	Number of bedrooms	Doesn’t select	Erroneous	Rejects as no input	Ensures that program rejects any non-inputs
13	Number of bedrooms	Selects “3”	Normal	Uses given number for prediction and displays this	Ensures that the correct values are being passed through into the program for prediction *Links to success criteria 4 – ‘Displays the number of bedrooms’*
14	Tenure	Enters “200”	Normal	Uses given number for prediction and displays this	Ensures that the correct values are being passed through into the program for prediction *Links to success criteria 7 – ‘Displays the tenure’*
15	Tenure	Enters “Two hundred”	Erroneous (must enter integer)	Uses given number for prediction and displays this	Ensures that only valid data types are fed into the model, which matches what the model has been trained on.

16	Type of property	Selects “Terrace Property”	Normal	Uses and displays given property type for prediction	Ensures that the correct property type is passed into the model for prediction *Links to success criteria 5 – ‘Displays size and type of property’*
17	Type of property	Selects “Flat/Maisonette”	Normal	Uses and displays given property type for prediction	Ensures that the correct property type is passed into the model for prediction *Links to success criteria 5 – ‘Displays size and type of property’*

Post development testing

This testing requires a series of run-throughs of users using my program. Here is the testing plan for some of them:

Test 1 (Based on stakeholder 1 – Property buyers):

Assuming the model has already been trained – the user will not have any input on when this occurs.

Step	Input	Type of data	Expected outcome	Justification
1	Enter postcode: "N13 5LWd"	Erroneous	Rejects postcode	This is needed to ensure that in a real-life scenario, any typos by the user will be recognised and rejected. *Links to success criteria 3 – 'Validates all inputs'*
2	Enter postcode: "N13 5LW"	Normal	Does not reject postcode but accepts	This checks that the program does accept the correct postcode *Links to success criteria 9 – 'Displays postcode'*
3	Enter Address: "72 Hamilton Crescent"	Normal	Accepts address and produces this later	This ensures the program can output the given user address *Links to success criteria 8 – 'Displays the address'*
4	Select property type 'Mid Terrace House	Normal	Accepts property type	This ensures the program uses the user's inputs for a prediction, and the user can select the property type *Links to success criteria 5 – 'Displays size and type of property'*
5	Select number of bedrooms '3'	Normal	Accepts number of bedrooms	Ensures user can select specific number of bedrooms to be entered for prediction *Links to success criteria 4 – 'Displays the number of bedrooms'*
6	Select number of bathrooms '1'	Normal	Accepts number of bathrooms	Ensures user can select specific number of bathrooms for the prediction *Links to success criteria 6 – 'Displays the number of bathrooms'*
7	Select tenure 'Freehold'	Normal	Accepts tenure	Ensures user can select correct leasehold for their property

				Links to success criteria 7 – 'Displays the tenure.'
8	Enters Property area 'hundred and two'	Erroneous	Rejects property area as incorrect data type	Ensures no invalid inputs for property area is accepted, as this will not be in the same format as the data that the model was trained on. *Links to success criteria 3 – 'Validates all inputs'*
9	Enters Property area '102'	Normal	Accepts property area	Ensures that program accepts a valid input for property area *Links to success criteria 5 – 'Displays size and type of property'*
10	Enters submit to enter	Normal	Submits user data for model prediction and returns a predicted price	Ensures that user is able to submit their information for a prediction *Links to success criteria 2 – 'Allows user to input parameters'*
11	Clicks on first similar property	Normal	Displays information regarding the first similar property	Ensures that the similar properties will be displayed and available for interaction for the user. *Links to success criteria 19 – 'Program identifies similar properties'*

Test 2 (Based on stakeholder 2 – Property sellers):

Assuming the model has already been trained – the user will not have any input on when this occurs.

Step	Input	Type of data	Expected outcome	Justification
1	Enter postcode: "SW1X 8AE"	Normal	Accepts postcode	This is needed so that the user can see that their postcode for prediction is accepted by the program
2	Enter address: ""	Erroneous	Rejects address	This is needed to validate that an empty address is rejected
3	Enter Address: "Flat 3, 18 Eaton Place"	Normal	Accepts address	This tests that a more complex address is accepted by the program.

4	Select property type ‘Flat/Maisonette’	Normal	Accepts property type	Shows the user that the program can accept all types of property chosen from the drop-down menu
5	Select number of bedrooms ‘10’	Boundary	Accepts number of bedrooms	Shows that the program can accept and process the user input for number of bedrooms, even with upper limit
6	Select number of bathrooms ‘2’	Normal	Accepts number of bathrooms	Shows that the user can input number of bathrooms for prediction
7	Doesn’t select tenure	Erroneous	Rejects empty selection	Shows that the program will validate all inputs even if not selected
8	Select tenure ‘Leasehold’	Normal	Accepts tenure	Shows that the user can choose their tenure from the drop-down menu and enter their data
9	Enters Property area ‘270’	Normal	Accepts property area	Shows that the program allows user to enter property area for the prediction
10	Enters submit to enter	Normal	Submits user data for model prediction and returns a predicted price	Shows that the program is able to take all the information inputted by the user, and feed it to the model to output a prediction
11	Press predict property again	Normal	Form to submit prediction again	Shows that the button to predict again is working
12	Predict same property again, this time with 3 bedrooms	Normal	Accepts information	Testing with a different parameter to see how price is affected
13	Opens map	Normal	Map will open in window	Shows that the program can produce a map of the location
14	Navigates map	Normal	Map moves correspondingly to show what area the user wishes to see	Shows that the program can connect with the map, and the user can navigate the map, which is one of the requirements to show location of property.

Development and testing

To develop this program, I will use an iterative method of development, where the project will be split up into multiple phases of development. These phases match the testing method outlined in the development section.

These phases are:

Phase 1: Web scraping
Phase 2: Machine learning model
Phase 3: Developing a GUI
Phase 4: Calculating a prediction
Phase 5: Presenting similar properties
Phase 6: Range of estimates
Phase 7: Creating a map to show location

Phase 1: Web scraping development

Overview

In this phase of development, the aim is to develop code to successfully extract information from a website. The information that should be extracted include: address, number of bedrooms, number of bathrooms and price.

This relates to Requirement 7: Web scraping, and the following success criteria:

- *Success Criteria 21: 'Uses web scraping to extract information from property website.'*
- *Success Criteria 22: 'Extracts only necessary information'*
- *Success Criteria 23: 'Stores necessary data into a suitable data structure.'*

As I was initially unfamiliar with web scraping, I used the referenced tutorial to help me learn about the libraries.

Development

Imports

The first stage in development of this phase is to import the required libraries for this program

```
import requests #Used to connect to the website to scrape data from
from bs4 import BeautifulSoup #This is a web scraping library
import pandas as pd #This is a dataset analysis library
```

These are the libraries that will allow me to be able to scrape data from the website, clean up and extract the desired data, and store the data into a dataset.

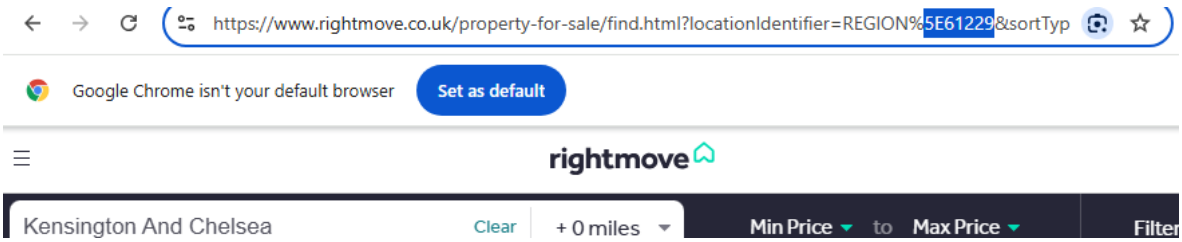
Region codes

This phase required me to web scrape the properties on sale from Rightmove. Having looked into the Rightmove website, I saw that they organise their location by regional codes. To start off with, I will look at the area of London, and divide the location by borough.

I first researched the boroughs in London, finding this list on Wikipedia:

1. City of London (not a London borough)
2. City of Westminster
3. Kensington and Chelsea
4. Hammersmith and Fulham
5. Wandsworth
6. Lambeth
7. Southwark
8. Tower Hamlets
9. Hackney
10. Islington
11. Camden
12. Brent
13. Ealing
14. Hounslow
15. Richmond upon Thames
16. Kingston upon Thames
17. Merton
18. Sutton
19. Croydon
20. Bromley
21. Lewisham
22. Greenwich
23. Bexley
24. Havering
25. Barking and Dagenham
26. Redbridge
27. Newham
28. Waltham Forest
29. Haringey
30. Enfield
31. Barnet
32. Harrow
33. Hillingdon

The Rightmove website has the regional code embedded within the links when searching for each borough. I've manually recorded these regional codes for each borough



I then created a dictionary which stored the regional codes for each borough, with the key being the borough name and the value being the Rightmove regional code.

```
"""
A dictionary that stores all the boroughs of London and their corresponding
regional
"""

boroughs = {
    "City of London":"5E61224",
    "City of Westminster":"5E61233",
    "Kensington and Chelsea":"5E61229",
    "Hammersmith and Fulham":"5E61407",
    "Wandsworth":"5E93977",
    "Lambeth":"5E93971",
    "Southwark":"5E61518",
    "Tower Hamlets":"5E61417",
    "Hackney":"5E93953",
    "Islington":"5E93965",
    "Camden":"5E93941",
    "Brent":"5E93935",
    "Ealing":"5E93947",
    "Hounslow":"5E93962",
    "Richmond upon Thames":"5E61415",
    "Kingston upon Thames":"5E93968",
    "Merton":"5E61414",
    "Sutton": "5E93974",
    "Croydon": "5E93944",
    "Bromley": "5E93938",
    "Lewisham": "5E61413",
    "Greenwich": "5E61226",
    "Bexley": "5E93932",
    "Havering": "5E61228",
    "Barking and Dagenham": "5E61400",
    "Redbridge": "5E61537",
    "Newham": "5E61231",
    "Waltham Forest": "5E61232",
    "Haringey": "5E61227",
    "Enfield": "5E93950",
    "Barnet": "5E93929",
    "Harrow": "5E93956",
    "Hillingdon": "5E93959"

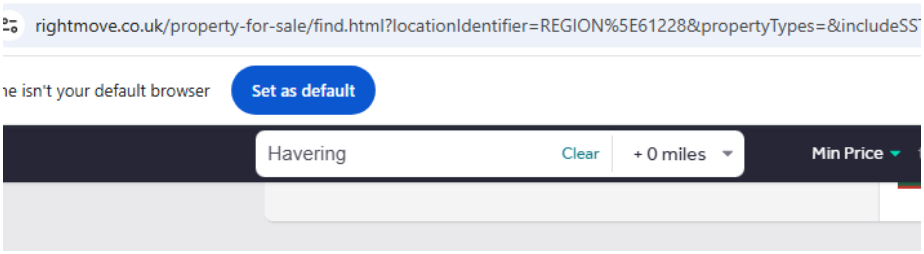
}
```

Initial scraping and research

I started off the scraping programming by setting up the variables that will be used later as I scrape data from the website. This included the lists that would store the data, as well as the index which would update the web link as I scraped through.

```
#Setting up the preliminary lists to store all the data of the properties
all_properties_address = [] # This stores the address of the property
all_properties_price = [] # This stores the price of the property
```

I started off by looking at the website and the links to each of the web pages. I found that the regional codes changed as well as the index, which Rightmove used to show the current property shown on the page.



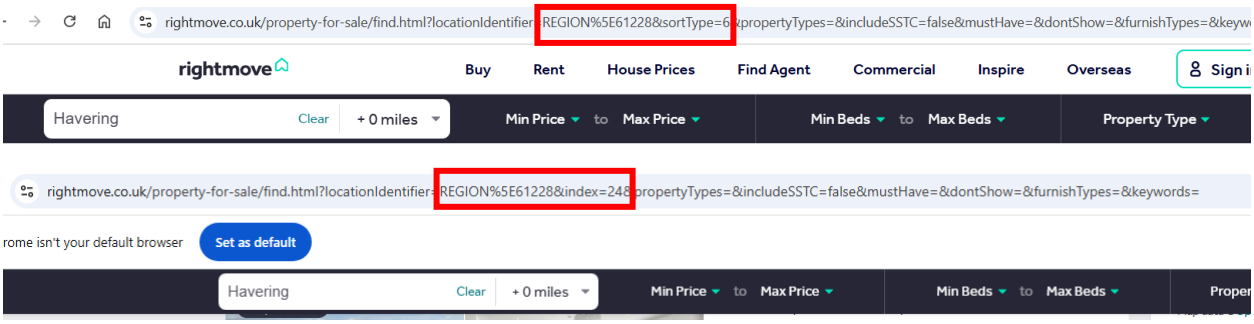
The program needed to iterate through all the boroughs, inserting the regional code into the web link to allow the scraper to crawl through and extract the html from all available properties on the website. I wrote this for loop to carry out this:

```
# Set up a loop to iterate through all the boroughs
for borough in list(boroughs.values()):

  # This will initialise the index to match the index on the rightmove website
  index = 0

  # This is to show what the program is currently trying to scrape
  current_key = [key for key,value in boroughs.items() if value == borough][0]
  print(current_key)
```

I also found that the web link on the first page was different to all the other pages. This can be seen in this comparison between the link on the first page (top image) vs the second page (bottom image)



To combat this, I then wrote a selection statement that changed the link depending on which page I was scraping. I used the f string printing to allow me to change the link based to adjust to what I was scraping.

```
#The links on the website are slightly different on the first page compared to others
if index == 0:
  rightmove_link = f"https://www.rightmove.co.uk/property-for-sale/find.html?locationIdentifier=REGION%{borough}&sortType=6&propertyTypes=&inc
elif index != 0:
  rightmove_link = f"https://www.rightmove.co.uk/property-for-sale/find.html?locationIdentifier=REGION%{borough}&sortType=6&index={index}&prop
```

Once I had the link to the webpage, I used the requests library to send HTTP requests for the specific link that I wanted. I then used the beautiful soup library to get the HTML of the particular page.

```
#The requests with the specific link and headers are sent to the website
request = requests.get(rightmove_link, headers=user_headers)
request.raise_for_status()

#This receives all the html of the web page using the BeautifulSoup library
page_html = BeautifulSoup(request.text, "html.parser")
return page_html
```

To test that I was able to return the html, I created a small test function that wasn't in a loop. This meant that instead of looping through all of the boroughs, I could pass only one borough and return the html for the first page to test it worked for development purposes:

```
#Code outside of for loop for testing html web scraping

#Set initial values so that we only scrape from one page to begin with
borough = "5E61224"
index = 0

#Function that will be used to test the web scraping
def html_test(borough, index):
  #This is needed to match the header of the browser when scraping
  user_headers = {
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Ch
  }

  #The links on the website are slightly different on the first page compared to others
  if index == 0:
    rightmove_link = f"https://www.rightmove.co.uk/property-for-sale/find.html?locationIdentifier=REGION
  elif index != 0:
    rightmove_link = f"https://www.rightmove.co.uk/property-for-sale/find.html?locationIdentifier=REGION

  #The requests with the specific link and headers are sent to the website
  request = requests.get(rightmove_link, headers=user_headers)
  request.raise_for_status()

  #This receives all the html of the web page using the BeautifulSoup library
  page_html = BeautifulSoup(request.text, "html.parser")
  return page_html
```

The output from this test function was as expected, and it produced the full html of the web page without any issues:

```
#This outputs it for development and testing purposes
print(html_test(borough, index))
```

```
<!DOCTYPE html>

<html class="is-not-modern property-for-sale channel--buy" lang="en-GB">
<head>
<meta charset="utf-8"/>
<title>Properties For Sale in City Of London | Rightmove</title>
<meta content="IE=Edge" http-equiv="X-UA-Compatible">
<meta content="no-cache, no-store, must-revalidate" http-equiv="Cache-Control"/>
<meta content="no-cache" http-equiv="Pragma"/>
<meta content="width=device-width, shrink-to-fit=no, initial-scale=1.0, user-scalable=no" name="viewport
<meta content="telephone=no" name="format-detection"/>
<meta content="True" name="HandheldFriendly"/>
```

Extracting address

Having received the full html of the web page, I needed to locate and extract the data that I needed, such as property price and address for example. I realised that the html corresponding with each of the 24 properties available on each page began with class “l-searchResult is-list” as shown here:

```
</div>
</div>
</div>
</div>
<div class="l-searchResult is-list" data-bind="attr: { 'id': 'property-' + id() },
<div class="propertyCard" data-bind="css: {'propertyCard--premium': premiumListing
<a class="propertyCard-anchor" id="prop156183263"></a>
<div class="propertyCard-wrapper">
<div class="propertyCard-images">
<div class="propertyCard-main-img-mask aspect-3x2">
<div class="propertyCard-main-img">
<a class="propertyCard-img-link aspect-3x2" data-test="property-img" href="/proper
<div class="propertyCard-img">
```

Hence using the find.all() method, which returned a list with each element being an instance of the target string occurring, I was able to create a list of all the html related to the properties. All the other html was then ignored, following *Success Criteria 22: 'Extracts only necessary information.'*

I used this code to extract the html relevant to the properties:

```
#This receives all the html of the web page using the BeautifulSoup library
page_html = BeautifulSoup(request.text, "html.parser")

#Having identified the class data is found in, this returns all relevant html
properties_html = page_html.find_all("div", class_="l-searchResult is-list")
return properties_html
```

The output was just the relevant information within a list as shown here:

```
[<div class="l-searchResult is-list" data-bind="attr: { 'id': 'property-' + id() }, css: { 'is-hidden': h
<div class="propertyCard propertyCard--featured" data-bind="css: {'propertyCard--premium': premiumListing
<a class="propertyCard-anchor" id="prop146546570"></a>
<div class="propertyCard-wrapper">
<div class="propertyCard-images">
<div class="propertyCard-main-img-mask aspect-3x2">
<div class="propertyCard-main-img">
<a class="propertyCard-img-link aspect-3x2" data-test="property-img" href="/properties/146546570#/?channe
<div class="propertyCard-img">
<span class="no-svg-camera camera">
<svg>
<use xlink:href="#core-icon--camera"></use>
</svg>
```

Looking through the html, I was able to locate the address of each property, and the hierarchy of the html in which this data was stored.

The address was found to be within the class “propertyCard-address”, and under the tag address.

```
<img alt="Dexters, Shoreditch Logo" class="propertyCard-branchLogo-image"/>
</a>
</div>
<div class="propertyCard-details">
<a class="propertyCard-link" data-test="property-details" href="/properties/156117188#/?channel=RES_BUY">
<h2 class="propertyCard-title" itemprop="name">
            2 bedroom flat for sale        </h2>
<address class="propertyCard-address" itemprop="address" itemscope="" itemtype="http://schema.org/PostalAddress">
<meta content="Black Friars Lane, Blackfriars" itemprop="streetAddress"/>
<meta content="GB" itemprop="addressCountry"/>
<span>Black Friars Lane, Blackfriars</span>
</address>
</a>
</div>
<div class="propertyCard-description">
<a class="propertyCard-link" href="/properties/156117188#/?channel=RES_BUY">
<span data-test="property-description" itemprop="description">This large, second floor flat located central to Blackfriars provides
</a>
</div>
```

I started by finding the information under the class “propertyCard-address”, using the following code, which used the method find() to locate all the html which was within that class. This would then be appended to the list of all the property addresses.

```
    #Obtaining the address of a specific property
    address = (
        property_info.find("address", class_="propertyCard-address")
    )
    all_properties_address.append(address)

#Outputting this list for testing purposes
return all_properties_address
```

The following was outputted:

```
</address>, <address class="propertyCard-address" itemprop="address" itemscope="" itemtype="http://schema.
<meta content="Moor Lane, London, EC2Y" itemprop="streetAddress"/>
<meta content="GB" itemprop="addressCountry"/>
<span>Moor Lane, London, EC2Y</span>
</address>, <address class="propertyCard-address" itemprop="address" itemscope="" itemtype="http://schema.
<meta content="Black Friars Lane, Blackfriars" itemprop="streetAddress"/>
<meta content="GB" itemprop="addressCountry"/>
<span>Black Friars Lane, Blackfriars</span>
</address>, <address class="propertyCard-address" itemprop="address" itemscope="" itemtype="http://schema.
<meta content="Minories, London" itemprop="streetAddress"/>
<meta content="GB" itemprop="addressCountry"/>
<span>Minories, London</span>
</address>, <address class="propertyCard-address" itemprop="address" itemscope="" itemtype="http://schema.
<meta content="Lauderdale Tower, Barbican, London" itemprop="streetAddress"/>
```

This was not what I had hoped for, as it simply returned all the html within that class, rather than just the text. However, as the text was within the html, I added the following code to solve the problem and isolate just the text:

```
  #Obtaining the address of a specific property
  address = (
      property_info.find("address", class_="propertyCard-address")
      .get_text() #This will extract the text from the html
      .strip() #This ensures only the desired text is saved
  )
  print(address)
  all_properties_address.append(address)

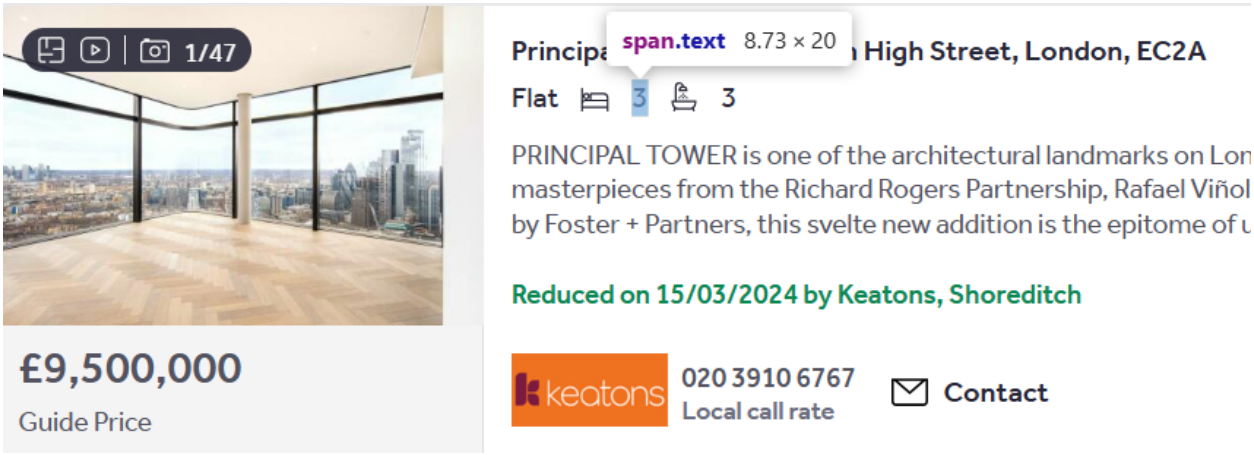
#Outputting this list for testing purposes
return all_properties_address
```

This solved the problem using the get_text() and strip() methods which were able to remove all the html surrounding the text, leaving just the address to be appended to the list. The output of all the addresses can be seen here:

```
The Haydon,
15-16 Minories,
London,
EC3N 1AX
Bartholomew Close, Dominion House
Crestway, Chatham, Kent
Portsoken Street, Aldgate
Black Friars Lane, Blackfriars
Barbican, Ben Jonson House
Wormwood Street, City Wall House, Wormwood Street
Moor Lane London EC2Y
Bishopsgate Plaza, EC3A
10 Hosier Lane, London
Bridgewater Square, Clerkenwell
Bridgewater Square, London, EC2Y
```

Problematic extraction

After extracting the address, I tried to extract the type of property, such as 'Flat' or 'House' etc. This was visible on the Rightmove page and the inspect element HTML, as shown:



The relevant code from the inspect element, containing all the desired information

```
<div class="property-information">flex == $0
  <span class="text">Flat</span>
  <span class="no-svg-bed-icon bed-icon seperator">...</span>
  <span class="text" aria-hidden="true">3</span>
  <span class="no-svg-bathroom-icon bathroom-icon seperator">...
  </span>
  <span class="text" aria-hidden="true">3</span>
</div>
</a>
```

I wrote some code to extract the text from the class "property-information", shown below:

```
#Obtaining the type of the property
type = (property_info.find_all("div", class_="property-information"))
print(type)
```

However, when I wanted to extract the information via the class "property-information" I encountered this error message:

```
            raise AttributeError(
                "ResultSet object has no attribute '%s'. You're probably treating a list of elements like a single element. Did
you call find_all() when you meant to call find()?" % key
            )

AttributeError: ResultSet object has no attribute 'get'. You're probably treating a list of elements like a single element. Did
you call find_all() when you meant to call find()?
```

I then changed the method from find_all() to find() in an effort to fix the error message. The error message did not appear, but when scraping, the code outputted no values, displaying 'None' as shown:

```
City of London
None
None
None
None
None
None
None
None
None
None
None
```

I then wrote a short piece of code to output the html of the whole website to check that if the lines of text I wanted were returned in the html.

```
#This loop will go through each individual properties' html found in the full list
for property_no in range(len(properties_html)):
  current_property = properties_html[property_no]
  #Temporary code to return full html
  print(current_property)
```

The relevant section in the output can be seen below. The corresponding code is marked in green. The red shows the code on the website which I am unable to extract.

```
<div class="propertyCard-content">
<div class="propertyCard-section">
<div class="propertyCard-branchLogo">
<a class="propertyCard-branchLogo-link" data-test="property-branch-logo" |
<img alt="Dexters, Shoreditch Logo" class="propertyCard-branchLogo-image".
</a>
</div>
<div class="propertyCard-details">
<a class="propertyCard-link" data-test="property-details" href="/properti
<h2 class="propertyCard-title" itemprop="name">
            2 bedroom flat for sale        </h2>
```

The extracted html didn't contain the relevant lines of code that were visible in the inspect element:



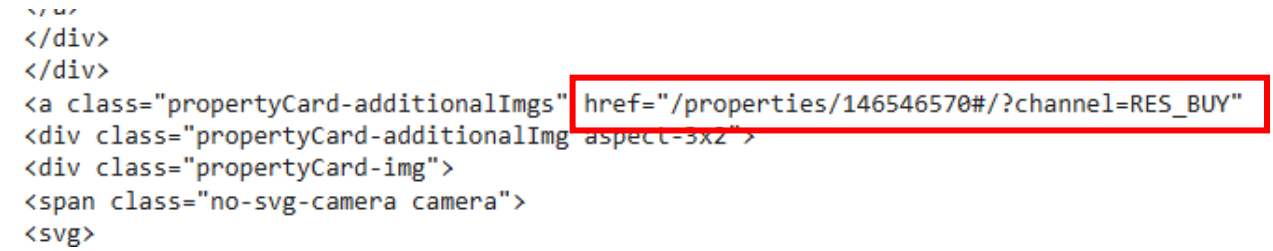
Some of the information that I was trying to extract, for example number of bedrooms and bathrooms were not visible. This I realised was because this information was dynamically loaded, and hence not contained within the html. Hence, I needed to store the web link to each property to scrape the more detailed information again later.

Extracting links

I first updated the original arrays to include a list of web links as well:

```
#Setting up the preliminary lists to store all the data of the properties
all_properties_address = [] # This stores the address of the property
all_properties_price = [] # This stores the price of the property
all_properties_links = [] # This stores the web link of the property
```

Looking at the returned html, I was able to see the individual links contained within this class:



I then wrote a function that found and isolated the link, before concatenating this with the top-level domain, second-level domain and subdomain to create the full link. I then appended this to the list containing all the web links.

```
#This function will be used to locate and extract all the data from the webpage
def data_retrieval_development(properties_html):

  #This loop will go through each individual properties' html found in the full list
  for property_no in range(len(properties_html)):
    current_property = properties_html[property_no]

    #Obtaining the link to the webpage of the specific property

    #This finds and extracts the class where the link is held
    property_info = current_property.find("a", class_="propertyCard-link")

    #Concatenate the individual link to form a proper weblink
    link = "https://www.rightmove.co.uk" + property_info.attrs["href"]

    #Append this link to our list
    all_properties_links.append(link)

    #Outputting this list for testing purposes
  return all_properties_links
```

This outputted all the links into a list:

```
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.uk/proper
```

Extracting price

The next step was to extract the price of each property. This was located under the class "propertyCard–priceValue" as shown below:

```
</div>
<div class="propertyCard-header"> <div class="propertyCard-price"> <a class="propertyCard-priceLink propertyCard-salePrice" data-bind="click: property
<div class="propertyCard-priceValue" data-bind="text: price.displayPrices()[0] ? price.displayPrices()[0].displayPrice : ''">£1,800,000 </div>
<!-- ko if: priceQualifierText -->
<div class="propertyCard-priceQualifier">
```

Then, similar to the code used to extract the address, I added this code to extract and append the price to the relevant list:

```
#Obtaining the price of a specific property
price = (
    current_property.find("div", class_="propertyCard-priceValue")
    .get_text() #Extract the price
    .strip() #Ensure it is only the price
)
print(price)
all_properties_price.append(price)
```

This outputted the prices of all the properties as shown here:

```
£850,000
£1,920,000
£200,000
£900,000
£949,000
£1,250,000
£759,500
£500,000
£3,400,000
```

Combining links, address and price

After I had successfully managed to develop the code which could extract the links, address and price, I combined them all into one singular function, which took in as a parameter the list of multiple html related to properties. This can be seen here below:

```
#This function will be used to locate and extract all the data from the webpage
def data_retrieval_development(properties_html):

  #This loop will go through each individual properties' html found in the full list
  for property_no in range(len(properties_html)):
    current_property = properties_html[property_no]


    #Obtaining the link to the webpage of the specific property

    #This finds and extracts the class where the link is held
    property_info = current_property.find("a", class_="propertyCard-link")

    #Concatenate the individual link to form a proper weblink
    link = "https://www.rightmove.co.uk" + property_info.attrs["href"]

    #Append this link to our list
    all_properties_links.append(link)


    #Obtaining the address of a specific property
    address = (
        property_info.find("address", class_="propertyCard-address")
        .get_text() #This will extract the text from the html
        .strip() #This ensures only the desired text is saved
    )
    all_properties_address.append(address)

    #Obtaining the price of a specific property
    price = (
        current_property.find("div", class_="propertyCard-priceValue")
        .get_text() #Extract the price
        .strip() #Ensure it is only the price
    )
    print(price)
    all_properties_price.append(price)
```

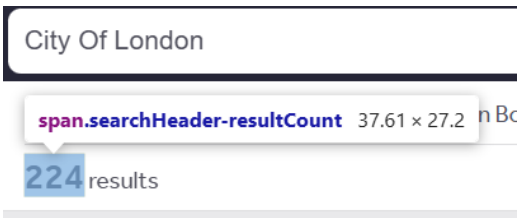
However, after running the program and allowing it to scrape from one particular borough to begin with, I noticed that the program was not able to stop the code when there were no more properties. This can be seen here in the output as the program carries on scraping even where there are no properties left to scrape.

```
£4,400,000
9 pages scraped
£1,125,000
£625,000
£880,000
£395,000
£50,000
£899,000
£835,000
10 pages scraped
£2,100,000
11 pages scraped
£550,000
12 pages scraped
£1,600,000
13 pages scraped
14 pages scraped
15 pages scraped
16 pages scraped
17 pages scraped
18 pages scraped
19 pages scraped
```

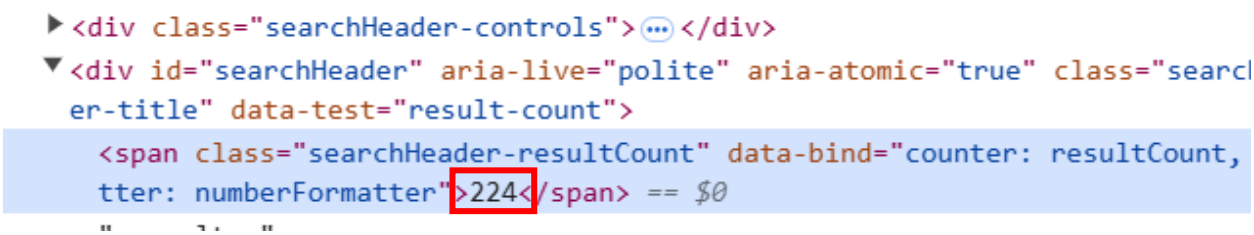
Finding total number of listings

One way in which this could be stopped was stopping the loop when the maximum number of properties had been reached. This meant scraping the total number of listings for a particular area.

On the Rightmove website, the total number of listings is displayed at the top of the screen as shown below:



The corresponding html in inspect element can also be seen, showing where the value is stored within the html:



Knowing this I then wrote the following code, which found that specific class "searchHeader-resultCount" :

```
#Find total number of listings displayed
total_no_listings = (
    page_html.find("span", {"class": "searchHeader-resultCount"})
)
```

However, this again outputted the full html rather than just the text as shown below:

```
City of London
<span class="searchHeader-resultCount" data-bind="counter: resultCount, formatter: numberFormatter">224</span>
```

Hence I added the get_text() function into the code:

```
#Find total number of listings displayed
total_no_listings = (
    page_html.find("span", {"class": "searchHeader-resultCount"})
    .get_text()
)
#Obtain just the integer value so can be used later
print(total_no_listings)
```

I then realised that it had to be an integer to be useful in the rest of the code, so I converted it to an integer, ensuring that no commas which were not removed initially were still present. Here is the full code to extract the total number of listings:

```
#Having identified the class data is found in, this returns all relevant html
properties_html = page_html.find_all("div", class_="l-searchResult is-list")

#Find total number of listings displayed
total_no_listings = (
    page_html.find("span", {"class": "searchHeader-resultCount"})
    .get_text()
)
#Obtain just the integer value so can be used later
total_no_listings = int(total_no_listings.replace(",", ""))
print(total_no_listings)
```

The output can be seen here, where the total number of listings is displayed after the borough name to show it has been working.

```
City of London
222
£2,355,000
£1,920,000
```

I then added this selection into the code to ensure that it moves onto the next borough if it has reached the maximum properties. Every page has 24 properties hence, the index increases by 24 after every page. The variable max_search can be used to limit the amount of scraping done to speed up the program, if not all the data available on Rightmove is required.

```
# Jump to the next page of properties on website
index = index + 24

#To stop the program when there are no more listings
if index >= total_no_listings or index >= max_search:
  break
```

I also could not see how many pages I was scraping through, or the progress I had made, so I added these lines of code to give the user a better idea:

```
# For development purposes to show I am actually scraping
print(f"{pages + 1} pages scraped")
print(f"You have {total_no_listings - index} listings left")
```

This worked well, and an example output can be seen here, where I scrape multiple boroughs:

```
City of London
1 pages scraped
You have 222 listings left
2 pages scraped
You have 198 listings left
3 pages scraped
You have 174 listings left
4 pages scraped
You have 150 listings left
5 pages scraped
You have 126 listings left
6 pages scraped
You have 102 listings left
7 pages scraped
You have 78 listings left
8 pages scraped
You have 54 listings left
9 pages scraped
You have 30 listings left
10 pages scraped
You have 6 listings left
City of Westminster
1 pages scraped
You have 4932 listings left
2 pages scraped
You have 4908 listings left
```

Maximum search

I used the variable max_search to be able to tell the program how many properties I wanted to search as a maximum before it moved onto the next borough. This is passed in when calling the function as shown below:

```
      #To stop the program when there are no more listings
      if index >= total_no_listings or index >= max_search:
        break

property_scraping(1000)
```

This worked as shown below in the output. Even though there are 3972 listings left in the borough, it still moves on to the next borough.

```
33 pages scraped
You have 4164 listings left
34 pages scraped
You have 4140 listings left
35 pages scraped
You have 4116 listings left
36 pages scraped
You have 4092 listings left
37 pages scraped
You have 4068 listings left
38 pages scraped
You have 4044 listings left
39 pages scraped
You have 4020 listings left
40 pages scraped
You have 3996 listings left
41 pages scraped
You have 3972 listings left
Kensington and Chelsea
1 pages scraped
You have 3084 listings left
2 pages scraped
You have 3060 listings left
3 pages scraped
You have 3036 listings left
4 pages scraped
```

Extracting additional information

As well as extracting the address, link and price of the property, according to my requirements (Requirement 1 – Information of property), I also needed to obtain more detailed information about each property. However, as shown previously, the information is not extractable from the static html. Hence, I needed to request the web page for each individual link and then scrape from it.

The first step was to iterate through the list of properties, requesting the corresponding link for each property. I achieved this by creating a new function with a while loop to iterate until there were no more links:

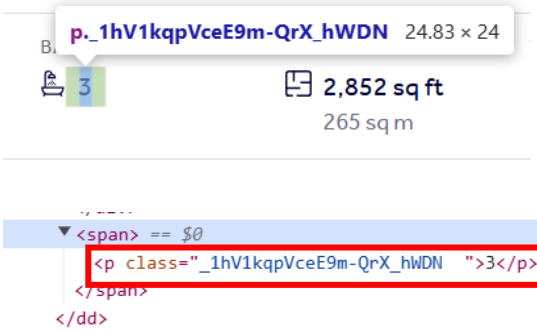
```
#Function to extract the more detailed information from each specific property
def detailed_info_scrape():
  #Iterate through all the properties found in the list of links
  property_no = 0
  while property_no < len(all_properties_links):
```

The next step was to request the specific property link. I was able to do this quite easily as I already had the list of all properties and their links, and all I needed to do was to request that specific link:

```
property_link = all_properties_links[property_no]

user_headers = {
  "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (K
}
request = requests.get(property_link, headers=user_headers)
request.raise_for_status()
```

I then looked at an example property page, and located the relevant class in the inspect element html:



All of the relevant information was contained within that specific class, so I was able to extract it using this code:

```
#This receives the html of the property web page using the BeautifulSoup library
property_html = BeautifulSoup(request.text, "html.parser")
properties_html = property_html.find_all("p", class_="_1hV1kqpVceE9m-QrX_hWDN")
print(properties_html)
```

The output was a list of all the instances where the class was used:

```
[<p class="_1hV1kqpVceE9m-QrX_hWDN">Apartment</p>, <p class="_1hV1kqpVceE9m-QrX_hWDN">2</p>, <p class="_1hV1kqpVceE9m
[<p class="_1hV1kqpVceE9m-QrX_hWDN">Apartment</p>, <p class="_1hV1kqpVceE9m-QrX_hWDN">2</p>, <p class="_1hV1kqpVceE9m
[<p class="_1hV1kqpVceE9m-QrX_hWDN">Apartment</p>, <p class="_1hV1kqpVceE9m-QrX_hWDN">2</p>, <p class="_1hV1kqpVceE9m
[<p class="_1hV1kqpVceE9m-QrX_hWDN">Flat</p>, <p class="_1hV1kqpVceE9m-QrX_hWDN">1</p>, <p class="_1hV1kqpVceE9m-QrX_
```

There were multiple lists for multiple properties. The items in each list are ordered in such a way that they correspond to the order on the web page:



However, a problem I encountered was that some properties did not contain all 5 pieces of information for example:



This caused a big issue as there was no possible way of knowing which piece of information I was missing, as the list had no label as to which number meant bathrooms or bedrooms. Hence, to solve this, I decided to amend the code by checking if there were five items in each list. If there was not, then I would ignore it. Initially I wrote this code:

```
# This code will only extract the information if everything is present
if len(properties_html) == 5:
  all_properties_type.append(properties_html[0].get_text().strip())
  all_properties_bedrooms.append(properties_html[1].get_text().strip())
  all_properties_bathrooms.append(properties_html[2].get_text().strip())
  all_properties_size.append(properties_html[3].get_text().strip())
  property_no+=1
```

However, this caused a problem as the lists containing the address and price still had that specific property's data stored. This caused the lists to be of different length, and they therefore were not compatible with each other.

This error was thrown when I tried to iterate through the lists:

```
IndexError: list index out of range
```

Hence, to solve this problem, I amended the code so that it would remove the corresponding data from the old lists if all the information was not present:

```
#This validates that all information is present otherwise I discard
if len(properties_html)!= 5:
  all_properties_links.pop(property_no)
  all_properties_price.pop(property_no)
  all_properties_address.pop(property_no)

#This will extract the information and append to the relevant list
else:
  all_properties_type.append(properties_html[0].get_text().strip())
  all_properties_bedrooms.append(properties_html[1].get_text().strip())
  all_properties_bathrooms.append(properties_html[2].get_text().strip())
  all_properties_size.append(properties_html[3].get_text().strip())
  property_no+=1
```

The evidence that the specific item has been deleted is shown in this output, where I asked the code to print “deleted” if it had removed a property:

```
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
deleted
https://www.rightmove.co.uk/properties/156519341#/?channel=RES_BUY
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
['https://www.rightmove.co.uk/properties/150781844#/?channel=RES_BUY', 'https://www.rightmove.co.u
```

To check it was working, I ran the program from the beginning and asked for a specific property’s data, which is was able to return and was correct:

```
test = 15
print(all_properties_links[test])
print(all_properties_address[test])
print(all_properties_price[test])
print(all_properties_type[test])
print(all_properties_bedrooms[test] + " bedrooms")
print(all_properties_bathrooms[test] + " bathrooms")
print(all_properties_area[test])
```

```
https://www.rightmove.co.uk/properties/154626818#/?channel=RES_BUY
10 High Timber Street, London, EC4V
£500,000
Flat
1 bedrooms
1 bathrooms
672 sq ft
```

The full function for to extract the more detailed information can be seen here:

```
#Function to extract the more detailed information from each specific property
def detailed_info_scrape():
  #Iterate through all the properties found in the list of links
  property_no = 0
  while property_no < len(all_properties_links):

    property_link = all_properties_links[property_no]

    user_headers = {
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Geck
    }
    request = requests.get(property_link, headers=user_headers)
    request.raise_for_status()

    #This receives the html of the property web page using the BeautifulSoup library
    property_html = BeautifulSoup(request.text, "html.parser")
    properties_html = property_html.find_all("p", class_="_1hV1kqpVceE9m-QrX_hWDN")


    #This validates that all information is present otherwise I discard
    if len(properties_html)!= 5:
      all_properties_links.pop(property_no)
      all_properties_price.pop(property_no)
      all_properties_address.pop(property_no)

    #This will extract the information and append to the relevant list
    else:
      all_properties_type.append(properties_html[0].get_text().strip())
      all_properties_bedrooms.append(properties_html[1].get_text().strip())
      all_properties_bathrooms.append(properties_html[2].get_text().strip())
      all_properties_area.append(properties_html[3].get_text().strip())
      property_no+=1


detailed_info_scrape()
```

Storing the data

Finally, I needed to store the data into a dataset. This was first done by converting the lists into a dictionary before converting it into a data frame using the panda array:

```
#Storing the data into a dictionary

data = {
    "Address": all_properties_address,
    "Type": all_properties_type,
    "Bedrooms": all_properties_bedrooms,
    "Bathrooms": all_properties_bathrooms,
    "Size": all_properties_size,
    "Price": all_properties_price
}

#Converting the dictionary into a data frame
df = pd.DataFrame.from_dict(data)
df.to_csv(r"sales_data.csv", encoding="utf-8", header="true", index = False)
```

The final outputted data frame can be seen below. I scraped only data from two boroughs, hence there is around 1000 properties:

```
                                              Address       Type Bedrooms  \
0                        Sugar Quay, Tower Hill, EC3R  Apartment        2
1                Bridgewater Square, Bridgewater Square       Flat        3
2     Flat 1 Aldersgate Court, 30 Bartholomew Close,...       Flat        1
3                          Golden Lane Estate, Barbican       Flat        1
4                                   Moor Lane, Moorgate       Flat        1
...                                                 ...        ...      ...
1170  The Conrad, Millbank, Westminster, London, SW1...  Penthouse        5
1171  The Astor, Millbank, Westminster, London, SW1P...  Penthouse        5
1172                          Edgware Road, Hyde Park, W2  Apartment        1
1173               Buckingham Palace Road, London, SW1W  Apartment        2
1174                               Park Road, London, NW1  Apartment        1

     Bathrooms         Size       Price
0            2  1,163 sq ft   2,355,000
1            2    Ask agent     799,500
2            1    Ask agent     240,000
3            1    Ask agent     450,000
4            1    Ask agent     750,000
...        ...          ...         ...
1170         7  7,219 sq ft  17,500,000
1171         8  9,582 sq ft  24,000,000
1172         1    581 sq ft     550,000
1173         2    933 sq ft   1,800,000
1174         1    Ask agent     550,000

[1175 rows x 6 columns]
```

Validation and testing

Validation is not needed as no user inputs are required. The only input type is normal for this phase.

The testing table previously outlined in the design is shown here with the results of each test:

Test	Input	Description of test	Expected outcome	Result	Evidence
1	Website address	Fetch html from specified website using web scraping	Returns full html code for specified website	PASS	*Picture 1.1*
2	Class/id in given html	Fetch html from a specific class or id from given website using web scraping	Returns html code for specific section of the website	PASS	Fetching address *Picture 1.2*
3	Specific class / id found in html	Fetch specific data from within the html	Returns the actual data values for the specified item	PASS	Returned string of address *Picture 1.3*
4	Multiple classes / id's found in html	Fetch multiple specified data from within the html	Returns the data values for all the items specified	PASS	*Picture 1.4*
5	Singular property found in multiple web pages	Fetch a singular characteristic / property of multiple web pages. For example, address in 3 different example pages	Returns the specified data value found in all different web pages	PASS	Scraping address *Picture 1.5*
6	Multiple properties found in multiple instances on the web site	Fetch multiple sets of data from multiple web pages. For example, address and price in 3 different example pages	Returns all the specified data found in all the different web pages	PASS	*Picture 1.6*
7	Multiple arrays of data	Store this information into a dataset, using panda library	Creates a dataset with all the information in the correct format	FAIL	Only postcode missing *Picture 1.7*

Failures

Test 7:

This test was failed as not “all the information” was scraped from the website. I failed in scraping the postcode from the website. This was mainly due to the fact that Rightmove does not format their addresses in a fixed way. This meant that some properties did not have the postcode on the address and others did. This made it nearly impossible to extract the postcode without any errors.

This particular website does display a scrapable postcode - however this is displayed dynamically using java script, making it impossible to retrieve the information when simply scraping the HTML of the web page. To scrape the postcode another method of scraping would be required, which after some attempts at using I found were too difficult beyond the scope of an A-Level project.

For the remainder of the program, I will use a pre-scraped dataset which does contain the postcode, as a demonstration that the program can still function. This dataset contains the same information as the dataset I would have scraped, but also includes the postcode, which is necessary for training my machine learning model.

Review

What has been done

The data on which the whole program is built upon has been scraped from a property market website. The data has then been formatted and processed into a panda data frame, so that it can be passed into the machine learning model for training.

How it has been tested

The functionality of the web scraping code has been tested using the phase 1 test made in the *Design: Iterative testing method*, and it has been confirmed that the web scraping code functions as intended and meets the requirements. The only failure on the success criteria has been explained and although not fully a success, it has been proven in concept.

How it meets success criteria and user expectations

The following success criteria have been met:

Requirement 7: Web scraping

1. Uses web scraping to extract information from property website
2. Extracts only necessary information
3. Stores necessary data into a suitable data structure

User expectations have been met as up-to-date information can be used to train the machine learning model.

Changes in design that have resulted from this section

None

Stakeholder Feedback

All stakeholders were happy with the progress and there were no concerns at this stage

Summary of the whole project as a prototype at this stage

The first stage as been completed, allowing for the rest of the program to be developed as the data can now be used to create the model

Phase 2: Machine learning model

Overview

In this phase of development, the aim was for the machine learning model to be trained with the property data, cleaning up any data, and making the data suitable for inputting into the machine learning model.

This relates to Requirement 8: Machine learning model and the following success criteria:

- *Success criteria 24: "Trains a machine learning model"*
- *Success criteria 25: "Tests a machine learning model"*
- *Success criteria 26: "Compares multiple machine learning models"*

Development

The first step was to import the dataset into the program:

```
                                    fullAddress  postcode outcode  bathrooms  bedrooms  floorAreaSqM     tenure         propertyType  currentPrice
0        1 Guinness Court, Mansell Street, London, E1 8AB     E1 8AB      E1        1.0       2.0          79.0   Freehold     Flat/Maisonette      555000.0
1       306 Marlyn Lodge, 2 Portsoken Street, London, ...     E1 8RB      E1        1.0       1.0          54.0  Leasehold  Purpose Built Flat      496000.0
2       Flat 1, White Rose Court, Widegate Street, Lon...     E1 7ES      E1        2.0       2.0          73.0  Leasehold  Purpose Built Flat      651000.0
3       Flat 1, White Rose Court, Widegate Street, Lon...     E1 7ES      E1        2.0       2.0          73.0  Leasehold  Purpose Built Flat      651000.0
4       Flat 1, White Rose Court, Widegate Street, Lon...     E1 7ES      E1        2.0       2.0          73.0  Leasehold  Purpose Built Flat      651000.0
...                                                   ...        ...     ...        ...       ...           ...        ...                 ...           ...
282867  Apartment 93, Gladstone House, 190 Strand, Lon...  WC2R 1AB    WC2R         NaN       NaN         180.0        NaN     Flat/Maisonette     3278000.0
282868  Apartment 94, Gladstone House, 190 Strand, Lon...  WC2R 1AB    WC2R         NaN       NaN         152.0  Leasehold     Flat/Maisonette     3592000.0
282869  Apartment 95, Gladstone House, 190 Strand, Lon...  WC2R 1AB    WC2R         NaN       NaN         200.0  Leasehold     Flat/Maisonette     3945000.0
282870                9 Arundel Street, London, WC2R 3DX  WC2R 3DX    WC2R         NaN       NaN           NaN  Leasehold     Flat/Maisonette     2505000.0
282871               11 Arundel Street, London, WC2R 3DX  WC2R 3DX    WC2R         NaN       NaN           NaN  Leasehold     Flat/Maisonette     2616000.0

[282872 rows x 9 columns]
```

At this point I realised that the dataset had some empty values such as those highlighted above. Hence, I wrote this function to remove any NaN (Not a Number) values from the code, and delete the entire row from the dataset:

```
#This function cleans the data
def clean_data(df):
    df = df.dropna()
    return df

df = clean_data(df)
print(df)
```

The dataset also contained a lot of data points (around 300,000), so I added a line of code to reduce the size of the dataset so it would be easier to train:

```
df= df.sample(frac=0.2)
```

My dataset also included string values such as the postcode, property type etc. However, a machine learning model is unable to take these values in, so I wrote another function to one hot encode the data, making a new column for each different postcode.

```
#This function preprocesses the data using one hot encoding
def preprocess_data(df):
    df = pd.get_dummies(df, columns=['postcode','outcode','tenure','propertyType'])
    return df

df = preprocess_data(df)
print(df)
```

Here the one hot encoded table can be seen, now with 40,232 columns instead of 9:

```
                                    fullAddress  bathrooms  ...  propertyType_Terraced  propertyType_Terraced Bungalow
0         1 Guinness Court, Mansell Street, London, E1 8AB        1.0  ...                  False                           False
1       306 Marlyn Lodge, 2 Portsoken Street, London, ...        1.0  ...                  False                           False
2       Flat 1, White Rose Court, Widegate Street, Lon...        2.0  ...                  False                           False
3       Flat 1, White Rose Court, Widegate Street, Lon...        2.0  ...                  False                           False
4       Flat 1, White Rose Court, Widegate Street, Lon...        2.0  ...                  False                           False
...                                                   ...        ...  ...                    ...                             ...
282861  Apartment 88, Gladstone House, 190 Strand, Lon...        1.0  ...                  False                           False
282862  Apartment 89, Gladstone House, 190 Strand, Lon...        2.0  ...                  False                           False
282863  Apartment 90, Gladstone House, 190 Strand, Lon...        2.0  ...                  False                           False
282864  Apartment 91, Gladstone House, 190 Strand, Lon...        1.0  ...                  False                           False
282865  Apartment 91, Gladstone House, 190 Strand, Lon...        1.0  ...                  False                           False

[219724 rows x 40232 columns]
```

However, when running this code, I was presented with this error message:

```
  File "C:\Users\victo\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8
e-packages\pandas\core\dtypes\astype.py", line 133, in _astype_nansafe
    return arr.astype(dtype, copy=True)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: could not convert string to float: '298A Grange Road, Plaistow, London, E13 0HG'
```

I then realised that I needed to drop the full addresses from the table, keeping only the postcode for a location marker. This was necessary as the machine learning model would require the full address to be one hot encoded as well, resulting in thousands of more columns.

```
#Split the data into dependent and independent variables
y = df['currentPrice']
x = df.drop(['currentPrice', 'fullAddress'], axis=1)
```

I then wrote some code to create a training dataset and a testing dataset:

```
#Split the data into training and testing data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
```

I then trained the machine learning model on the dataset, using the testing and training data to predict using the model as well, to obtain test statistics.

```
#This imports the Linear Regression model from the sklearn library
from sklearn.linear_model import LinearRegression

#The training data is then fitted into the Linear Regression model
lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
#The predicted prices are then assigned to the two variables
y_lr_train_pred = lr.predict(X_train)
y_lr_test_pred = lr.predict(X_test)
```

The accuracy of the model is then tested by using statistics measures such as mean squared error and r squared:

```
#The mean squared error function is imported, along with R squared
from sklearn.metrics import mean_squared_error, r2_score

#Both functions are then used with the data to produce values for mean squared error (MSE) and r squared (R2)
lr_train_mse = mean_squared_error(y_train, y_lr_train_pred)
lr_train_r2 = r2_score(y_train, y_lr_train_pred)
lr_test_mse = mean_squared_error(y_test, y_lr_test_pred)
lr_test_r2 = r2_score(y_test, y_lr_test_pred)
```

The same process is repeated for the random forest model:

```
#The Random Forest model is imported from the sklearn library
from sklearn.ensemble import RandomForestRegressor

#The Random Forest model is fitted with the training data
rf = RandomForestRegressor(random_state=100)
rf.fit(X_train, y_train)

#The predicted prices are then assigned to the two variables
y_rf_train_pred = rf.predict(X_train)
y_rf_test_pred = rf.predict(X_test)

print("Done")
#The MSE and R2 values are then calculated using the same function as previously used

rf_train_mse = mean_squared_error(y_train, y_rf_train_pred)
rf_train_r2 = r2_score(y_train, y_rf_train_pred)
rf_test_mse = mean_squared_error(y_test, y_rf_test_pred)
rf_test_r2 = r2_score(y_test, y_rf_test_pred)
```

The test statistics of both machine learning models can then be tested and compared against each other and outputted for testing purposes in the form of a table. I wrote this code to do this:

```
#These values are then outputted in the form of a table
rf_results = pd.DataFrame(['Random forest', rf_train_mse, rf_train_r2, rf_test_mse, rf_test_r2]).transpose()
rf_results.columns = ['Method', 'Training RMSE', 'Training R2', 'Test RMSE', 'Test R2']

#The MSE and R2 values of both models are then outputted together, to allow for comparisons

df_models = pd.concat([lr_results, rf_results], axis=0)
df_models.reset_index(drop=True)
```

This was the outputted table:

	Method	Training MSE	Training R2	Test MSE	Test R2
0	Linear regression	3539123220.686465	0.959887	6774026405.49507	0.937297
0	Random forest	874795981.991425	0.990085	4205354465.333535	0.961073

I then needed to save the trained model that was better so that it can be used later for prediction by the prediction program:

```
# Save the better model
if rf_test_r2 > lr_test_r2:
    with open('model.pkl', 'wb') as f:
        pickle.dump(rf, f)
else:
    with open('model.pkl', 'wb') as f:
        pickle.dump(lr, f)
```

I then re-organised the code into separate functions which is called by the main function:

```
# Main function to execute the workflow
def main():

    # Import the dataset
    df = pd.read_csv('london_house_price_data.csv')

    # Clean the data
    df = clean_data(df)

    # Sample the data
    df = df.sample(frac=0.01)

    # Preprocess the data
    df = preprocess_data(df)

    # Train and evaluate models
    df_models = train_and_evaluate_models(df)

    # Print the results
    print(df_models)
```

Validation and Testing

Validation is not required as there is no user input for this phase of development.

Here is the testing table as outlined in the *Design: Iterative testing method*

Test	Input	Input type	Description of test	Expected outcome	Pass	Evidence
1	Dataset, with the column for price identified	Normal (Only normal applicable for this phase)	Split the data into the x_values and the y_value	The program should be able to split the dataset into two different variables: the price, and all the other components	PASS	*Picture 2.1*
2	Dataset, with the given percentage split	Normal (Only normal applicable for this phase)	Split the data into training and testing data	The program should split this data into the training dataset and the testing dataset at a given percentage split.	PASS	Training dataset: *Picture 2.2a* Testing dataset: *Picture 2.2b*
3	Training dataset	Normal (Only normal applicable for this phase)	Training the first model with the training dataset	The model should be able to be trained based on this data and return some test statistics	PASS	*Picture 2.3*
4	Testing dataset	Normal (Only normal applicable for this phase)	Testing the first model with the testing dataset	The program should produce some statistics based on the performance for this testing data.	PASS	*Picture 2.3*
5	Training dataset	Normal (Only normal applicable for this phase)	Training another model with the training dataset	The program should train another type of machine learning model using the training data and obtain the performance statistics	PASS	*Picture 2.3*
6	Testing dataset	Normal (Only normal applicable for this phase)	Testing another model with the testing dataset	The program should test the second trained model with the unseen testing dataset, and evaluate the performance and return the statistics	PASS	*Picture 2.3*

7	Full dataset	Normal	Compare two models	Program should compare the test statistic of both models and find the better model	PASS	*Picture 2.4*

Review

What has been done

At this stage the machine learning model has now been trained based on the dataset of all the properties. This means that the program is ready for a prediction to be made and the machine learning model to be used.

How it has been tested

This phase of development has been tested according to the phase 2 testing plan outlined in the *Design: Iterative testing method*. All tests have been passed and evidenced, proving that this phase has been completed, and functions as intended.

How it meets success criteria and user expectations

The program has met the following success criteria:

Requirement 8: Machine learning model

1. Trains a machine learning model
2. Tests a machine learning model
3. Compares multiple machine learning models

All user expectations have been met as multiple machine learning models have been trained on the given data.

Changes in design that have resulted from this section

None

Stakeholder Feedback

Stakeholders were happy with both models being used and compared

Summary of the whole project as a prototype at this stage

At this stage, the models have been trained on the data, so the program is ready for creating a predicted price based on the user's inputs. The next step would be to obtain the user's inputs through the use of a GUI.

Phase 3: User Interface

Overview

In this phase of development, the Graphical User Interface would be developed. This UI should allow the user to enter their specific details and see their returned prediction.

This links to Requirement 5: Simple UI and Requirement 1: Information of property as well as the following success criteria:

- *Success criteria 16: "UI is displayed when program starts"*
- *Success criteria 17: "UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties"*
- *Success criteria 18: "User can clearly see which buttons to press in the UI"*

Development

The first step was to import the relevant libraries into python:

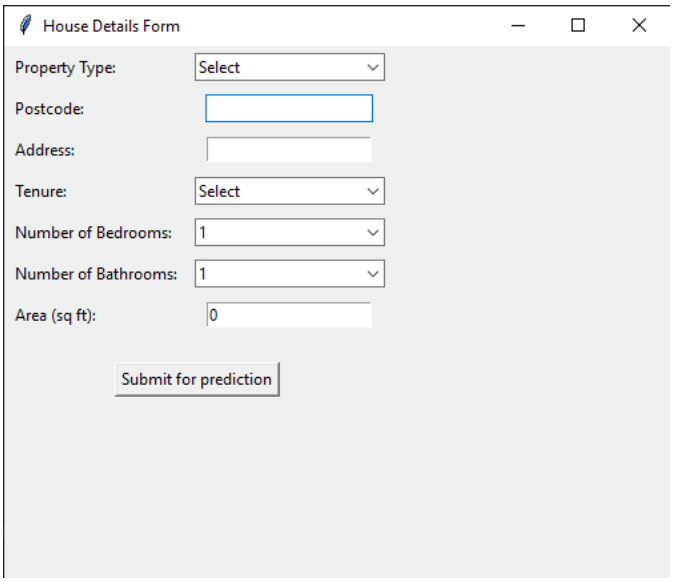
```
import tkinter as tk
from tkinter import ttk, messagebox
```

I then created the window of the UI using this code:

```
# Function to create the main window
def create_main_window():
    root = tk.Tk()
    root.title("House Details Form")
    root.geometry("500x400")
    return root
```



After creating this, I filled the window with the relevant text boxes and drop-down menus:



This was done by adding this function:

```
def create_widgets(root, property_types, tenures):
    # Property Type
    tk.Label(root, text="Property Type:").grid(row=0, column=0, pady=5, padx=5, sticky="w")
    property_type_var = tk.StringVar(value="Select")
    property_type_menu = ttk.Combobox(root, textvariable=property_type_var, values=property_types)
    property_type_menu.grid(row=0, column=1, pady=5, padx=5)

    # Postcode
    tk.Label(root, text="Postcode:").grid(row=1, column=0, pady=5, padx=5, sticky="w")
    postcode_var = tk.StringVar()
    postcode_menu = ttk.Entry(root, textvariable=postcode_var)
    postcode_menu.grid(row=1, column=1, pady=5, padx=5)

    # Address
    tk.Label(root, text="Address:").grid(row=2, column=0, pady=5, padx=5, sticky="w")
    address_var = tk.StringVar()
    address_entry = tk.Entry(root, textvariable=address_var)
    address_entry.grid(row=2, column=1, pady=5, padx=5)

    # Tenure
    tk.Label(root, text="Tenure:").grid(row=3, column=0, pady=5, padx=5, sticky="w")
    tenure_var = tk.StringVar(value="Select")
    tenure_menu = ttk.Combobox(root, textvariable=tenure_var, values=tenures)
    tenure_menu.grid(row=3, column=1, pady=5, padx=5)

    # Number of Bedrooms
    tk.Label(root, text="Number of Bedrooms:").grid(row=4, column=0, pady=5, padx=5, sticky="w")
    bedrooms_var = tk.IntVar(value=1)
    bedrooms_menu = ttk.Combobox(root, textvariable=bedrooms_var, values=list(range(1, 11)))
    bedrooms_menu.grid(row=4, column=1, pady=5, padx=5)
```

```
# Number of Bathrooms
tk.Label(root, text="Number of Bathrooms:").grid(row=5, column=0, pady=5, padx=5, sticky="w")
bathrooms_var = tk.IntVar(value=1)
bathrooms_menu = ttk.Combobox(root, textvariable=bathrooms_var, values=list(range(1, 11)))
bathrooms_menu.grid(row=5, column=1, pady=5, padx=5)

# Area
tk.Label(root, text="Area (sq ft):").grid(row=6, column=0, pady=5, padx=5, sticky="w")
area_var = tk.IntVar()
area_entry = tk.Entry(root, textvariable=area_var)
area_entry.grid(row=6, column=1, pady=5, padx=5)

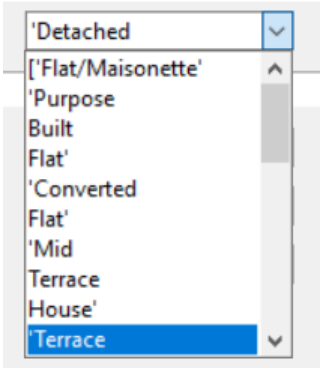
# Submit Button
submit_button = tk.Button(root, text="Submit for prediction", command=lambda: submit_details(root, property_type_var, postco
submit_button.grid(row=7, column=0, columnspan=2, pady=20)
```

At this point, I needed to know what options were available for the user to select. This was because the model will only be trained with certain types of data and hence will not recognise any other values for property type for example. I was able to achieve this by using the unique() function which produces a list of all different values found in the column. I wrote a dedicated function to return a list of all the options:

```
# Function to get options for property types, postcodes, and tenures from the CSV file
def get_options():
    df = pd.read_csv('london_house_price_data.csv')
    df = df.dropna()

    # Get unique values for propertyType, postcode, and tenure
    property_types = df['propertyType'].unique()
    postcodes = df['postcode'].unique()
    tenures = df['tenure'].unique()
```

However, when looking at the UI, the options were formatted weirdly as shown here:



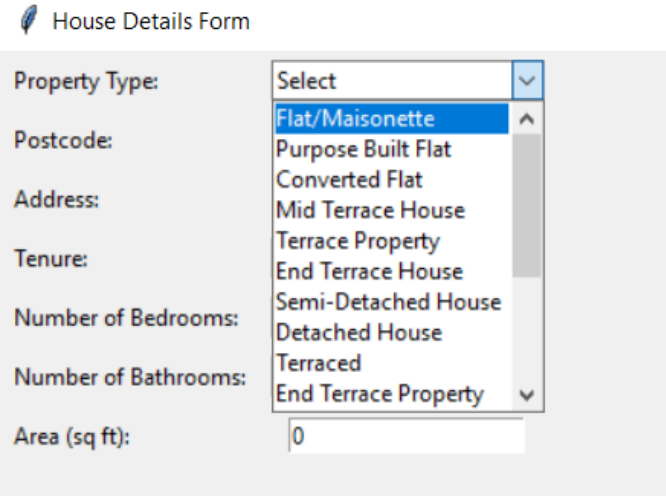
To check what was going wrong, I printed the property_types and this was the output:

```
['Flat/Maisonette' 'Purpose Built Flat' 'Converted Flat'
 'Mid Terrace House' 'Terrace Property' 'End Terrace House'
 'Semi-Detached House' 'Detached House' 'Terraced' 'End Terrace Property
 'Detached Bungalow' 'Semi-Detached Bungalow' 'Bungalow Property'
 'Terraced Bungalow' 'End Terrace Bungalow' 'Mid Terrace Bungalow'
 'Semi-Detached Property' 'Mid Terrace Property' 'Detached Property']
```

Having checked the documentation of the .unique() function I realised that it returns a numpy array instead of a list. So I added these lines of code to convert the numpy array into lists.

```
property_types = property_types.tolist()
postcodes = postcodes.tolist()
tenures = tenures.tolist()
return property_types, postcodes, tenures
```

This then fixed the problem as shown here:

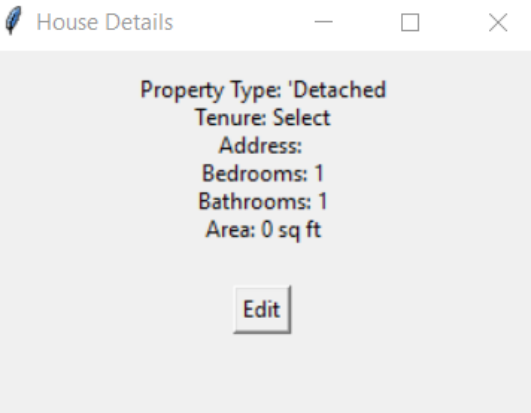


I then wanted to allow the user to be able to edit their changes after submitting their information. This was achieved using this code:

```
def edit_details():
    details_window.destroy()

tk.Button(details_window, text="Edit", command=edit_details).pack(side=tk.LEFT, padx=10, pady=10)
```

This was the resultant window that was outputted to the user:



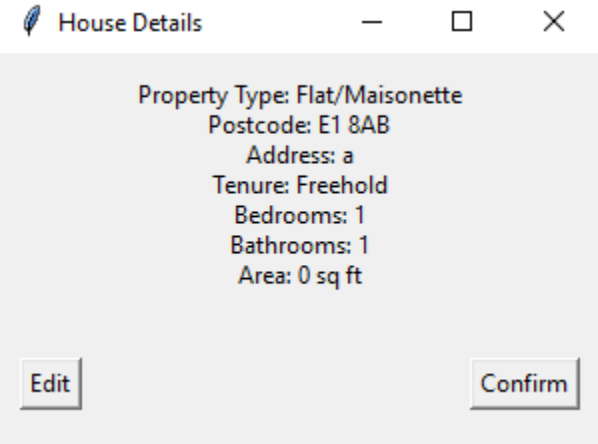
I then wanted the user to be able to confirm their options as well as editing them, so I added another button to allow the user to confirm their choice:

```python
# Function to handle editing of details
def edit_details():
    details_window.destroy()

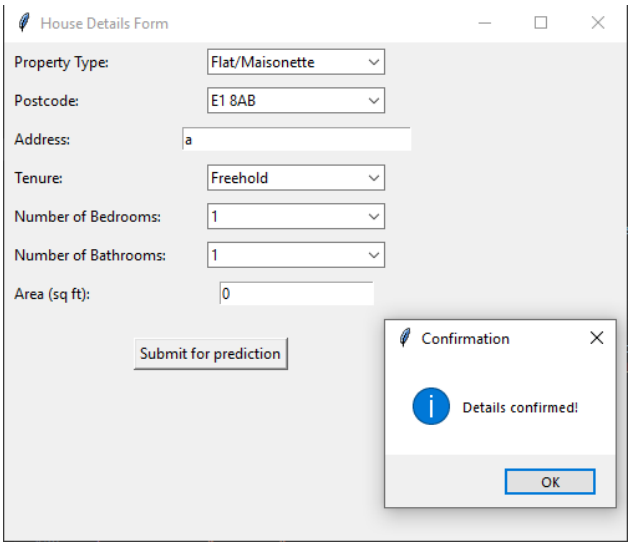
# Function to handle confirmation of details
def confirm_details():
    user_details = [property_type, postcode, address, tenure, bedrooms, bathrooms, area]
    confirmed_details_list.append(user_details)
    messagebox.showinfo("Confirmation", "Details confirmed!")
    details_window.destroy()

# Add Edit and Confirm buttons
tk.Button(details_window, text="Edit", command=edit_details).pack(side=tk.LEFT, padx=10, pady=10)
tk.Button(details_window, text="Confirm", command=confirm_details).pack(side=tk.RIGHT, padx=10, pady=10)
```

The updated buttons can be seen here with both options:



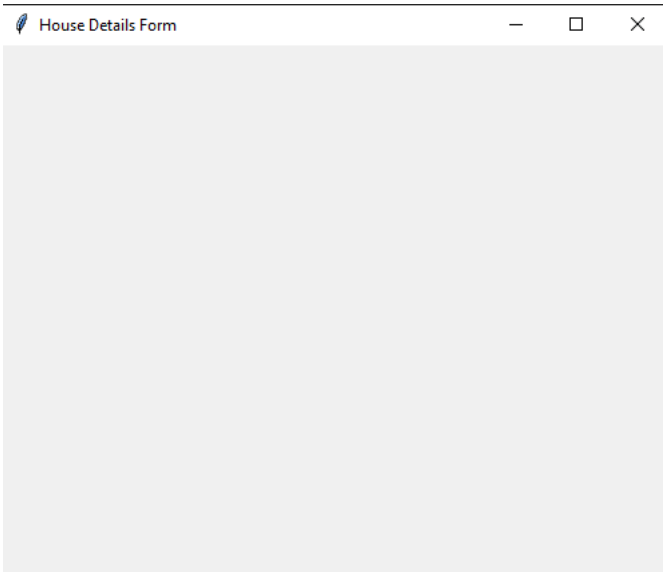
The highlighted code also adds another window to tell the user that their details have been confirmed as seen here:



After the user had confirmed their choice, I wanted a blank UI again to display the predicted price and any relevant information afterwards. So, I had to clear the UI by adding this iterative statement:

```
details_window.destroy()
for widget in root.winfo_children():
    widget.destroy()
```

Which was able to delete all the widgets from the window as shown here:



Having done this, the next step was to display the predicted price once the users data had been sent for the prediction. To do this, I had to write some functions that were able to load and call the prediction function from the other file, and output this prediction into the UI:

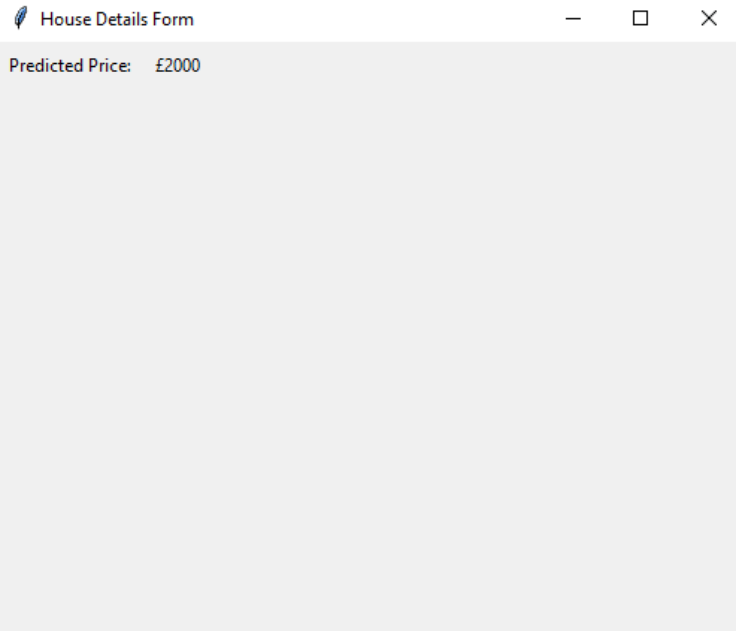
```
# Function to get the predicted price from nea_phase_4.py
def get_predicted_price(user_details):
    # Import the predict_price function from nea_phase_4.py
    spec = importlib.util.spec_from_file_location("nea_phase_4", "nea_phase_4.py")
    nea_phase_4 = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(nea_phase_4)
    return nea_phase_4.predict_price(user_details)

# Function to display the predicted price
def display_prediction(root, predicted_price):
    # Display the predicted price
    tk.Label(root, text="Predicted Price:").grid(row=0, column=0, pady=5, padx=5, sticky="w")
    tk.Label(root, text=f"£{predicted_price}").grid(row=0, column=1, pady=5, padx=5, sticky="w")
```

To check if this was working, I wrote a quick function in the nea_phase_4.py file to return an integer:

```
def predict_price(user_details):
    return 2000
```

I then ran the program, and the correct price was then returned and displayed as shown here:



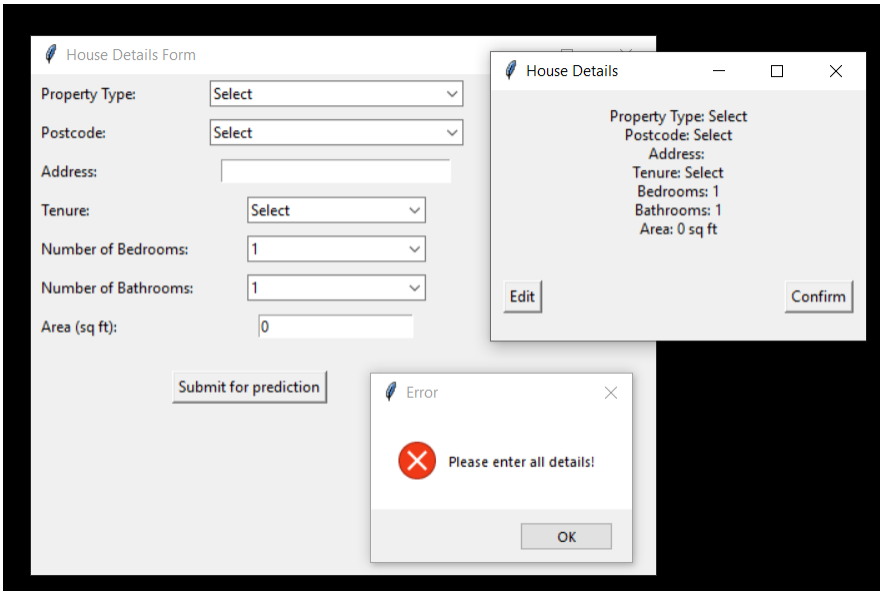
I also realised I needed to write some code to validate the inputs, to ensure there were no empty inputs or invalid inputs. I first wrote some code to take the empty input error:

```
# Function to validate the inputs
def validate_inputs(user_details):
    # Check if all details have been entered
    if "" or "Select" in user_details:
        messagebox.showerror("Error", "Please enter all details!")
        return False
    return True
```

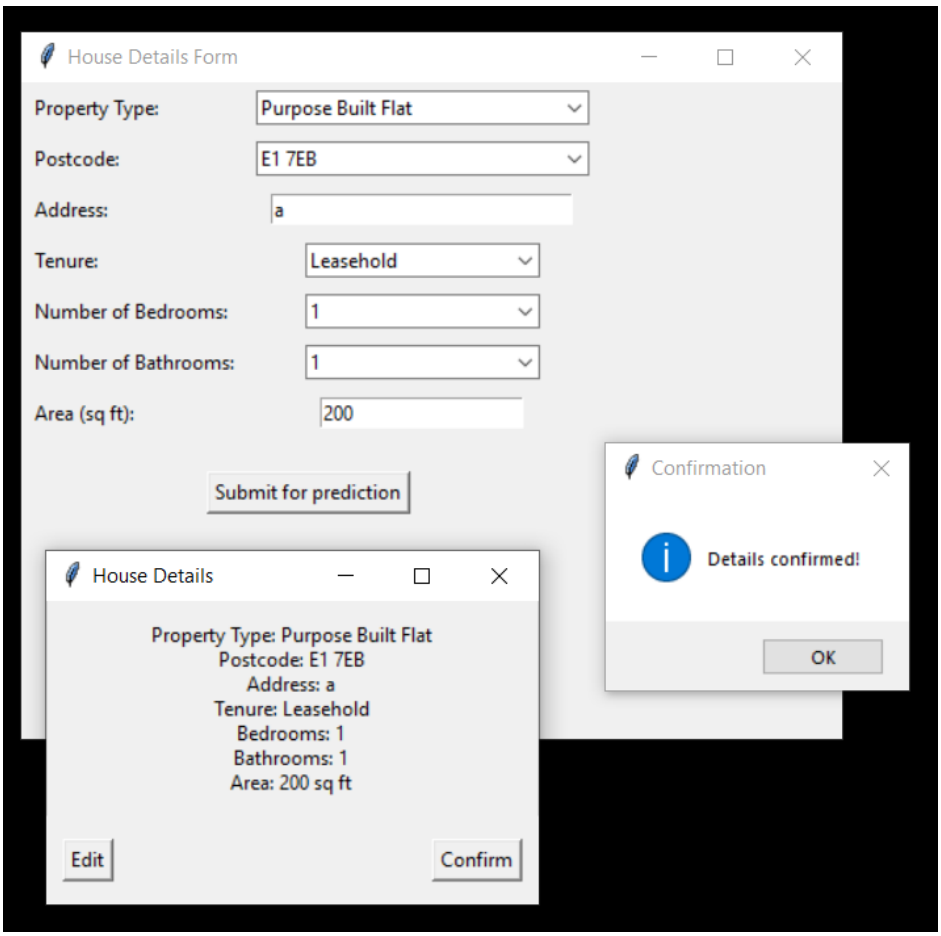
I then called this function within the confirm_details() function, using the selection statements to only confirm once the data was checked to be valid. This modified the original function to this:

```
# Function to handle confirmation of details
def confirm_details():
    user_details = [property_type, postcode, address, tenure, bedrooms, bathrooms, area]
    # Validate the inputs
    if validate_inputs(user_details) == True:
        confirmed_details_list.append(user_details)
        messagebox.showinfo("Confirmation", "Details confirmed!")
        details_window.destroy()
        for widget in root.winfo_children():
            widget.destroy()
        prediction = get_predicted_price(user_details)
        display_prediction(root, prediction)
    # If details are not valid, return to original submission form
    else:
        details_window.destroy()
```

This worked to create the error message as shown here:



But still accepts input when all boxes are filled as shown here:



However, when testing the inputs, and inputting a string for the area, the program crashed and I received the following error message:

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2544.0_x64__qbz5n2kfra8p0\Lib\tkinter\__init__.py", line 595, in get
    return self._tk.getint(value)
           ^^^^^^^^^^^^^^^^^^^^^^
_tkinter.TclError: expected integer but got "a"
```

I then realised that I needed to create a separate function away from the ui.py file that handled all the inputs. This meant that the UI program was solely responsible for displaying inputs. I wrote this function in the phase 4 file:

```
# Function to format data for prediction
def validate_inputs(user_details):
    # Check if property type has been selected
    if user_details[0] == "Select":
        return "type"
    # Check if postcode has been selected
    if user_details[1] == "Select":
        return "postcode"
    # Check if address has been entered
    if user_details[2] == "":
        return "address"
    # Check if tenure has been selected
    if user_details[4] == "Select":
        return "tenure"
    #Check if area is a number
    try:
        area = int(user_details[6])
        if area <= 0:
            return "area"
    except ValueError:
        return "area_type"
    #If all inputs are valid
    else:
        return "valid"
```

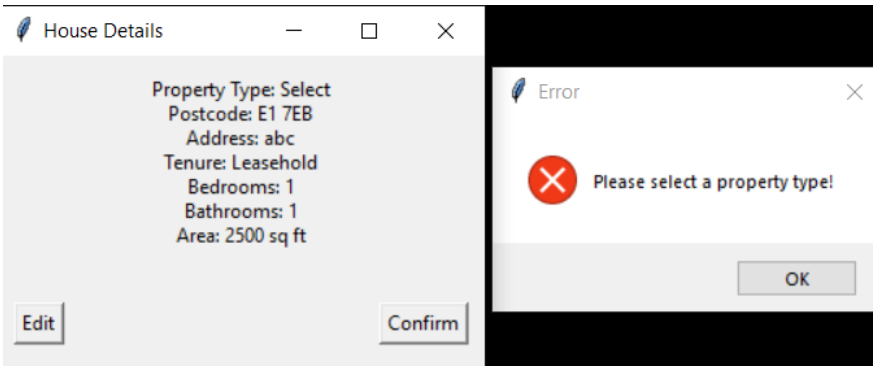
I moved the importing code outside of the get_predicted_price() function so that it could be accessed by the whole file at any time:

```
# Import the predict_price function from phase 4 file
spec = importlib.util.spec_from_file_location("predictPrice", "predictPrice.py")
predictPrice = importlib.util.module_from_spec(spec)
spec.loader.exec_module(predictPrice)
```

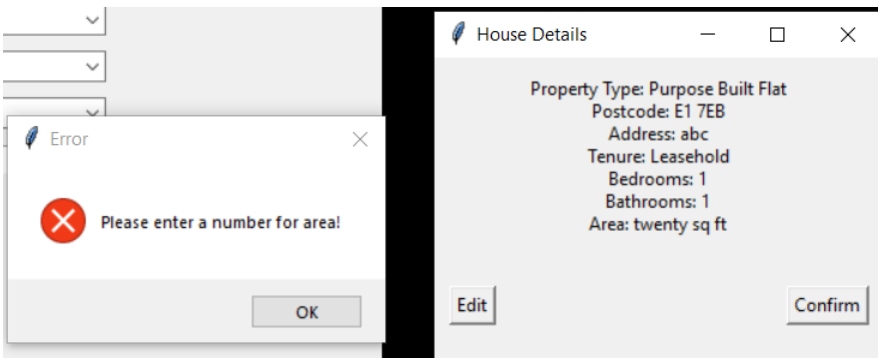
I then wrote some code for the confirmation function to display the corresponding error message for each error as shown:

```python
# Function to handle confirmation of details
def confirm_details():
    user_details = [property_type, postcode, address, tenure, bedrooms, bathrooms, area]
    # Validate the inputs
    validation = nea_phase_4.validate_inputs(user_details)
    if validation == "valid":
        confirmed_details_list.append(user_details)
        messagebox.showinfo("Confirmation", "Details confirmed!")
        details_window.destroy()
        for widget in root.winfo_children():
            widget.destroy()
        prediction = get_predicted_price(user_details)
        display_prediction(root, prediction)
    # If details are not valid, return to original submission form
    elif validation == "address":
        messagebox.showerror("Error", "Please enter your address!")
        details_window.destroy()
    elif validation == "area":
        messagebox.showerror("Error", "Please enter the area!")
        details_window.destroy()
    elif validation == "postcode":
        messagebox.showerror("Error", "Please select a postcode!")
        details_window.destroy()
    elif validation == "tenure":
        messagebox.showerror("Error", "Please select a tenure!")
        details_window.destroy()
    elif validation == "type":
        messagebox.showerror("Error", "Please select a property type!")
        details_window.destroy()
    elif validation == "area_type":
        messagebox.showerror("Error", "Please enter a number for area!")
        details_window.destroy()
```

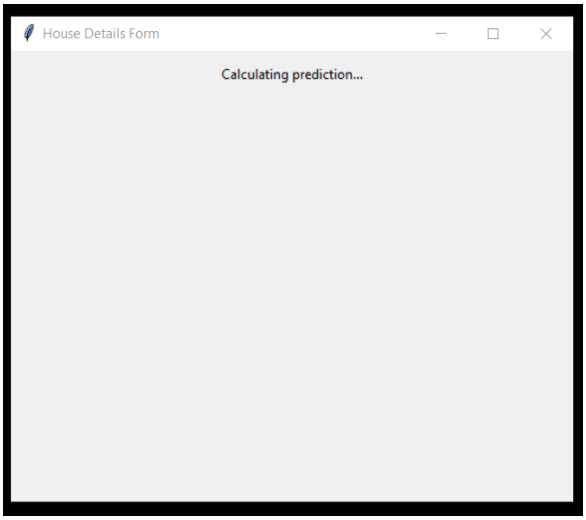
When an invalid input is inputted the appropriate error message appears as shown here for property type not inputted:



And here for a string inputted for area:



I then also wanted to add a loading screen because I felt the wait between the confirmation of the input and the output of the prediction was too slow. I initially just tried to clear the window and add some text as shown:



But this resulted in this error being thrown, as both the predicted price and the loading screen were trying to occupy the same space in the window.

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2544.0_x64__qbz5n2kfra8p0\Lib\tkinter\__init__.py", line 1968, in __call__
    return self.func(*args)
           ^^^^^^^^^^^^^^^^
  File "c:\Users\victo\OneDrive - King Edward VI Grammar School\Documents\Computer Science\NEA\NEA Code\ui1.py", line 136, in confirm_details
    display_prediction(root, prediction)
  File "c:\Users\victo\OneDrive - King Edward VI Grammar School\Documents\Computer Science\NEA\NEA Code\ui1.py", line 169, in display_prediction
    tk.Label(root, text="Predicted Price:").grid(row=0, column=0, pady=5, padx=5, sticky="w")
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2544.0_x64__qbz5n2kfra8p0\Lib\tkinter\__init__.py", line 2580, in grid_configure
    self.tk.call(
_tkinter.TclError: cannot use geometry manager grid inside . which already has slaves managed by pack
```

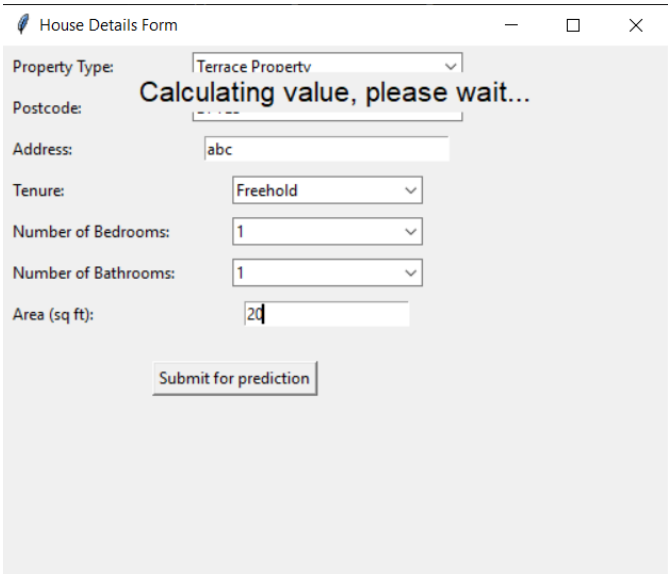
I then tried adding this code to remove the loading screen when the prediction was ready:

```
# Clear the loading screen
for widget in root.winfo_children():
    widget.destroy()
```

However, this resulted in the loading screen not being displayed at all, so I then tried using a label to display the loading message:

```
# Show loading screen on the main window
loading_label = tk.Label(root, text="Calculating value, please wait...", font=("Helvetica", 16))
loading_label.pack(pady=20, padx=20)
root.update_idletasks()
```

With the output shown here:

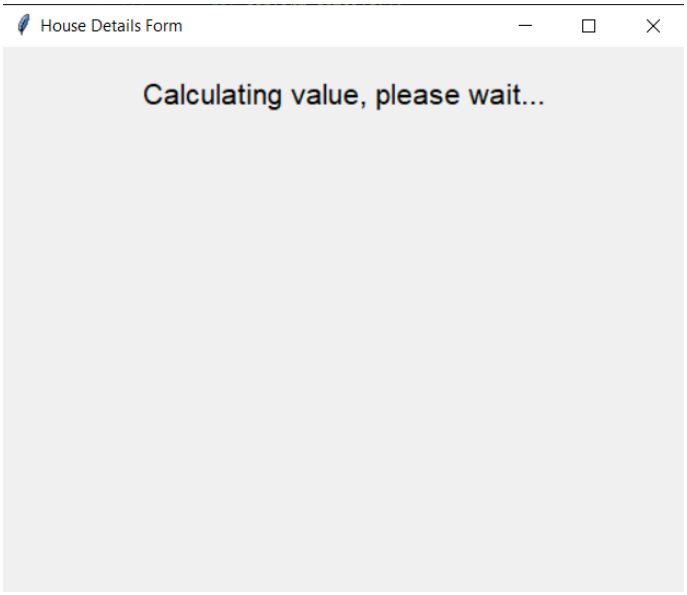


However, for some reason the information behind the loading message was not clearing. I realised that this was because the Tkinter library in python did not constantly refresh the GUI, so I had to manually update it by adding the line below:

```
# Clear the main window
for widget in root.winfo_children():
    widget.destroy()

# Refresh the GUI so everything is deleted
root.update()
```

This solved the problem as shown below:

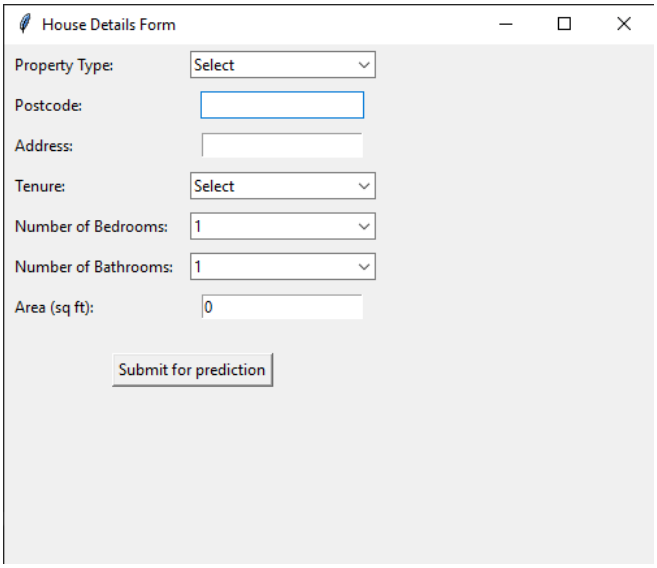


Testing

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	N/A	Does UI pop up when program runs	When program runs the UI should be displayed	PASS	*Video 3.1*
2	Sample data to enter for prediction	Does program allow for data to be entered	The UI should allow the user to enter all the information required for the program to run as required.	PASS	*Picture 3.2*
3	Sample data to enter for prediction	Is information visible and easily accessible for user	UI should display large enough text for user to clearly see options	FAIL	See below

Correcting Errors

This failure in testing was due to the fact that the UI was not displayed clearly enough and could be difficult for the user to understand. An example is shown below:



To change this, I added code to open a full-sized window and a function which changes the size of the text to a larger size based on the size of the window. The code can be seen here:

```
# Function to create the main window
def create_main_window():
    root = tk.Tk()
    root.title("House Details Form")
    root.state("zoomed")  # Maximize the window
    root.configure(bg="#f0f0f0")  # Light grey background
    root.bind("<Configure>", lambda event: adjust_font_size(root))
    return root

# Function to adjust font size dynamically
def adjust_font_size(root):
    width = root.winfo_width()
    height = root.winfo_height()
    new_size = max(10, int(min(width, height) / 50))
    global font_style
    font_style = ("Helvetica", new_size)
    for widget in root.winfo_children():
        if isinstance(widget, (tk.Label, tk.Button, ttk.Combobox, tk.Entry)):
            widget.config(font=font_style)
```

Testing again

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	N/A	Does UI pop up when program runs	When program runs the UI should be displayed	PASS	*Video 3.1*
2	Sample data to enter for prediction	Does program allow for data to be entered	The UI should allow the user to enter all the information required for the program to run as required.	PASS	*Picture 3.2*
3	Sample data to enter for prediction	Is information visible and easily accessible for user	UI should display large enough text for user to clearly see options	PASS	*Video 3.3*

Review

What has been done

The UI has been made which allows the user to input their information. This will then be returned into the prediction program for a prediction in phase 4.

How it has been tested

The program has been tested according to the phase 3 testing tables outlined in the design section. All tests have been passed after some corrections and evidenced appropriately. Therefore, the phase can be marked as complete.

How it meets success criteria and user expectations

The following success criteria have been met:

Requirement 5: Simple UI

1. UI is displayed when program starts
2. UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties
3. User can clearly see which buttons to press in the UI

Changes in design that have resulted from this section

Slight changes were made to the initial design of the UI. This included removing boxes surrounding the different headings and instead using columns in the page to title the different sections. The colour scheme was not implemented as shown in the design, as I found the simple black and grey colour scheme simple enough to be effective. Potentially, the blue colour scheme outlined in *Design: UI Design – Visualisation* can be implemented in future developments.

Stakeholder Feedback

All stakeholders were happy with the current progress in the UI.

Summary of the whole project as a prototype at this stage

At this stage, the program is ready to calculate a prediction, as the user is able to enter their details to be predicted. The model has been trained, so should be able to quickly predict a price.

Phase 4: Calculating a prediction

Overview

In this phase of development, the trained machine learning model will be used to predict a price based on the inputs given by the user. The data needs to be processed and formatted so it can be plugged directly into the machine learning model.

This links to Requirement 3: Predicted Price as well as the following success criteria:

- *Success criteria 12: "Calculates and displays a predicted price using a machine learning model"*
- *Success criteria 13: "The predicted price should be as accurate as possible"*

Development

I started by first loading the model that was trained using the pickle load function as shown below. This allowed the trained model in phase 2 to be used for the prediction in this phase of development.

```
# Function to return a predicted price
def predict_price(user_details):
    # Open the machine learning model
    with open('model.pkl', 'rb') as file:
        # Call load method to deserialze
        model = pickle.load(file)
```

I then wrote a function that would return the headings of the dataset. This meant that later on in development, I could create a dictionary with all the columns that the machine learning model was trained to and adjust and amend the appropriate values.

```
def format_data():
    # Get headings of dataset
    headings = df.columns.values.tolist()
    print(headings)
```

The output of all the headings of the columns can be seen here:

```
['fullAddress', 'bathrooms', 'bedrooms', 'floorAreaSqM', 'currentPrice', 'postcode_E1 1PP', 'postcode_E1 4AR', 'postcode_E1 6LW',
'postcode_E1 6NG', 'postcode_E10 5BU', 'postcode_E10 5LT', 'postcode_E10 6ND', 'postcode_E10 6NF', 'postcode_E10 6QB', 'postcode_
10 7LD', 'postcode_E11 2LA', 'postcode_E11 2RH', 'postcode_E11 3DN', 'postcode_E13 0AZ', 'postcode_E14 3ER', 'postcode_E14 8AY',
postcode_E15 3QR', 'postcode_E17 5AU', 'postcode_E17 6QW', 'postcode_E17 7LG', 'postcode_E18 2BT', 'postcode_E18 2FE', 'postcode_
18 2NJ', 'postcode_E2 6BJ', 'postcode_E2 8GF', 'postcode_E3 5BT', 'postcode_E6 3PE', 'postcode_E7 8HZ', 'postcode_E8 3LT', 'postc
de_E9 5EG', 'postcode_E9 6FJ', 'postcode_E9 7DB', 'postcode_EC1A 9LS', 'postcode_EC1V 0BQ', 'postcode_EC2Y 8DN', 'postcode_EC2Y 8
D', 'postcode_EC4A 1BX', 'postcode_EC4V 3PA', 'postcode_EC4V 3PQ', 'postcode_EC4Y 0DD', 'postcode_N1 2EU', 'postcode_N1 3JD', 'pc
tcode_N1 5EN', 'postcode_N1 7SZ', 'postcode_N10 1NU', 'postcode_N10 2HX', 'postcode_N11 1RL', 'postcode_N11 2TR', 'postcode_N11 3
S', 'postcode_N12 0DE', 'postcode_N13 5PW', 'postcode_N14 6AH', 'postcode_N15 5QA', 'postcode_N15 5RP', 'postcode_N16 0ES', 'post
ode_N16 5BB', 'postcode_N16 8QR', 'postcode_N16 9NP', 'postcode_N17 7NY', 'postcode_N17 8JX', 'postcode_N17 9PL', 'postcode_N18 1
X', 'postcode_N18 1QQ', 'postcode_N18 2QG', 'postcode_N2 8BS', 'postcode_N20 9DZ', 'postcode_N21 2SD', 'postcode_N21 3DU', 'postc
de_N21 3SB', 'postcode_N22 6BS', 'postcode_N22 6HT', 'postcode_N22 6QD', 'postcode_N22 8JL', 'postcode_N22 8PJ', 'postcode_N22 8X
', 'postcode_N4 1DU', 'postcode_N6 4BY', 'postcode_N6 5NH', 'postcode_N6 6EB', 'postcode_N8 0NL', 'postcode_N8 8QT', 'postcode_NW
 6EJ', 'postcode_NW1 6TD', 'postcode_NW1 9SG', 'postcode_NW10 5UA', 'postcode_NW10 9NU', 'postcode_NW10 9UT', 'postcode_NW11 8AE'
 'postcode_NW11 8EH', 'postcode_NW11 9PR', 'postcode_NW4 4LU', 'postcode_NW5 4EF', 'postcode_NW5 4JL', 'postcode_NW6 1JG', 'postc
de_NW6 7XY', 'postcode_NW7 1EA', 'postcode_NW7 1FX', 'postcode_NW7 3PR', 'postcode_NW8 8UN', 'postcode_SE1 2LJ', 'postcode_SE1 7
```

The next step was to use the list of all the headings and create a dictionary, with an initial value of 0 for all the headings. This was because the one hot encoding meant the majority of my values were 0, apart from the specific postcodes and property addresses. The training dataset also did not include the final price or the address, so I deleted them from the dictionary as shown below:

```
def format_data():
    # Get headings of dataset
    headings = df.columns.values.tolist()

    # Create a dictionary with all headings and default values
    new_property = {category: "0" for category in headings}
    del new_property['currentPrice']
    del new_property['fullAddress']
```

The training dataset also included an outcode, which was the first part of the postcode. In order to extract this, I needed to write some code to split the user entered postcode into the outcode and the rest of the postcode. This can be seen working below:

```
    # Get the outcode from the postcode
    outcode = user_details[1].split(" ")[0]
    print(outcode)

format_data(["Detached", "SW1A 1AA", "10 Downing Street", "Freehold", "1", "1", "100"])
```

```
/Microsoft/WindowsApps/python3.
Code/nea_phase_4.py"
SW1A
```

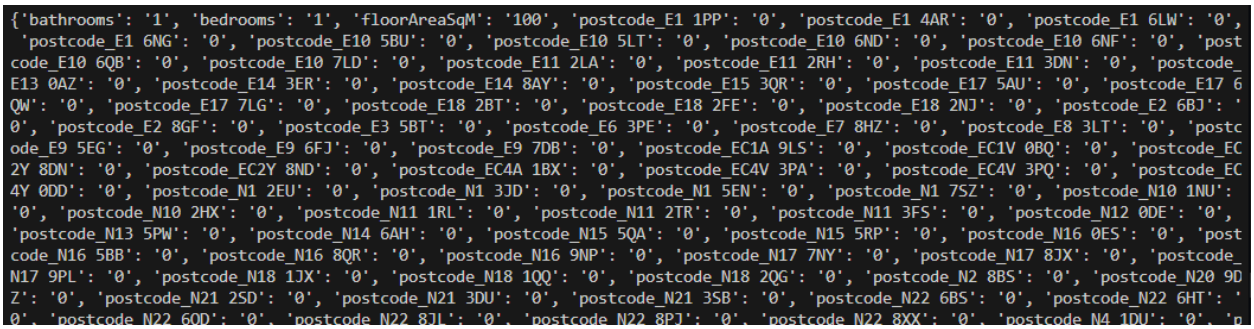
After extracting all the necessary data for the prediction, I updated the original dictionary, modifying the values according to the user inputs. The property type, postcode, outcode and tenure were concatenated in the same format as found in the dataset as seen below:

```
# Update the dictionary with the user's inputs
new_property.update(
    {
        "propertyType_" + user_details[0]: "1",
        "postcode_" + user_details[1]: "1",
        "tenure_" + user_details[3]: "1",
        "bedrooms": user_details[4],
        "bathrooms": user_details[5],
        "floorAreaSqM": user_details[6],
        "outcode_" + outcode: "1"

    }
)

print(new_property)
```

The dictionary was outputted for testing purposes and can be seen below. The user values have been inputted appropriately:



I then wrote some code to call the format_data function and use the loaded model to predict a price. This price was then rounded to be a recognisable price. This was then returned and when called in the UI file, then predicted price would be displayed.

```
new_property = format_data(user_details)

# Predict the price
predicted_price = model.predict(pd.DataFrame(new_property, index=[0]))
predicted_price = round(predicted_price[0], 2)

return predicted_price
```

To test that the code was functioning, I called the function with some test values:

```
print(predict_price(["Semi-Detached House", "E10 5LT", "1 Test Street", "Freehold", 3, 2, 100]))
```

The price was outputted as shown below:

```
/Microsoft/WindowsApps/python3.12.exe "c:/User
Code/nea_phase_4.py"
852000.0
PS C:\Users\victo\OneDrive - King Edward VI Gi
```

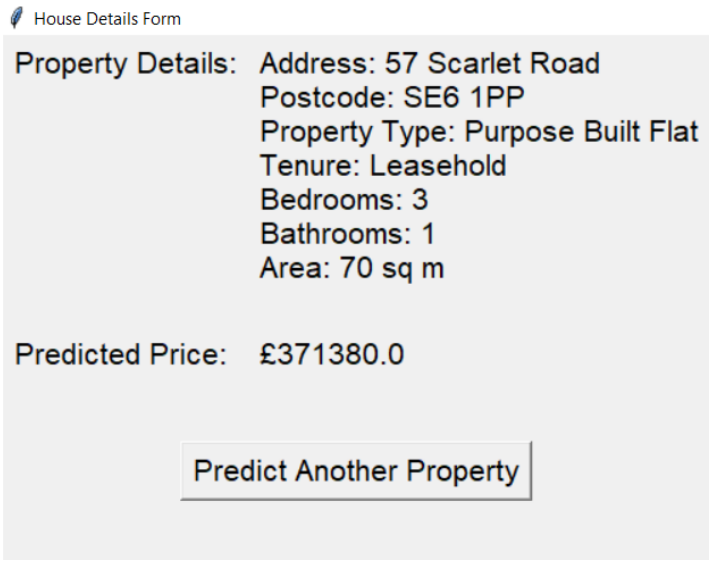
I then tested the code properly, running from the UI file, and inputting all the data. I wrote this code to display the predicted price:

```python
# Function to display the predicted price
def display_prediction(root, predicted_price, user_details):
    # Display the details about the users property
    tk.Label(root, text="Property Details:",).grid(row=0, column=0, pady=5, padx=5, sticky="nw")
    details = (
        f"Address: {user_details[2]}\n"
        f"Postcode: {user_details[1]}\n"
        f"Property Type: {user_details[0]}\n"
        f"Tenure: {user_details[3]}\n"
        f"Bedrooms: {user_details[4]}\n"
        f"Bathrooms: {user_details[5]}\n"
        f"Area: {user_details[6]} sq m"
    )

    tk.Label(root, text=details, anchor="w", justify="left").grid(row=0, column=1, pady=5, padx=5, sticky="w")


    # Display the predicted price
    tk.Label(root, text="Predicted Price:").grid(row=1, column=0, pady=5, padx=5, sticky="w")
    tk.Label(root, text=f"£{predicted_price}").grid(row=1, column=1, pady=5, padx=5, sticky="w")
```

It then outputted a price as shown here:



Testing

The testing method outlined in the design section for phase 4 has been followed:

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	Sample data for prediction	Does the program output a predicted price	The program should output a valid predicted price based on the inputs	PASS	*Picture 4.1*
2	Data already in the dataset (or valid existing data on market)	Does the program predict an accurate price that matches the market	The program should output a price that is comparable to the price listed in the real-world market	PASS	*Picture 4.2*

Review

What has been done

The machine learning model previously trained in Phase 2 has been used for a prediction where the predicted price is outputted to the user through the UI.

How it has been tested

The testing method outlined in the *Design* section has been followed and carried out. All sections have passed the tests.

How it meets success criteria and user expectations

The following success criteria have been met:

Requirement 3: Predicted price

1. Calculates and displays a predicted price using a machine learning model
2. The predicted price should be as accurate as possible

Changes in design that have resulted from this section

No changes.

Stakeholder Feedback

All stakeholders were happy with the program being able to output a prediction.

Summary of the whole project as a prototype at this stage

At this stage, the program has already achieved the primary aim of calculating a prediction and is ready to be used to identify similar properties.

Phase 5: Presenting similar properties

Overview

In this phase of development, similar real-world properties to the property entered for prediction by the user will be displayed. The program should prioritise location of the property, as well as the property type and tenure. It should also display the closest property to the user's entry if there are no exact copies.

This links to Requirement 6: Similar properties as well as the following success criteria:

- *Success criteria 19: "Program identifies similar properties"*
- *Success criteria 20: "The properties share similar characteristics"*

Development

This phase of development revolved heavily around the location of the property, so the first step was to identify any properties in a similar location. The first step to achieve this was to extract a list of all postcodes from the list of properties. I wrote a function to carry out this task as shown below:

```
# Function to get all unique postcodes from the dataset
def get_postcodes():
    df = pd.read_csv('london_house_price_data.csv')
    df = df.dropna()
    postcodes = df['postcode'].unique()
    return postcodes
```

This function returned a list of all postcodes within the dataset, as seen here:

```
ard VI Grammar School/Documents/Computer Science/NEA/NEA Code/nea_ph
['E1 8AB' 'E1 8RB' 'E1 7ES' ... 'WC2R 1BA' 'WC2R 3DX' 'WC2R 1AB']
```

The next step was to extract the outcode (first half of postcode) and the sector (outcode + first character of second half). This would be needed as the location of similar properties would be based around this. The two functions can be seen here:

```
# Function to extract the outcode from a postcode
def get_outcode(postcode):
    return postcode.split(" ")[0]

# Function to extract the sector from a postcode
def get_sector(postcode):
    return postcode.split(" ")[0] + " " + postcode.split(" ")[1][0]
```

I then needed to extract all the properties from the dataset which had the same sector as the one entered by the user. To do this efficiently, I first wrote a merge sort algorithm to sort the dataset by the postcode. The flowchart for this sort algorithm is shown in the design section (*Design: Algorithms*). The code for this can be seen here:

```python
# Function to sort the dataset by postcode using merge sort
def merge_sort(dataset, factor):
    if len(dataset) > 1:
        mid = len(dataset) // 2
        left_half = dataset[:mid]
        right_half = dataset[mid:]

        # Recursive call on each half
        merge_sort(left_half, factor)
        merge_sort(right_half, factor)

        i = j = k = 0

        # Copy data to temp arrays left_half[] and right_half[]
        while i < len(left_half) and j < len(right_half):
            if left_half[i][factor] < right_half[j][factor]:
                dataset[k] = left_half[i]
                i += 1
            else:
                dataset[k] = right_half[j]
                j += 1
            k += 1

        while i < len(left_half):
            dataset[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            dataset[k] = right_half[j]
            j += 1
            k += 1
    # Return the sorted dataset
    return dataset
```

I then needed to search for the specific location in question. To do this I used a binary search to first find an instance when there was a property in that sector. This was based on the algorithm outlined in the design section (*Design: Algorithms*). The code for the binary search can be seen here:

```python
# Function to find postcode with the same sector
def binary_search(dataset, postcode):
    left = 0
    right = len(dataset) - 1
    # Binary search
    while left <= right:
        mid = left + (right - left) // 2
        # Check if postcode is in sector by using starts with
        if dataset[mid][1].startswith(postcode):
            return mid
        elif dataset[mid][1] < postcode:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

The next step was to extract all the properties that where in the same sector. As the dataset had already been sorted using merge sort, I simply had to use linear search to find the lower and upper bounds before slicing the dataset to return. This algorithm was again described in the design section. (*Design: Algorithms*). The code can be seen here:

```
# Function to find all properties with the same sector after binary search
def find_contiguous(dataset, postcode, start):
    left = start
    right = start
    while left >= 0 and dataset[left][1].startswith(postcode):
        left -= 1
    while right < len(dataset) and dataset[right][1].startswith(postcode):
        right += 1
    # Return the dataset with the sliced values
    return dataset[left + 1:right]
```

To call all the functions in the correct order, and pass the corresponding parameters correctly, I then wrote another function which returned the full dataset. This can be seen here:

```
# Main function to find all properties with the same sector
def similar_dataset(postcode):
    # Read the dataset, removing duplicates and Nan values
    df = pd.read_csv('london_house_price_data.csv')
    df = df.dropna()
    df = remove_duplicates(df)

    # Convert dataframe to list and sort by postcode
    df_list = df.values.tolist()
    df_list = merge_sort(df_list, 1)

    # Use sector to find similarly located properties
    sector = get_sector(postcode)
    start_value = binary_search(df_list, sector)
    if start_value is False:
        return False
    df = find_contiguous(df_list, sector, start_value)

    # Convert list back to dataframe
    df = pd.DataFrame(df, columns=['fullAddress', 'postcode', 'outcoc
    return df
```

I tested this function with the postcode “SE4 1DT” and the output was correct as shown:

	fullAddress	postcode	outcode	propertyType	currentPrice	tenure	bedrooms	bathrooms	floorAreaSqM
0	5 Manwood Road, London, SE4 1AA	SE4 1AA	SE4	2.0	5.0	119.0	Freehold	Mid Terrace House	908000.0
1	85A Manwood Road, London, SE4 1AB	SE4 1AB	SE4	1.0	2.0	69.0	Leasehold	Converted Flat	437000.0
2	79 Manwood Road, London, SE4 1AB	SE4 1AB	SE4	1.0	3.0	106.0	Freehold	Terrace Property	794000.0
3	65 Manwood Road, London, SE4 1AB	SE4 1AB	SE4	1.0	3.0	110.0	Freehold	Mid Terrace House	798000.0
4	59 Manwood Road, London, SE4 1AB	SE4 1AB	SE4	1.0	3.0	112.0	Freehold	Terrace Property	785000.0
..	...	...	...	...	...	...	...	...	...
440	1 Adelaide Avenue, London, SE4 1YT	SE4 1YT	SE4	1.0	2.0	71.0	Leasehold	Flat/Maisonette	419000.0
441	38B St Margarets Road, Crofton Park, London, S...	SE4 1YU	SE4	1.0	1.0	47.0	Leasehold	Flat/Maisonette	427000.0
442	50 Chalsey Road, London, SE4 1YW	SE4 1YW	SE4	1.0	4.0	148.0	Freehold	Mid Terrace House	1348000.0
443	36 Chalsey Road, London, SE4 1YW	SE4 1YW	SE4	1.0	4.0	119.0	Freehold	End Terrace House	1299000.0
444	30 Chalsey Road, London, SE4 1YW	SE4 1YW	SE4	1.0	2.0	75.0	Leasehold	Purpose Built Flat	614000.0

[445 rows x 9 columns]

After obtaining a dataset of all properties in a similar location, I then needed to retrieve properties which had similar characteristics as the property entered by the user. I wrote functions that used a try and except method to return a dataset with only matching characteristics. The code can be seen here:

```
# Function to find properties with matching type
def matching_type(type, dataset):
    try:
        mod_dataset = dataset[dataset['propertyType'] == type]
        return mod_dataset
    except KeyError:
        return dataset

# Function to find properties with matching tenure
def matching_tenure(tenure, dataset):
    try:
        mod_dataset = dataset[dataset['tenure'] == tenure]
        return mod_dataset
    except KeyError:
        return dataset

# Function to find properties with matching bedrooms
def matching_bedrooms(bedrooms, dataset):
    try:
        mod_dataset = dataset[dataset['bedrooms'] == bedrooms]
        return mod_dataset
    except KeyError:
        return dataset

# Function to find properties with matching bathrooms
def matching_bathrooms(bathrooms, dataset):
    try:
        mod_dataset = dataset[dataset['bathrooms'] == bathrooms]
        return mod_dataset
    except KeyError:
        return dataset
```

However, when I entered data that did not have a corresponding match, it returned just an empty data frame as shown below.

```
Empty DataFrame
Columns: [fullAddress, postcode, outcode, bathrooms, bedrooms, floorAreaSqM, tenure, propertyType, currentPrice]
Index: []
```

Hence, I rewrote all the functions to use an if statement instead, checking if the dataset was empty or not. If it was not empty, I would return it, otherwise I would simply return the dataset that was passed into it. The modified functions can be seen here:

```python
# Function to find properties with matching type
def matching_type(type, dataset):
    mod_dataset = dataset[dataset['propertyType'] == type]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset

# Function to find properties with matching tenure
def matching_tenure(tenure, dataset):
    mod_dataset = dataset[dataset['tenure'] == tenure]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset

# Function to find properties with matching bedrooms
def matching_bedrooms(bedrooms, dataset):
    mod_dataset = dataset[dataset['bedrooms'] == bedrooms]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset

# Function to find properties with matching bathrooms
def matching_bathrooms(bathrooms, dataset):
    mod_dataset = dataset[dataset['bathrooms'] == bathrooms]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset
```

For the function to return properties with a similar size, I realised that not all properties would be exactly matching in square meters. Hence, I created a range of +/- 20 which I classed as similar in area.

```python
# Function to find properties with similar size
def similar_size(size, dataset):
    mod_dataset = dataset[(dataset['floorAreaSqM'] >= size - 20) & (dataset['floorAreaSqM'] <= size + 20)]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset
```

I then wrote a main function which called all the other functions and passed in the correct information to obtain a dataset of similar properties. It would then return these properties when called. The function can be seen here:

```python
# Function to find similar properties based on user details
def find_similar(user_details):
    dataset = similar_dataset(user_details[1])
    if dataset is False:
        return False
    # Find matching properties based on user details
    else:
        dataset = matching_type(user_details[0], dataset)
        dataset = matching_tenure(user_details[3], dataset)
        dataset = matching_bedrooms(user_details[4], dataset)
        dataset = matching_bathrooms(user_details[5], dataset)
        dataset = similar_size(user_details[6], dataset)
    return dataset
```

I then tested the function passing in some data as shown:

```python
# Example usage
print(find_similar(["Purpose Built Flat", "SW1A 1JJ", "10 Downing Street", "Leasehold", 3, 2, 230]))
```

However, I came across an error as shown below.

```
  File "C:\Users\victo\OneDrive - King Edward VI Grammar School\Documents
    mod_dataset = dataset[(dataset['floorAreaSqM'] >= size - 20) & (datas
                                                    ~~~~~~~~^~~~~
TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

I realised that this was because the size in the dataset was read as a string, which then caused problems when comparing it with the size submitted by the user. After correcting the error by casting the size to be an integer, the program was able to output similar properties as shown below:

	fullAddress	postcode	outcode	bathrooms	bedrooms	floorAreaSqM	tenure	propertyType	currentPrice
213307	Flat 1, 24 St James's Place, London, SW1A 1NH	SW1A 1NH	SW1A	3.0	3.0	248.0	Leasehold	Purpose Built Flat	6355000.0

I then tested the program again, this time with data that didn't have an exact match in the dataset:

```python
# Example usage
print(find_similar(["Purpose Built Flat", "E1 7FB", "10 Downing Street", "Leasehold", 3, 2, 230]))
```

The program was able to find and output the most similar properties as shown here:

	fullAddress	postcode	outcode	bathrooms	bedrooms	floorAreaSqM	tenure	propertyType	currentPrice
211	Flat 21, Bernard House, Toynbee Street, London...	E1 7NR	E1	2.0	3.0	107.0	Leasehold	Purpose Built Flat	632000.0
1633	Flat 21, Sloane Apartments, 54 Old Castle Stre...	E1 7AJ	E1	2.0	3.0	88.0	Leasehold	Purpose Built Flat	843000.0

After testing and making sure that the similar properties function was working correctly, I then wrote some code to output these properties onto the main UI.

The first step was to import the similar properties file into the UI file so that the required functions would be accessible. The code for this was very similar to the code required to import the prediction file in phase 4. This code can be seen here:

```
# Import the similar properties function from phase 5 file
spec2 = importlib.util.spec_from_file_location("similarProperties", "similarProperties.py")
similarProperties = importlib.util.module_from_spec(spec2)
spec2.loader.exec_module(similarProperties)
```

The next step was to create a button for the user to signal that they wanted similar properties. This was done by adding a button after displaying the predicted price. This code can be seen here:

```
# Button to find similar properties
similar_button = tk.Button(root, text="Find Similar Properties", command=lambda: find_similar_properties(root))
similar_button.grid(row=1, column=0, columnspan=2, pady=20)
```

I then created the similar properties function, retrieving the users details from the global list and calling the function from the similar properties file as shown:

```
# Function to find similar properties
def find_similar_properties(root):
    # Get the user details
    user_details = confirmed_details_list[0]

    # Find similar properties
    similar_properties = nea_phase_5.find_similar(user_details)
```

The next step was to create the window from which the similar properties could be viewed. If there were no similar properties, a pop up would be created telling the user that there were no properties in their area.

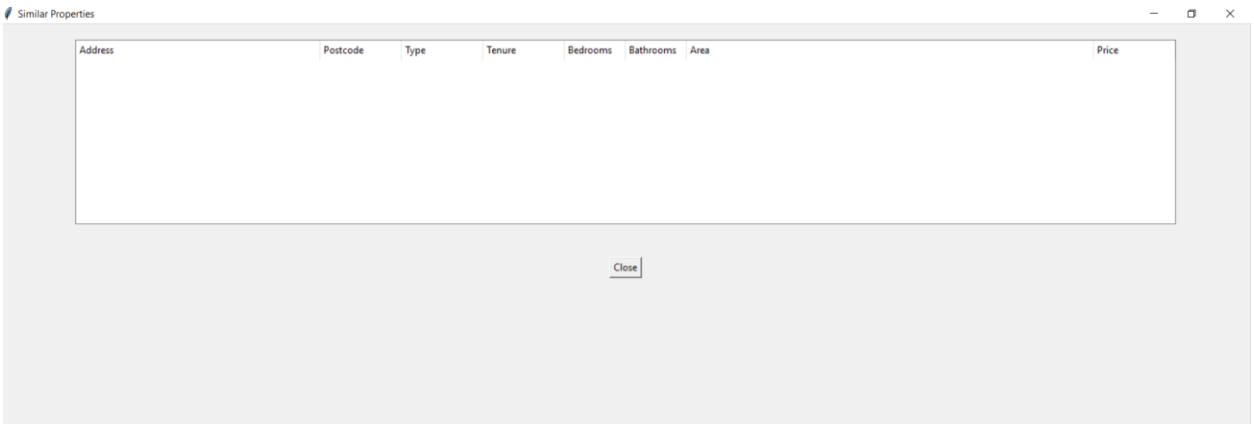
```
# Check if there are similar properties
if similar_properties is False:
    messagebox.showinfo("No Similar Properties", "No similar properties found!")
else:
    # Display the similar properties
    similar_properties_window = tk.Toplevel(root)
    similar_properties_window.title("Similar Properties")
    similar_properties_window.geometry("500x400")
```

To display the information, I used a tree view to maintain the table-like format which the data was in. This involved creating both columns and headings as seen here:

```
# Create a treeview to display the similar properties
tree = ttk.Treeview(similar_properties_window)
tree["columns"] = ("Address", "Postcode", "Type", "Tenure", "Bedrooms", "Bathrooms", "Area", "Price")
tree.column("#0", width=0, stretch=tk.NO)
tree.column("Address", anchor=tk.W, width=300)
tree.column("Postcode", anchor=tk.W, width=100)
tree.column("Type", anchor=tk.W, width=100)
tree.column("Tenure", anchor=tk.W, width=100)
tree.column("Bedrooms", anchor=tk.W, width=75)
tree.column("Bathrooms", anchor=tk.W, width=75)
tree.column("Area", anchor=tk.W, width=500)
tree.column("Price", anchor=tk.W, width=100)

tree.heading("#0", text="", anchor=tk.W)
tree.heading("Address", text="Address", anchor=tk.W)
tree.heading("Postcode", text="Postcode", anchor=tk.W)
tree.heading("Type", text="Type", anchor=tk.W)
tree.heading("Tenure", text="Tenure", anchor=tk.W)
tree.heading("Bedrooms", text="Bedrooms", anchor=tk.W)
tree.heading("Bathrooms", text="Bathrooms", anchor=tk.W)
tree.heading("Area", text="Area", anchor=tk.W)
tree.heading("Price", text="Price", anchor=tk.W)
```

This created the empty table from which I could display the similar properties as shown here:

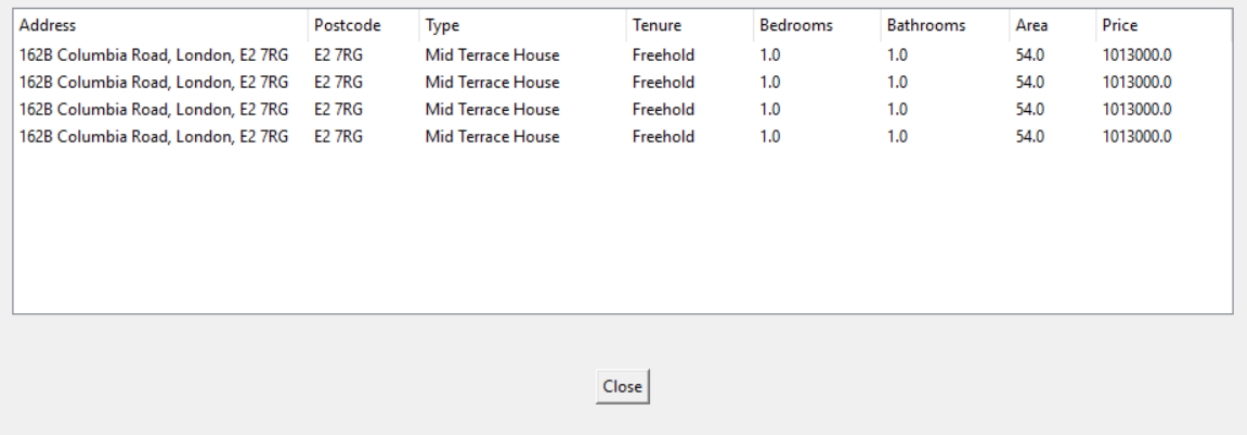


All I then needed to do was to add the information about each property to the tree view. I did this using the code shown below:

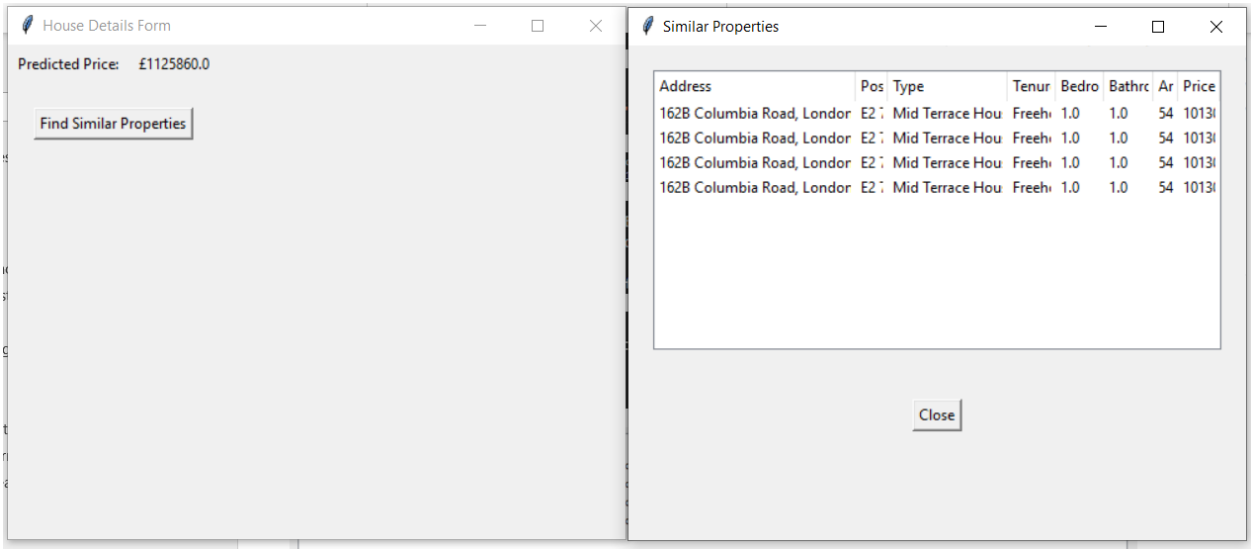
```
for index, row in similar_properties.iterrows():
    tree.insert("", tk.END, values=(row['fullAddress'], row['postcode'], row['propertyType'],  row['tenure'], row['bedro

tree.pack(pady=20, padx=20)
```

After running with this code, I was able to output a table of similar properties as shown in this screenshot:



The window created was a separate one to which the predicted price was displayed in. Both windows being open can be seen here:



Finally, the close button required some code to close the window:

```
# Button to close the window
close_button = tk.Button(similar_properties_window, text="Close", command=similar_properties_window.destroy)
close_button.pack(pady=20)
```

At this point I realised that the similar properties algorithm was sometimes outputting the same property multiple times, as shown above. This was probably because the data was repeated by error in the main dataset. To fix this I wrote a small function that removed any identical houses as shown here:

```
def remove_duplicates(dataset):
    dataset = dataset.drop_duplicates(subset=['fullAddress'])
    return dataset
```

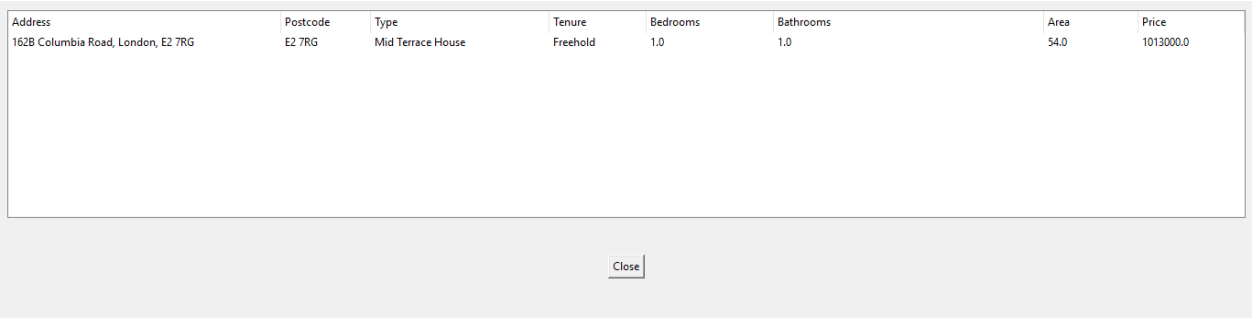
I then called it in the similar properties main function shown here:

```
    dataset = matching_bathrooms(user_details[5], dataset)
    dataset = similar_size(user_details[6], dataset)

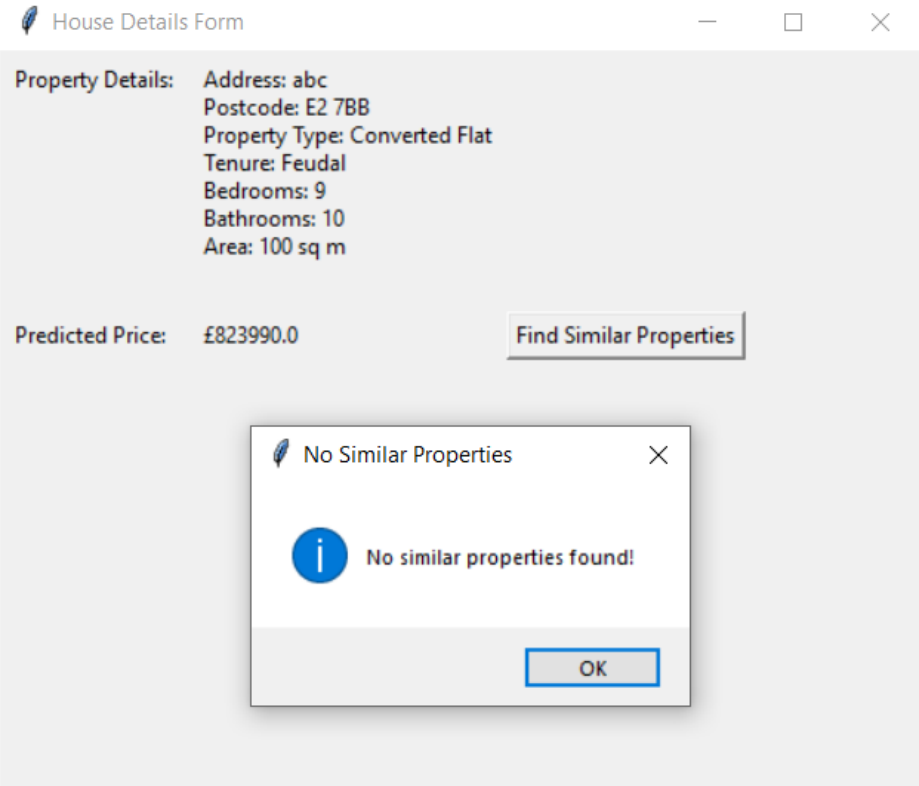
    # Remove duplicates
    dataset = remove_duplicates(dataset)

return dataset
```

This then worked in removing any identical values as shown here:



The program can also display a message when there are no similar properties. This can be seen here:



Testing

The testing table outlined in the *Design: Iterative testing method* has been shown here again.

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	Sample data for prediction	Is the program able to find properties in the same geographical area	Should return properties that are in the same geographical area	PASS	*Picture 5.1*
2	Sample data for prediction	Is the program able to find similar properties which share multiple similarities	Should return a list of properties that are similar to the inputted data	FAIL	*Picture 5.2a – there are similar properties but are not exact matches so not displayed*
3	Sample data to obtain similar properties	Is the program able to output the information about a similar property	Should display number of bedrooms/ bathrooms, size, address, tenure (if applicable)	PASS	*Picture 5.3*

Correcting Errors and Improvements

Exact matches

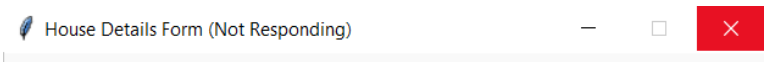
Test 2 was classed a failure, as the similar properties were hard to find, and exact matches were often not available. I addressed this error by changing the logic of the program, so that any bedrooms and bathrooms that were 1 +/- were accepted. The modified code can be seen here:

```
# Function to find properties with matching bedrooms
def matching_bedrooms(bedrooms, dataset):
    mod_dataset = dataset[dataset['bedrooms'] == bedrooms]
    if not mod_dataset.empty:
        return mod_dataset
    # if no exact match, find properties with similar number of bedrooms
    return dataset[abs(dataset['bedrooms'] - bedrooms) <= 1]

# Function to find properties with matching bathrooms
def matching_bathrooms(bathrooms, dataset):
    mod_dataset = dataset[dataset['bathrooms'] == bathrooms]
    if not mod_dataset.empty:
        return mod_dataset
    # if no exact match, find properties with similar number of bathrooms
    return dataset[abs(dataset['bathrooms'] - bathrooms) <= 1]
```

Program crashing

The program was also suffering repeatedly from crashing, as shown in the screenshot below:



To mitigate this, I researched online and found a method of using threading, which processed the UI separate to the similar properties. This meant that the UI was still able to display even though it was processing the prediction and the similar properties. The code for the threading can be seen here:

```
# Function to find similar properties
def find_similar_properties(root):
    # Get the user details
    user_details = confirmed_details_list[0]

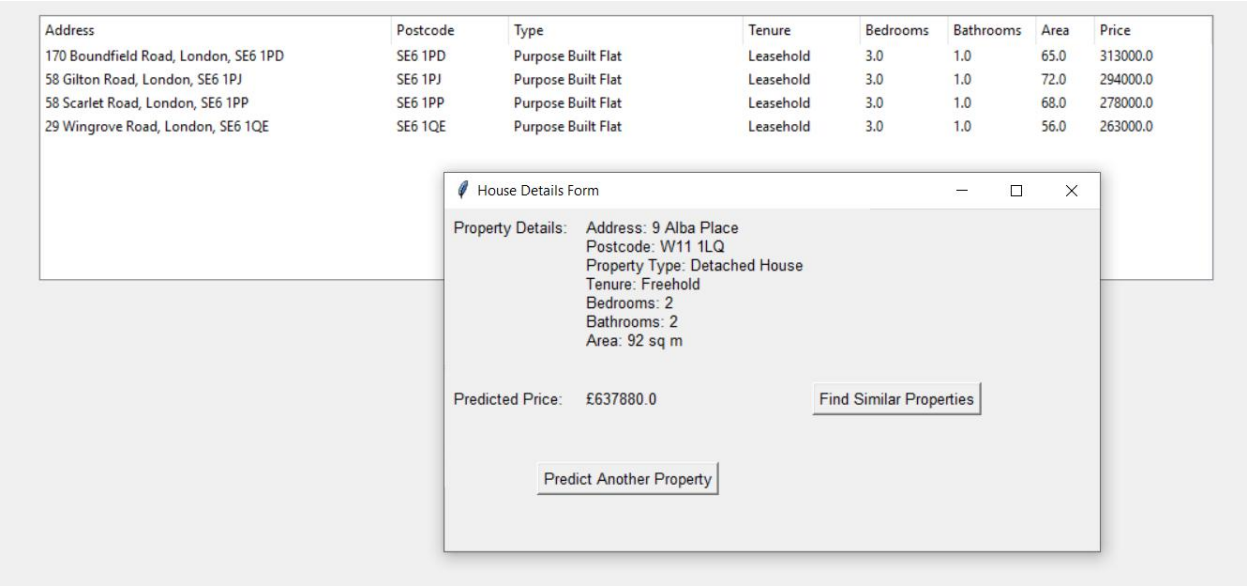
    # Run the similar properties search in a separate thread
    threading.Thread(target=lambda: run_similar_properties_search(root, user_details)).start()

# Function to run the similar properties search in a separate thread
def run_similar_properties_search(root, user_details):
    # Find similar properties
    similar_properties = similarProperties.find_similar(user_details)

    # Display the similar properties
    display_similar_properties(root, similar_properties)
```

Repeated searches

Another error I found was that when submitting another property for prediction, using the submit another button, the predicted price was changed, however the similar properties did not change accordingly. This can be seen in this screenshot, where the similar properties are not similar to the entered property.



To fix this I added some code to the reset function which allowed for another property to be predicted, this time clearing the confirmed list variable as shown:

```
def reset(root):
    # Clear the main window
    for widget in root.winfo_children():
        widget.destroy()

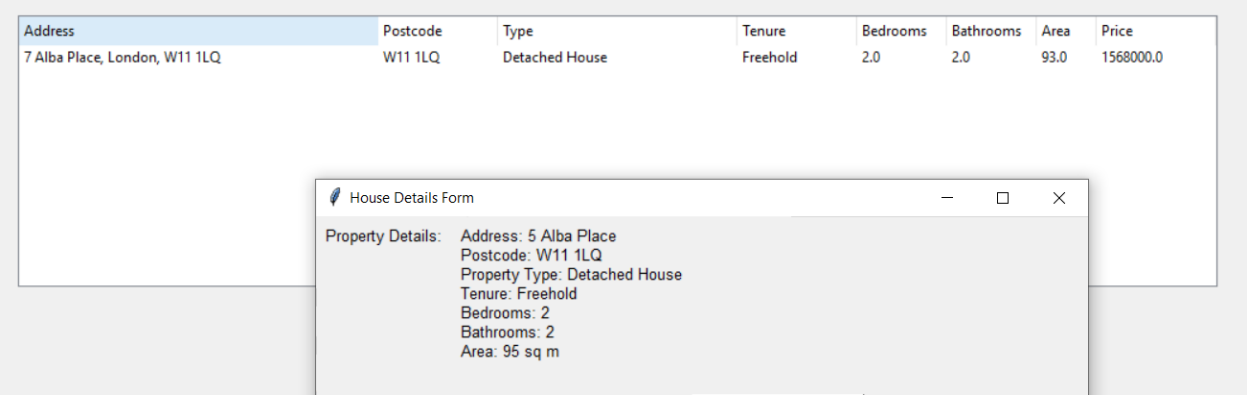
    # Clear the confirmed details list
    confirmed_details_list.clear()

    # Refresh the GUI so everything is deleted
    root.update()

    # Get the property types and tenures
    property_types, tenures = get_options()

    # Create the widgets again
    create_widgets(root, property_types, tenures)
```

This fixed the issue as shown in a repeated test here:



Testing again

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	Sample data for prediction	Is the program able to find properties in the same geographical area	Should return properties that are in the same geographical area	PASS	*Picture 5.1*
2	Sample data for prediction	Is the program able to find similar properties which	Should return a list of properties that are similar to the inputted data	PASS	*Picture 5.2b*

		share multiple similarities			
3	Sample data to obtain similar properties	Is the program able to output the information about a similar property	Should display number of bedrooms/ bathrooms, size, address, tenure (if applicable)	PASS	*Picture 5.3*

Review

What has been done

The details entered by the user has been used to find similar properties and outputs them in the UI for the user to see.

How it has been tested

The program has been tested according to the phase 5 testing tables outlined in the design section. All corrections have been made, and tests have been passed and evidenced appropriately. Therefore, the phase can be marked as complete.

How it meets success criteria and user expectations

The following success criteria have been met:

Requirement 6: Similar properties

1. Program identifies similar properties
2. The properties share similar characteristics

Changes in design that have resulted from this section

None

Stakeholder Feedback

All stakeholders were happy with the similar properties being outputted

Summary of the whole project as a prototype at this stage

At this stage, the program is nearly complete, as all key functions of the program have been completed. Non-essential phases still stand to be completed such as the range of estimates, however a lot of the framework for these phases have been completed already.

Phase 6: Range of estimates

Overview

For this phase of development, the similar properties would be used to find the lowest and highest sale prices of real-world houses. The program should then output this for the user to see.

This links to Requirement 4: Range of Estimates and the following success criteria:

- *Success criteria 14: "Displays lowest price of similar properties"*
- *Success criteria 15: "Displays highest price of similar properties"*

Development

The first step was to write some functions that returned the highest and lowest prices in the list of similar properties. These two functions can be seen here:

```
# This function finds the highest price in the list of similar properties
def highest_price(similar_properties):
    return similar_properties['currentPrice'].max()
# This function finds the lowest price in the list of similar properties
def lowest_price(similar_properties):
    return similar_properties['currentPrice'].min()
```

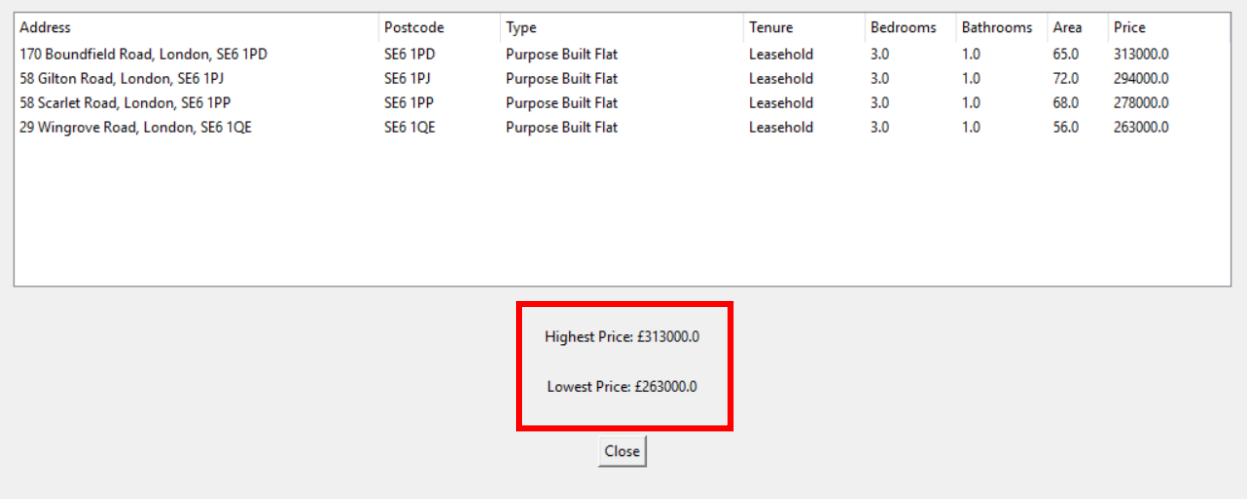
The two functions would then need to be called and the highest and lowest values returned back to the UI for display.

```
# This function returns the range of estimates for the property
def range_estimates(similar_properties):
    highest = highest_price(similar_properties)
    lowest = lowest_price(similar_properties)
    return highest, lowest
```

I then needed to output these prices in the UI, and I wrote this code to do this:

```
# Display the range of estimates
highest, lowest = rangeEstimates.range_estimates(similar_properties)
highest_label = tk.Label(similar_properties_window, text=f"Highest Price: £{highest}")
highest_label.pack(pady=10)
lowest_label = tk.Label(similar_properties_window, text=f"Lowest Price: £{lowest}")
lowest_label.pack(pady=10)
```

This then produced the highest and lowest prices along with the similar properties as shown here in the output:



Testing

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	Sample data for prediction	Is the program able to output the lowest sold price seen for a similar property	Should return a minimum price for similar properties	PASS	*Picture 6.1*
2	Sample data for prediction	Is the program able to output the highest sold price seen for a similar property	Should return a maximum price for similar properties	PASS	*Picture 6.1*

Review

What has been done

The range of estimates for the similar properties has been calculated and outputted to the user along with the similar properties

How it has been tested

The program has been tested according to the phase 6 testing tables outlined in the design section. All tests have been passed and evidenced.

How it meets success criteria and user expectations

The following success criteria have been met:

Requirement 4: Range of estimates

1. Displays lowest price of similar properties
2. Displays highest price of similar properties

Changes in design that have resulted from this section

None

Stakeholder Feedback

All stakeholders were happy with the way the similar properties were outputted.

Summary of the whole project as a prototype at this stage

At this stage, the program is complete in all the key core functions. Only the optional map is left to develop in phase 7.

Phase 7: Interactive Map

Overview

In this phase of development, an interactive map of the user's specified location will be displayed, allowing the user to be able to truly understand where the property is geographically.

This links to Requirement 2: Location of Property and the following success criteria:

- *Success criteria 10: "Displays map of local area"*
- *Success criteria 11: "Map is navigable"*

Development

For this phase of development, I first began by researching the different options for displaying interactive maps. I realised that a map API was required to obtain the map. Upon research, I came across a free API called Yandex that produced maps.

The first step was to use the geopy library to return the coordinates for the specific postcode. I imported the library and wrote this code to return the coordinates:

```
#Fetch and display the map for a given postcode.
geolocator = Nominatim(user_agent="pyqt_map_app")
location = geolocator.geocode(postcode)
latitude, longitude = location.latitude, location.longitude
print(f"Postcode: {postcode} \nLocation: {latitude}, {longitude}")
```

This returned the coordinates as shown in the output when called:

```
Postcode: SW1A 1AA
Location: 51.5008349, -0.1430045
```

Looking into Yandex maps, I realised I could generate a static map by plugging the coordinates into the API URL, before retrieving the output from the html. This was done using the following lines of code:

```
# Fetch the static map with a pin
url = f"https://static-maps.yandex.ru/1.x/?ll={longitude},{latitude}&z=14&l=map&size=600,400"
response = requests.get(url)
```

I then moved on to developing the window to display the map. As outlined in the *Design: UI design – Outlining method of development* section, the map will be displayed using the PyQt5 library.

Following a tutorial for PyQt5 (https://build-system.fman.io/pyqt5-tutorial), I wrote some code to first create the window. The function is shown here:

```python
def create_window():
    #Create the PyQt5 application window.
    app = QApplication(sys.argv)

    # Create the main window
    window = QMainWindow()
    window.setWindowTitle("Map")
    window.setGeometry(100, 100, 800, 600)

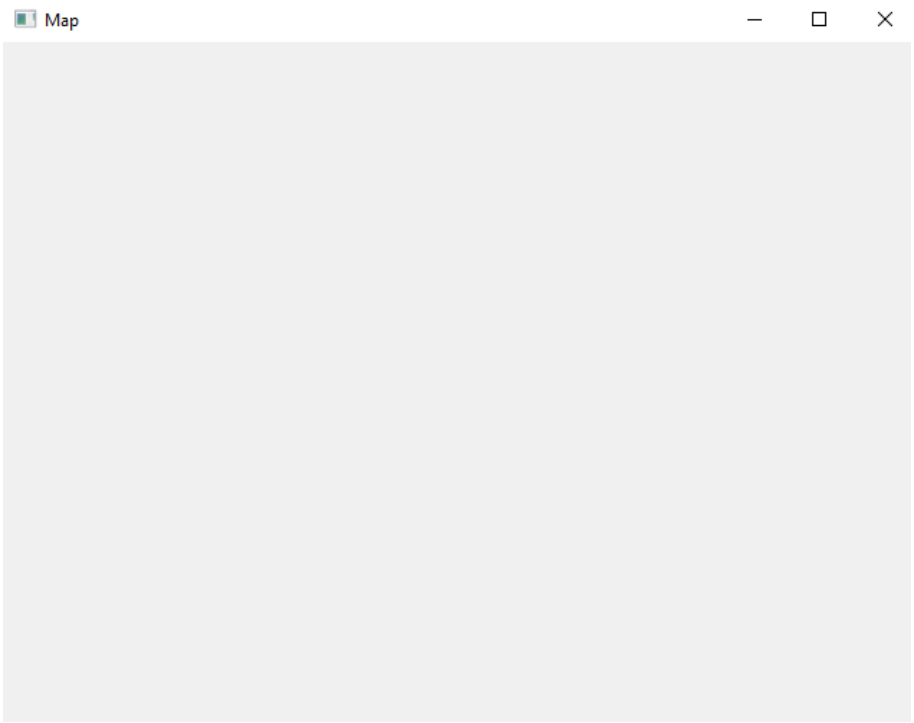
    # Central widget for the window
    central_widget = QWidget(window)
    layout = QVBoxLayout(central_widget)

    # Label for displaying the map
    map_label = QLabel()
    map_label.setFixedSize(600, 400)
    layout.addWidget(map_label)

    # Set the central widget
    window.setCentralWidget(central_widget)

    # Show the window
    window.show()
    sys.exit(app.exec_())
```

This creates the window as shown here:



I then needed to write some code which added the map onto the label that was created for displaying the map. This used the API URL shown earlier and added it to the QPixmap inbuilt module from PyQt5, which can be used to display images. The code is shown here:

```
# Load the map image if the request is successful
if response.status_code == 200:
    pixmap = QPixmap()
    pixmap.loadFromData(BytesIO(response.content).read())
    label.setPixmap(pixmap)
else:
    status_message = "Error loading map image."
```

This was added with the code to obtain the map from the API to create the full display_map function as shown:

```
def display_map(postcode, label):
    #Fetch and display the map for a given postcode.
    geolocator = Nominatim(user_agent="pyqt_map_app")
    location = geolocator.geocode(postcode)

    # Set the status message
    if location:
        latitude, longitude = location.latitude, location.longitude
        status_message = f"Location: {latitude}, {longitude}"

        # Fetch the static map
        url = f"https://static-maps.yandex.ru/1.x/?ll={longitude},{latitude}&z=14&l=map&size=600"
        response = requests.get(url)

        # Load the map image if the request is successful
        if response.status_code == 200:
            pixmap = QPixmap()
            pixmap.loadFromData(BytesIO(response.content).read())
            label.setPixmap(pixmap)
        else:
            status_message = "Error loading map image."
    else:
        status_message = "Location not found!"

    return status_message
```

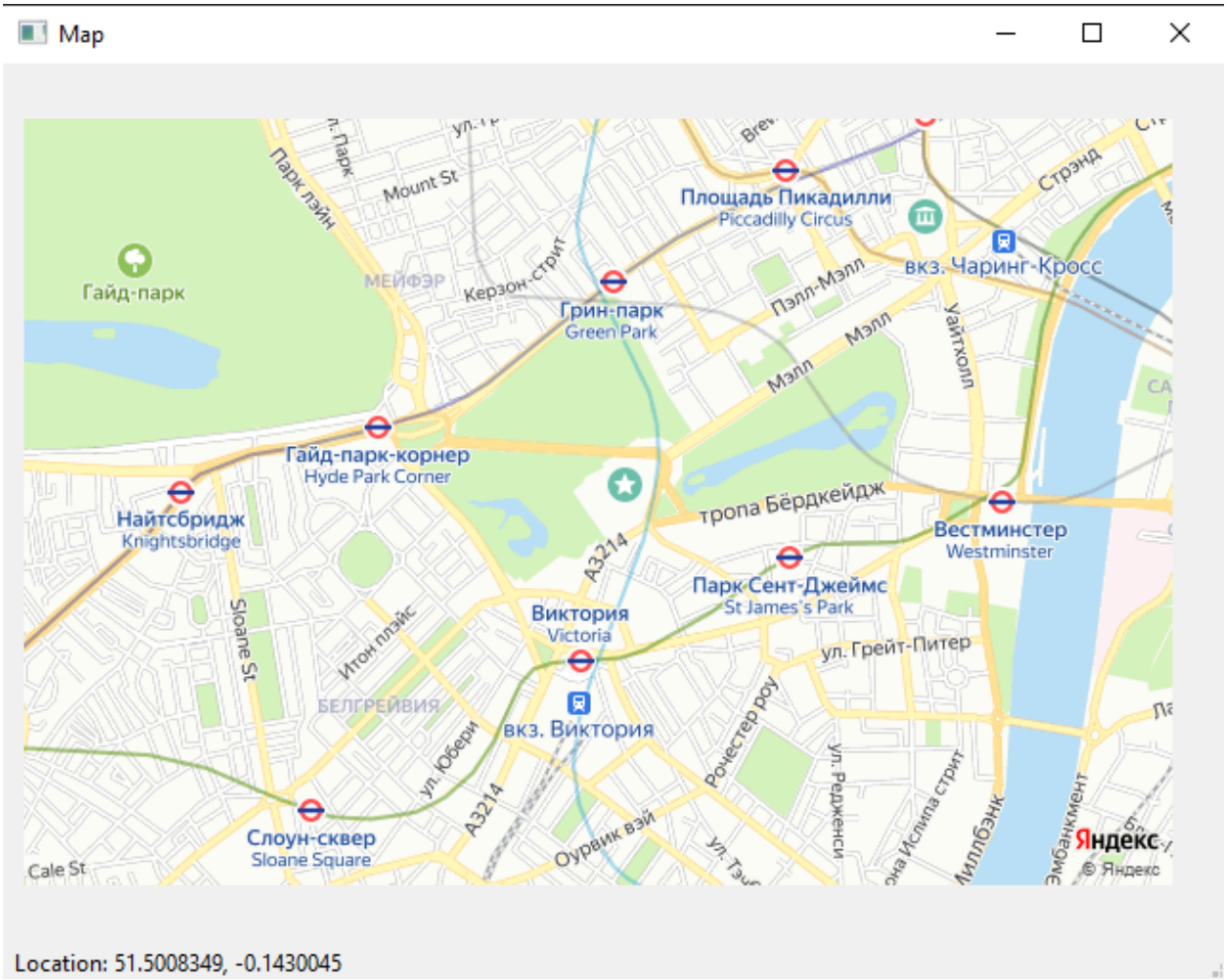
I also added selection statements, to cover the case where the map is not returned from the API, to add further validation to the program.

This function was then called in the create_window function using the following lines of code:

```
# Display the map immediately
status_message = display_map(postcode, map_label)

# Set the status message
window.statusBar().showMessage(status_message)
```

I then tested the program by calling the function and passing in a postcode. This was the output:



Hence, I knew that the program was able to generate a map for a specific location.

I then moved onto try to make the map navigable. However, this was not possible using the API I had previously used and required a new completely different approach to this phase of development, including using APIs that required payment. This was **beyond the scope of this project**; hence I decided to stop developing this phase at this stage.

The final part of the development was integrating this new file into the rest of the program.

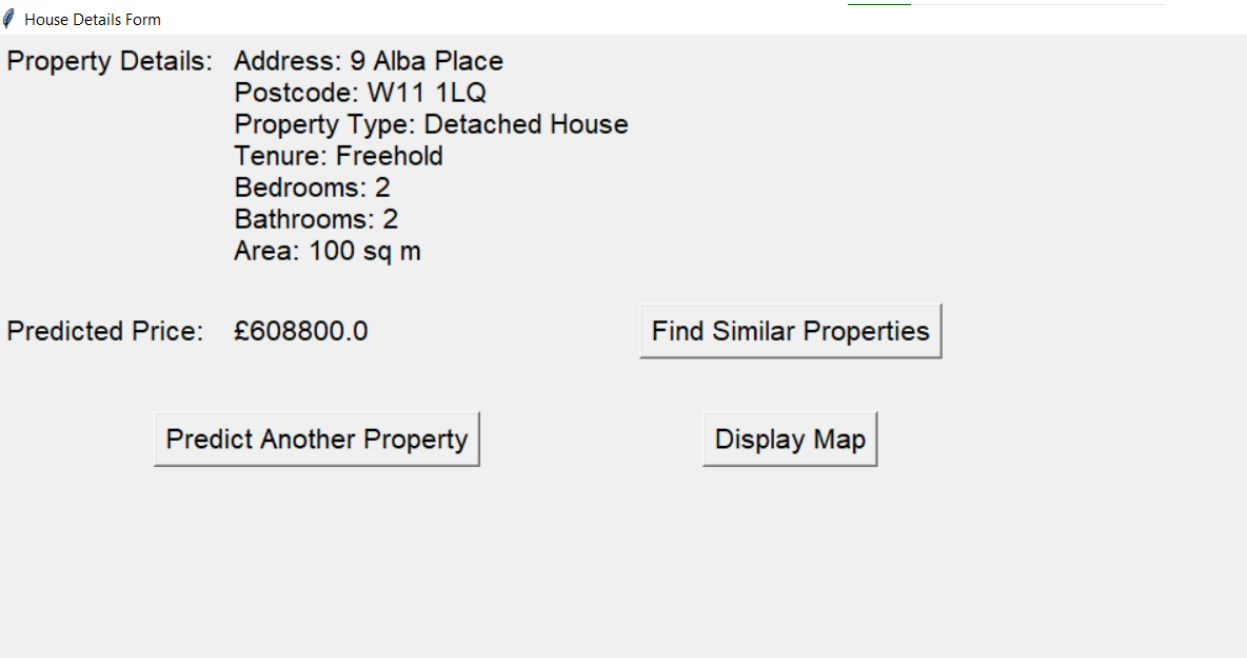
In the ui.py file I imported the map program using this code:

```
# Import the map function from map.py
spec4 = importlib.util.spec_from_file_location("map", "map.py")
map = importlib.util.module_from_spec(spec4)
spec4.loader.exec_module(map)
```

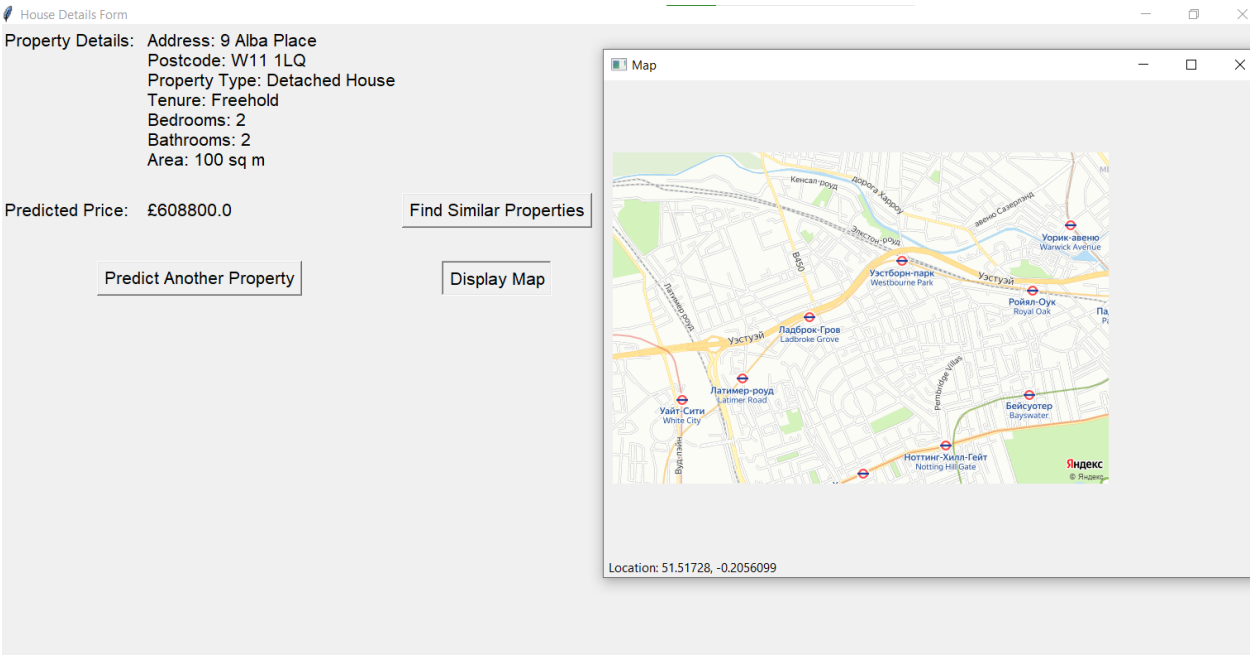
This allowed me to call the map function, passing in the postcode as a parameter, using this button on the display prediction function:

```
# Button to display the map
map_button = tk.Button(root, text="Display Map", command=lambda: map.create_window(user_details[1]))
map_button.grid(row=2, column=2, columnspan=2, pady=20)
```

This worked in the UI as shown here:



And the map is displayed when the display map button is clicked



Testing

This phase of development was tested according to the test table outlined in the design section. The table is repeated again here for reference:

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	Sample data for input (focussed on address)	Is the program able to connect to maps using the API	Should display a map with the specified address	PASS	*Picture 7.1*
2	Sample data for input (focussed on address)	Is the map easy to understand and locate the property	Should make it clear to the user where the specific location is	FAIL	*Picture 7.1*
3	Sample data for input (focussed on address)	Can the map be closed and opened properly	When open button clicked, map should display, and when closed clicked it should disappear	FAIL	*Picture 7.1 – no button to open or close map*
4	Sample data for input (focussed on address)	Is the map interactive	The map should be movable and respond to movements by the cursor	FAIL	

Correcting errors

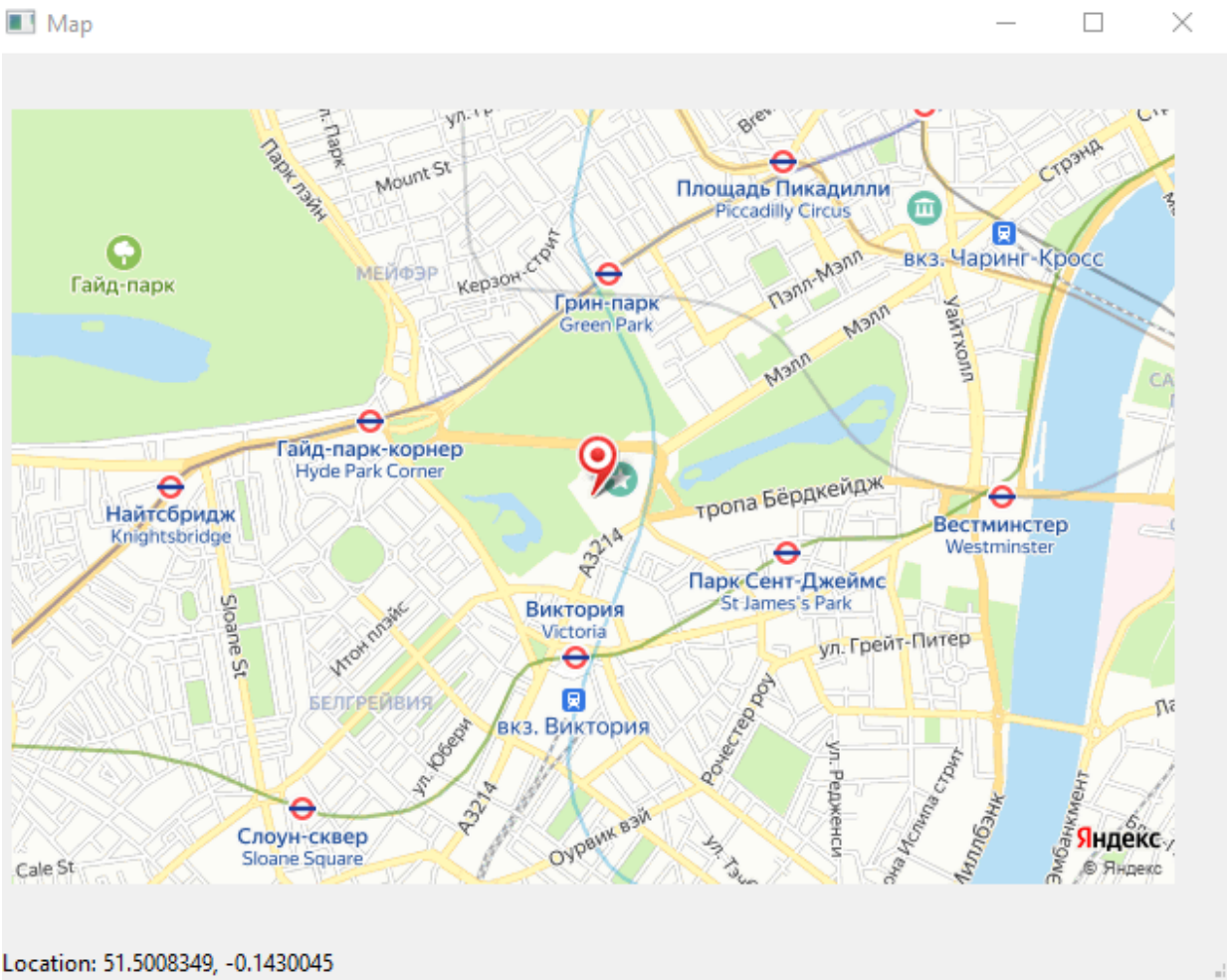
Marking specific location

To successfully complete Test 2 of Phase 7, the program "should make it clear to the user where the specific location is". This could be done by placing a marker for where the specific location of the property is.

After researching the Yandex API, I found that a map with a marker could be generated by slightly modifying the URL. This change was made:

```
# Fetch the static map with a pin
url = f"https://static-maps.yandex.ru/1.x/?ll={longitude},{latitude}&z=14&l=map&size=600,400&pt={longitude},{latitude},pm2rd1"
response = requests.get(url)
```

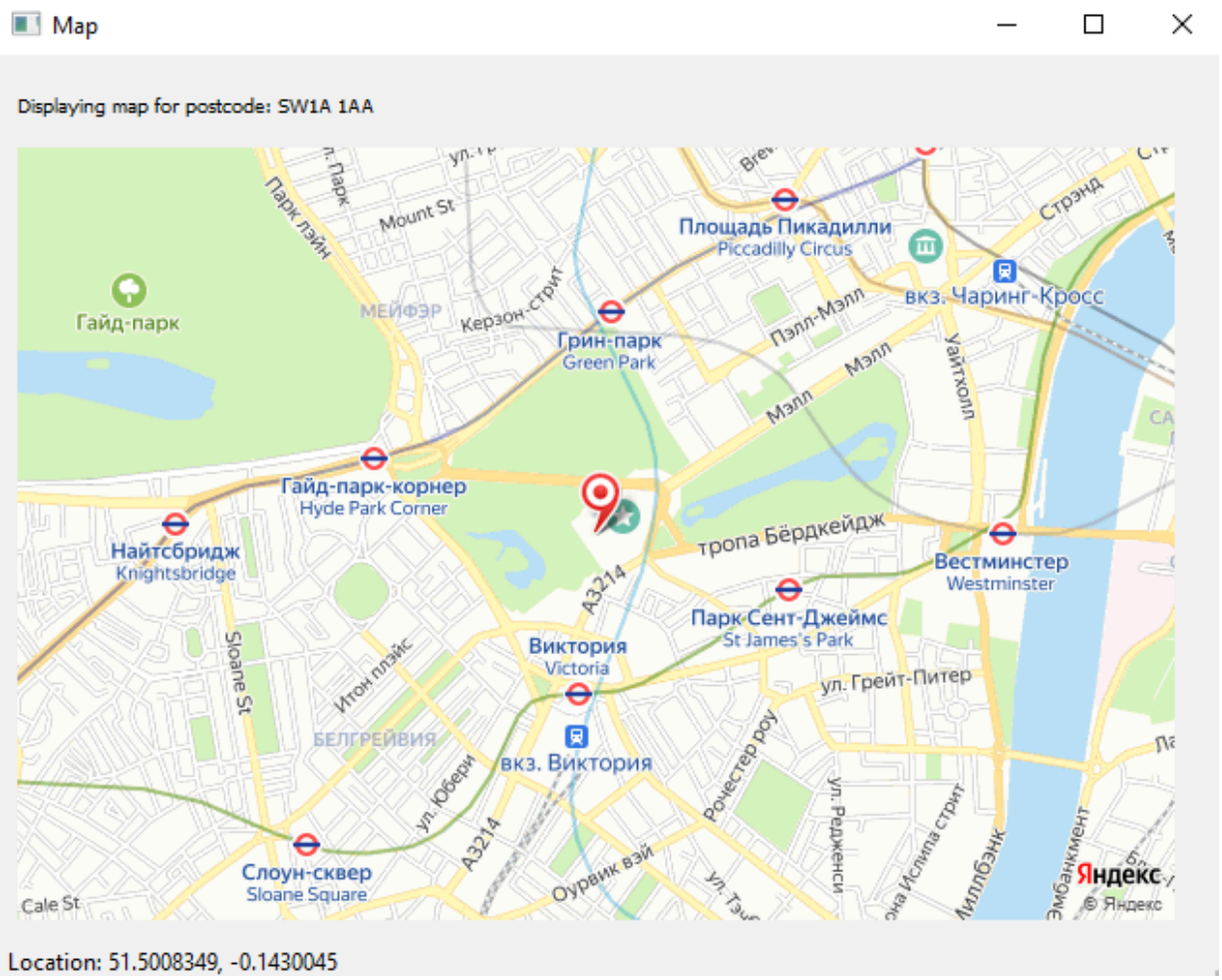
This generated a map with a marker as shown:



I also decided to add another label above the map, displaying the postcode at which the marker was pointing out. This was done by adding a QLabel as shown here:

```
# Label displaying postcode
postcode_label = QLabel(f"Displaying map for postcode: {postcode}")
layout.addWidget(postcode_label)
```

This added the text above the map as shown here:

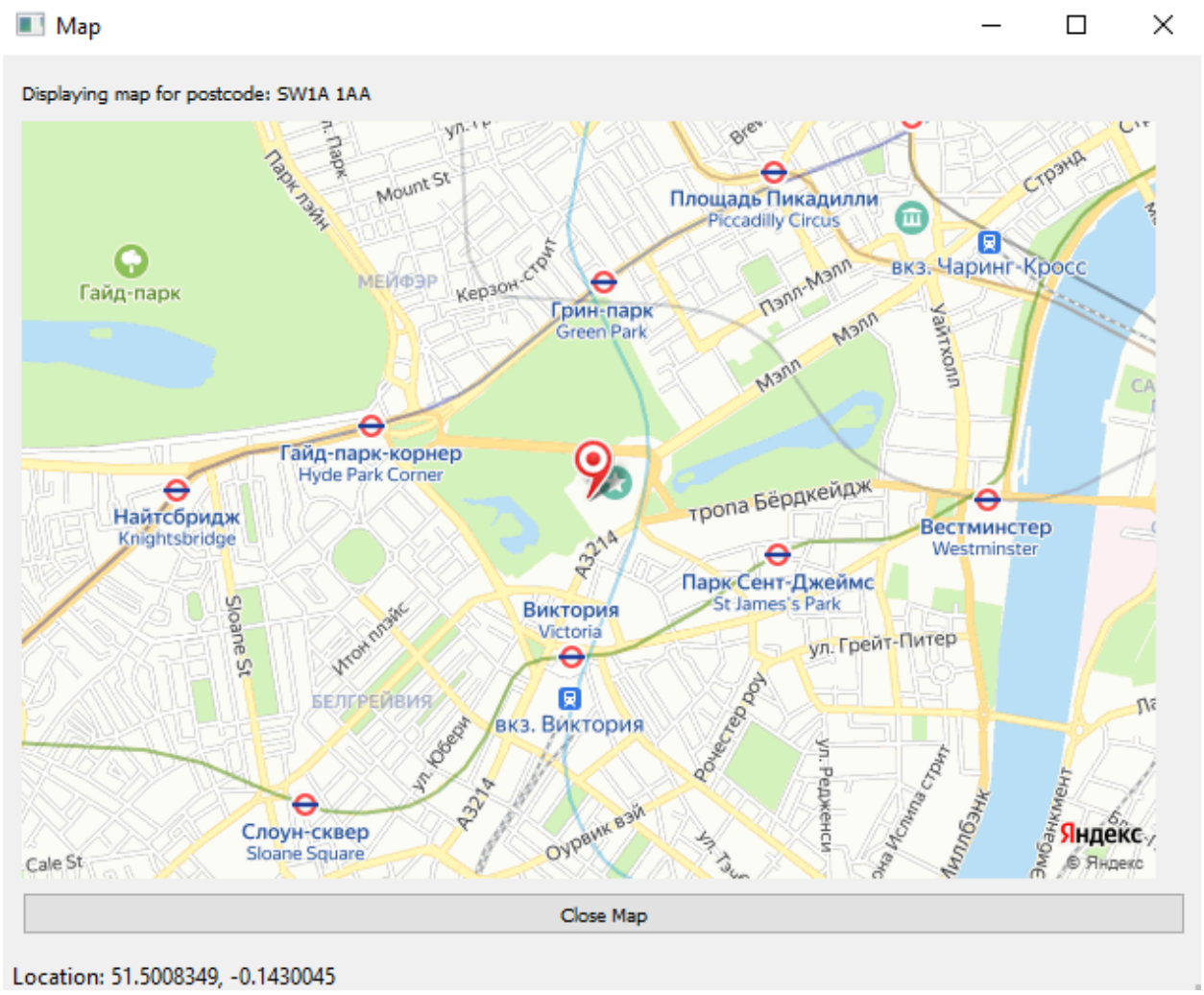


Close button

To meet Test 3 of Phase 7, I needed a way to close the map once it was opened. This was done by adding a button below the map using the QPushButton module from PyQt5. The code can be seen here:

```
# Button to close the window
close_button = QPushButton("Close Map")
close_button.clicked.connect(window.close)
layout.addWidget(close_button)
```

This added the button as shown here:


Map
Displaying map for postcode: SW1A 1AA
Гайд-парк
Парк лэйн
Mount St
МЕЙФЭР
Керзон-стрит
Площадь Пикадилли
Piccadilly Circus
Стрэнд
вкз. Чаринг-Кросс
Грин-парк
Green Park
Пэлл-Мэлл
Мэлл
Уайтхолл
Гайд-парк-корнер
Hyde Park Corner
Найтсбридж
Knightsbridge
тропа Бёрдкейдж
Вестминстер
Westminster
A3214
Парк Сент-Джеймс
St James's Park
Виктория
Victoria
Sloane St
Итон плэйс
БЕЛГРЕЙВИЯ
ул. Грейт-Питер
вкз. Виктория
ул. Рочестер роу
ул. Редженси
Лиллбэнк
Слоун-сквер
Sloane Square
ул. Юбери
Оурвик вэй
Cale St
Яндекс
© Яндекс
Close Map
Location: 51.5008349, -0.1430045

Testing again

Here is the test table again after errors had been corrected:

Test	Input	Description of test	Expected outcome	Pass	Evidence
1	Sample data for input (focussed on address)	Is the program able to connect to maps using the API	Should display a map with the specified address	PASS	*Picture 7.1*
2	Sample data for input (focussed on address)	Is the map easy to understand and locate the property	Should make it clear to the user where the specific location is	PASS	*Picture 7.2*
3	Sample data for input (focussed on address)	Can the map be closed and opened properly	When open button clicked, map should display, and when closed clicked it should disappear	PASS	*Video 7.3*
4	Sample data for input (focussed on address)	Is the map interactive	The map should be movable and respond to movements by the cursor	FAIL	*Video 7.4*

Review

What has been done

A map has been generated showing the location of the property that the user has entered for submission. This map has a marker at the location, but is not interactive.

How it has been tested

The program has been tested according to the phase 7 testing tables outlined in the design section. All corrections have been made and tests have been passed and evidenced appropriately, except for the last test.

How it meets success criteria and user expectations

The following success criteria have been met:

Requirement 2: Location of property

1. Displays postcode
2. Displays map of local area
3. Map is navigable

Changes in design that have resulted from this section

Map is no longer interactive

Stakeholder Feedback

All stakeholders were happy with the current map, except **Stakeholder 1 Andrew** who expressed slight disappointment at its exclusion, saying:

“Hi,

Thanks for showing the program again after the recent progress. I understand the map is not interactive as discussed in the design. This is not preferable as I can no longer see local stations or zoom in and out to view other areas. Hopefully this can be something that can be included later?

Many thanks,

Andrew”

Summary of the whole project as a prototype at this stage

At this stage, all core features of the project have been developed and tested.

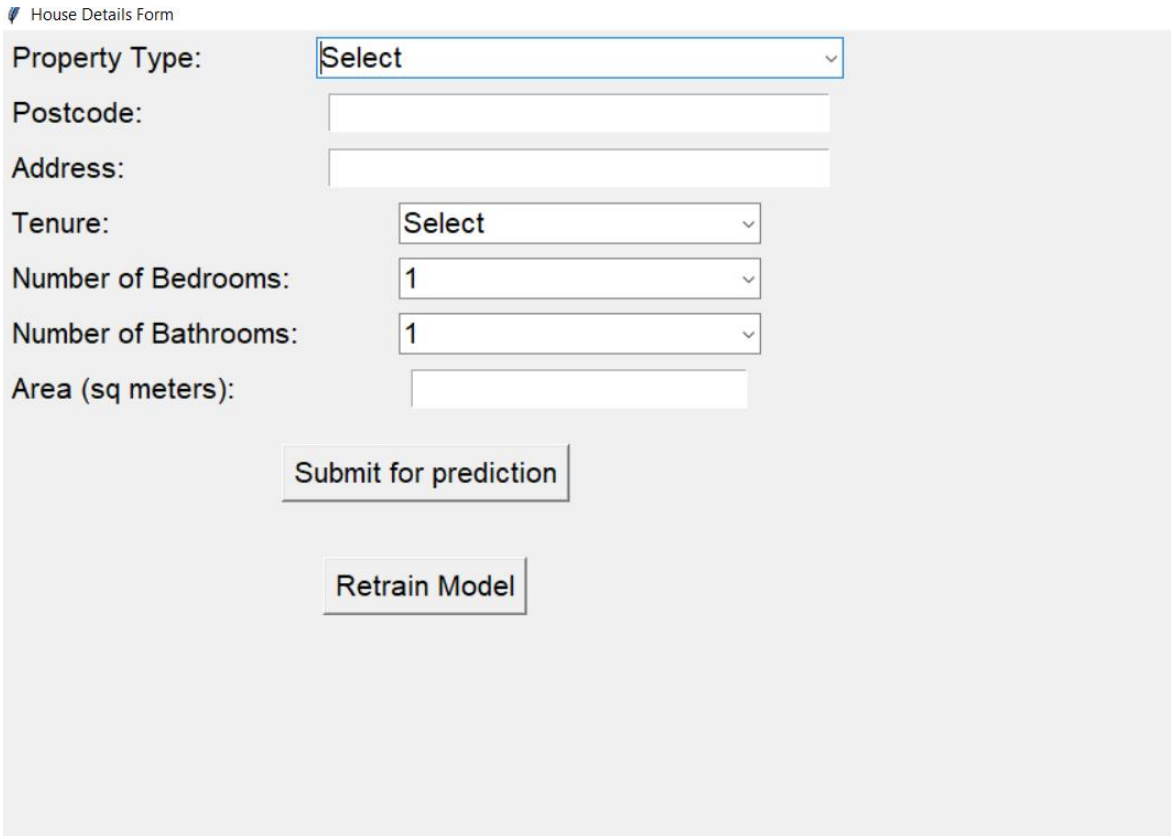
Additional developments

After all seven of the phases were completed, I realised that there was no option for the user to call phases 1 or 2 of the program to retrain the model. I realised I had to add another button to the UI to allow the user to scrape data and retrain the model and new market data.

Here is the code to create the new button:

```
# Button to retrain the model
retrain_button = tk.Button(root, text="Retrain Model", command=lambda: reset_model(root))
retrain_button.grid(row=8, column=0, columnspan=2, pady=20)
```

The button can be seen here:



I then imported the web scraper and model trainer files as shown:

```
# Import the web scraper function from webScraper.py
spec5 = importlib.util.spec_from_file_location("webScraper", "webScraper.py")
webScraper = importlib.util.module_from_spec(spec5)
spec5.loader.exec_module(webScraper)

# Import the model trainer function from trainModel.py
spec6 = importlib.util.spec_from_file_location("trainModel", "trainModel.py")
trainModel = importlib.util.module_from_spec(spec6)
spec6.loader.exec_module(trainModel)
```

This code called on the webScraper.py file which I modified by adding the main function as seen here:

```
  df = pd.DataFrame.from_dict(data)
  df.to_csv(r"london_house_price_data.csv", encoding="utf-8", header="true", index = False)

  return df

#Main function from which all other functions are called
def webScraper():
  property_scraping(1000)
  detailed_info_scrape()
  dataset_creation()
```

I also modified the model training file as shown, creating the main function to just train the model

```
# Main function to execute the workflow
def main():

    df = pd.read_csv('london_house_price_data.csv')
    df = clean_data(df)
    df = df.sample(frac=0.01)
    df.to_csv('sampled_data.csv', index=False)
    df_dummies = preprocess_data(df)
    df_dummies.to_csv('sampled_data_dummies.csv', index=False)
    train_and_evaluate_models(df_dummies)
```

I then called both of these functions in the ui.py file, running them in another thread to avoid crashing the main window:

```
      # Run the prediction in a separate thread
      threading.Thread(target=lambda: retrain_model(root)).start()

  # Function to retrain the model
  def retrain_model(root):
      # Call the web scraper function
      webScraper.webScraper()
      trainModel.main()

      # Message to show the model has been retrained
      messagebox.showinfo("Model Retrained", "Model has been retrained!")

      # Reseting the prediction
      reset(root)
```

This allowed the user to call the first phases of the development. Thus, this concluded my development section.

Evaluation

Validation testing for usability and robustness

Once the development was complete, I needed to test the validity of the program, to ensure that it did not crash randomly, or accept invalid inputs. The tests followed the tests outlined in the *Design: Iterative testing method* section. The test is copied again for reference (justification is omitted in copy).

Testing

Test number	Component	Input	Type of data	Expected outcome	Pass	Evidence
1	Postcode	Valid postcode e.g. "SE6 2QE"	Normal	Accepts postcode	PASS	*Validation 1*
2	Postcode	Another postcode e.g. "W12 7GN"	Normal	Accepts postcode	PASS	*Validation 1*
3	Postcode	"IGK36 3NM"	Erroneous (exceeds maximum length)	Rejects postcode	PASS	*Validation 1*
4	Postcode	"1 6"	Erroneous (below minimum length)	Rejects postcode	PASS	*Validation 1*
5	Postcode	"SW99 8GP"	Erroneous (not a real postcode)	Rejects postcode	PASS	*Validation 1*
6	Property area	"50"	Normal	Accepts area	PASS	*Validation 1*
7	Property area	"100a"	Erroneous (only integer values allowed)	Rejects area	PASS	*Validation 1*
8	Property area	"0"	Erroneous (no value given)	Rejects area	PASS	*Validation 2*
9	Address	"21 St John's Road"	Normal	Displays given address when outputting price	PASS	*Validation 1*

10	Address	“abc”	Boundary	Accepts address and displays in output	PASS	*Validation 1*
11	Number of bathrooms	Selects “1”	Normal	Uses given number for prediction and displays this	PASS	*Validation 1*
12	Number of bathrooms	Selects “3”	Normal	Uses given number for prediction and displays this	PASS	*Validation 2*
13	Number of bedrooms	Selects “1”	Normal	Uses given number for prediction and displays this	PASS	*Validation 1*
14	Number of bedrooms	Selects “3”	Normal	Uses given number for prediction and displays this	PASS	*Validation 2*
15	Tenure	Selects “Freehold”	Normal	Uses given choice for prediction and displays this	PASS	*Validation 1*
16	Tenure	Doesn’t select	Erroneous (must select tenure)	Rejects this data	PASS	*Validation 1*
17	Type of property	Selects “Terraced”	Normal	Uses and displays given property type for prediction	PASS	*Validation 1*
18	Type of property	Selects “Purpose built flat”	Normal	Uses and displays given property type for prediction	PASS	*Validation 1*

Review

Overall, all the test had passed, and the program validated all inputs appropriately. All other methods of interaction between the user and program were defined by buttons, and therefore did not need any validation. Hence, the program is tested to be robust.

Post development testing

Test 1

The first post development test from the *Design: Post development testing* section has been copied again and carried out. The **evidence** can be found in **Post development testing 1 video**

Step	Input	Type of data	Expected outcome	Pass	Evidence
1	Enter postcode: "N13 5LWd"	Erroneous	Rejects postcode	PASS	Post development 1
2	Enter postcode: "N13 5LW"	Normal	Does not reject postcode but accepts	PASS	Post development 1
3	Enter Address: "72 Hamilton Crescent"	Normal	Accepts address and produces this later	PASS	Post development 1
4	Select property type 'Mid Terrace House	Normal	Accepts property type	PASS	Post development 1
5	Select number of bedrooms '3'	Normal	Accepts number of bedrooms	PASS	Post development 1
6	Select number of bathrooms '1'	Normal	Accepts number of bathrooms	PASS	Post development 1
7	Select tenure 'Freehold'	Normal	Accepts tenure	PASS	Post development 1
8	Enters Property area 'hundred and two'	Erroneous	Rejects property area as incorrect data type	PASS	Post development 1
9	Enters Property area '102'	Normal	Accepts property area	PASS	Post development 1
10	Enters submit to enter	Normal	Submits user data for model prediction and returns a predicted price	PASS	Post development 1
11	Clicks on first similar property	Normal	Displays information regarding the first similar property	PASS	Post development 1

Test 2

The second post development test from the *Design: Post development testing* section has been copied again and carried out. The **evidence** can be found in **Post development testing 2 video**

Step	Input	Type of data	Expected outcome	Pass	Evidence
1	Enter postcode: “SW1X 8AE”	Normal	Accepts postcode	PASS	Post development 2 video
2	Enter address: “”	Erroneous	Rejects address	PASS	Post development 2 video
3	Enter Address: “Flat 3, 18 Eaton Place”	Normal	Accepts address	PASS	Post development 2 video
4	Select property type ‘Flat/Maisonette’	Normal	Accepts property type	PASS	Post development 2 video
5	Select number of bedrooms ‘10’	Boundary	Accepts number of bedrooms	PASS	Post development 2 video
6	Select number of bathrooms ‘2’	Normal	Accepts number of bathrooms	PASS	Post development 2 video
7	Doesn’t select tenure	Erroneous	Rejects empty selection	PASS	Post development 2 video
8	Select tenure ‘Leasehold’	Normal	Accepts tenure	PASS	Post development 2 video
9	Enters Property area ‘270’	Normal	Accepts property area	PASS	Post development 2 video
10	Enters submit to enter	Normal	Submits user data for model prediction and returns a predicted price	PASS	Post development 2 video
11	Press predict property again	Normal	Form to submit prediction again	PASS	Post development 2 video
12	Predict same property again, this time with 3 bedrooms	Normal	Accepts information	PASS	Post development 2 video

13	Opens map	Normal	Map will open in window	PASS	Post development 2 video
14	Navigates map	Normal	Map moves correspondingly to show what area the user wishes to see	FAIL	Post development 2 video

Feedback

Test 1 – Stakeholder 1 Property Buyers (Andrew):

1. What worked?
   "All the functions that I would require to use worked for me. I could enter the information into the program, and it gave me a price and allowed me to also view similar properties."
2. What errors were there?
   "I didn't encounter any errors when running through what I would do on the program. However, the similar properties opening on a different window was a bit fiddly, and I had to resize it to make it big enough to see all the information."
3. How did these errors impact your experience?
   "Yeah, again it just made it slightly tacky to use the program and slightly decreased my user experience."
4. What areas for improvement are needed?
   "For me, it would just be improving the UI, perhaps changing the design slightly to make it a bit more user friendly. Maybe also having all the different functions opening on the same window to make it easier to see. Otherwise, that's all."

Test 2 – Stakeholder 2 Property Sellers (Bianca):

1. What worked?
   "The core functions of the program worked and helped me understand a bit more about how much my house was worth."
2. What errors were there?
   "The main thing was the map not being able to move, and it was just a still image."
3. How did these errors impact your experience?
   "It just meant the utility was impacted slightly."
4. What areas for improvement are needed?
   "For me, it would be getting the map to work properly. I noticed that closing the map also impacted the rest of the program as well, maybe that is something to look at in the future?"

Review

The overall feedback showed that the program was both robust and functioning. Robustness was also proven in the validation testing; hence it has met standards for both function and robustness.

Have the success criteria been met?

Requirement 1

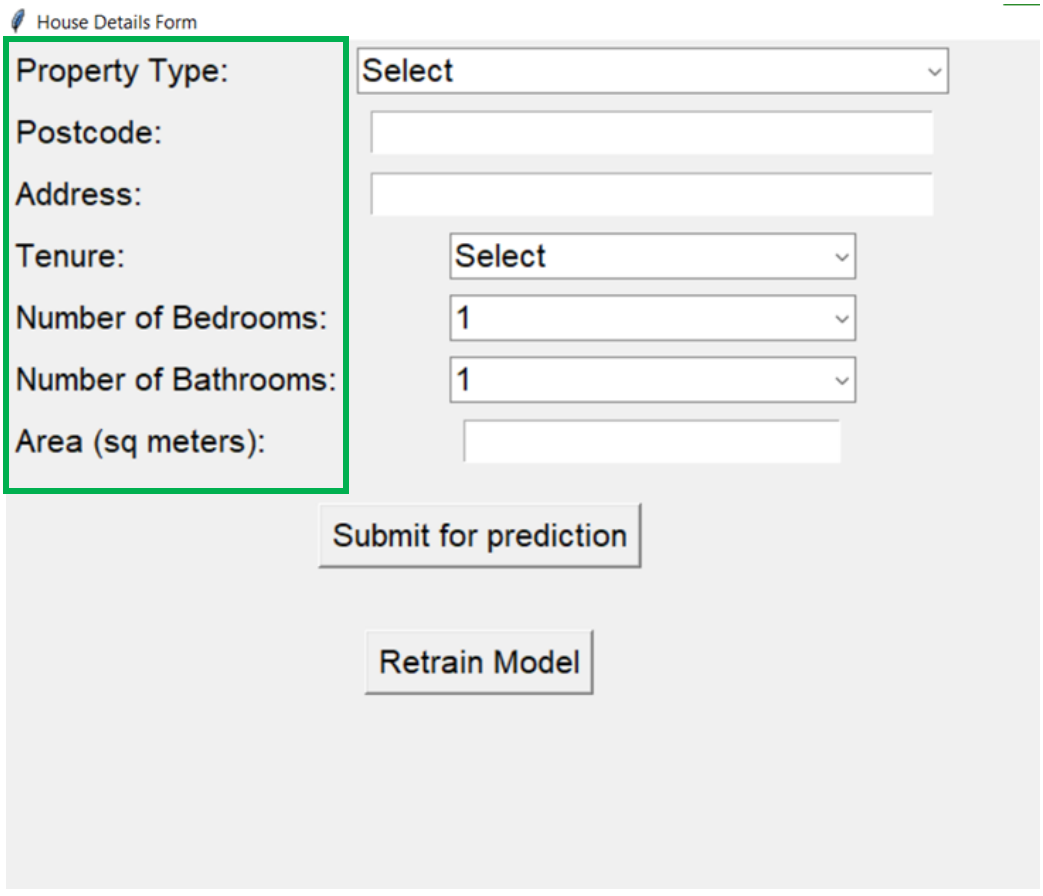
Requirement 1 is shown for reference below:

Requirement 1: Information of property

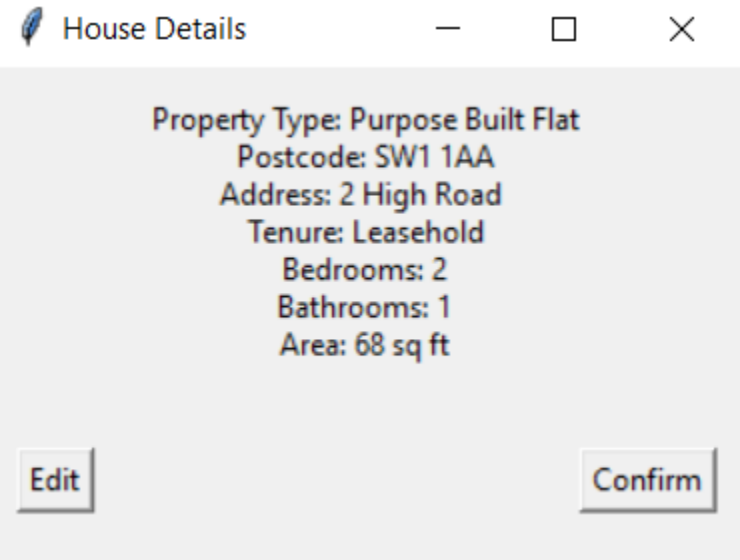
1. Displays number of bedrooms
2. Displays size and type of property
3. Displays number of bathrooms
4. Displays the tenure
5. Displays the address

Evidence:

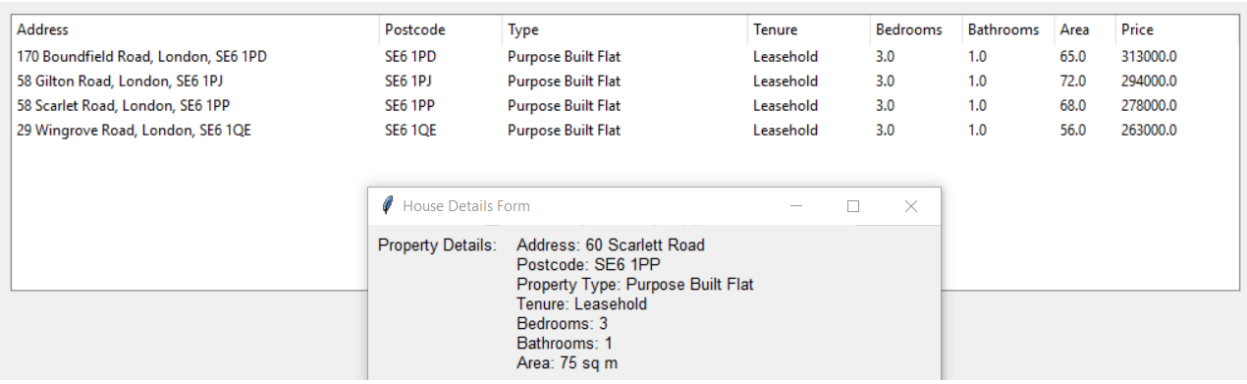
In prediction submission



In prediction confirmation



In similar properties



Explaination of how success criteria met:

1.1 – "Displays number of bedrooms"

This success criteria was met as the number of bedrooms is clearly labelled and displayed as evidenced above. The user can clearly see the number of bedrooms.

1.2 – "Displays size and type of property"

This success criteria was met as both the size and types of property can be clearly seen throughout the program. Both have labels clearly showing what the data represents.

1.3 – "Displays number of bathrooms"

This success criteria was met as the number of bathrooms is clearly displayed throughout the program, as shown in the above screenshots.

1.4 – "Displays the tenure"

This success criteria was met as the tenure can be clearly seen in the program.

1.5 – "Displays the address"

This success criteria has been met as the address can be seen displayed throughout the program.

Summary:

This shows that all success criteria in requirement 1 have been met, as demonstrated in the above evidence.

Requirement 2

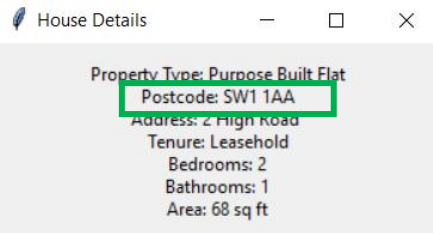
Requirement 2 is displayed again for reference below:

Requirement 2: Location of property

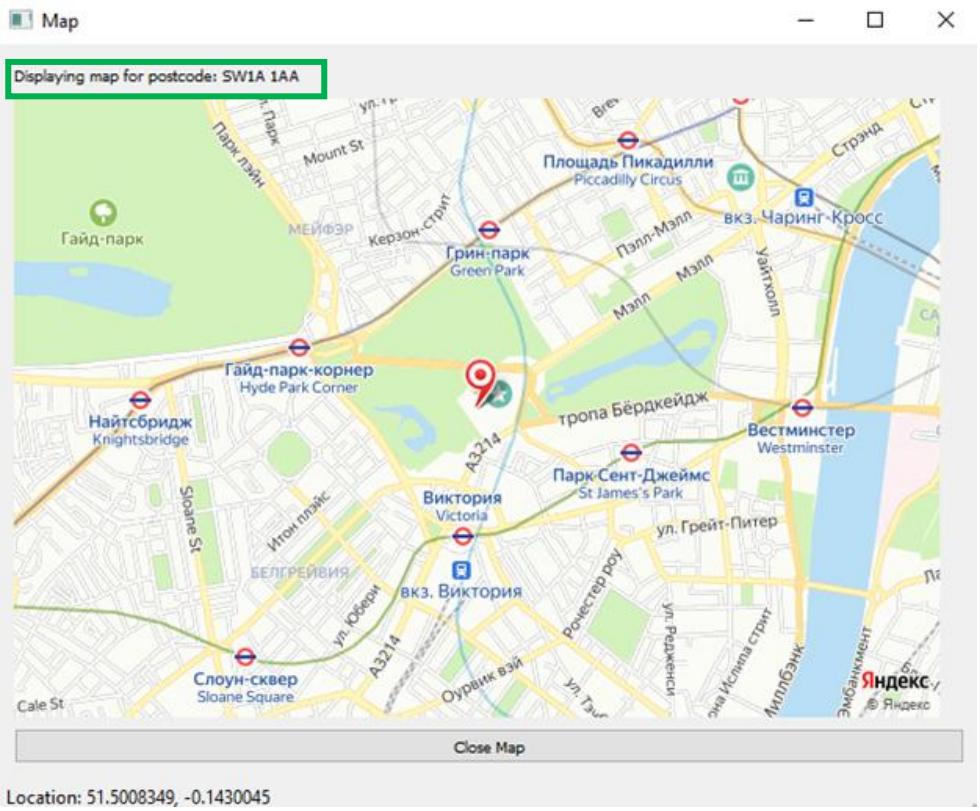
1. Displays postcode
2. Displays map of local area
3. Map is navigable

Evidence:

Success criteria 2.1 – Displays postcode:



Success criteria 2.1 – Displays postcode and 2.2 – Displays map of local area



Success criteria 2.3 – Map is navigable

This success criteria was **not met**. This was discussed in the development of phase 7.

Explanation of how success criteria met:

2.1 – "Displays postcode"

This success criteria has been met as the postcode can be seen in the submission page, the confirmation message, the prediction and the map. This shows that the program displays the postcode.

2.2 – "Displays map of local area"

This success criteria has been met as the map is produced when the user clicks to display the map. This can be seen in the screenshots above as well as the *Post development 2 video*.

2.3 – "Map is navigable"

This success criteria has **not been met** as the map is not interactive, but rather just a still image. I looked at the different options of creating an interactive map. Unfortunately, the options available required the use of **complex and new programming** techniques and **beyond the scope** of an A-level program. This would require a lot of time and resources to learn how to use certain libraries to access java script for example. Hence, when discussed with the stakeholders, it was agreed to overlook this success criteria.

Summary:

All success criteria have been met except from the 2.3, and these have been evidenced in both the testing tables and above.

Requirement 3

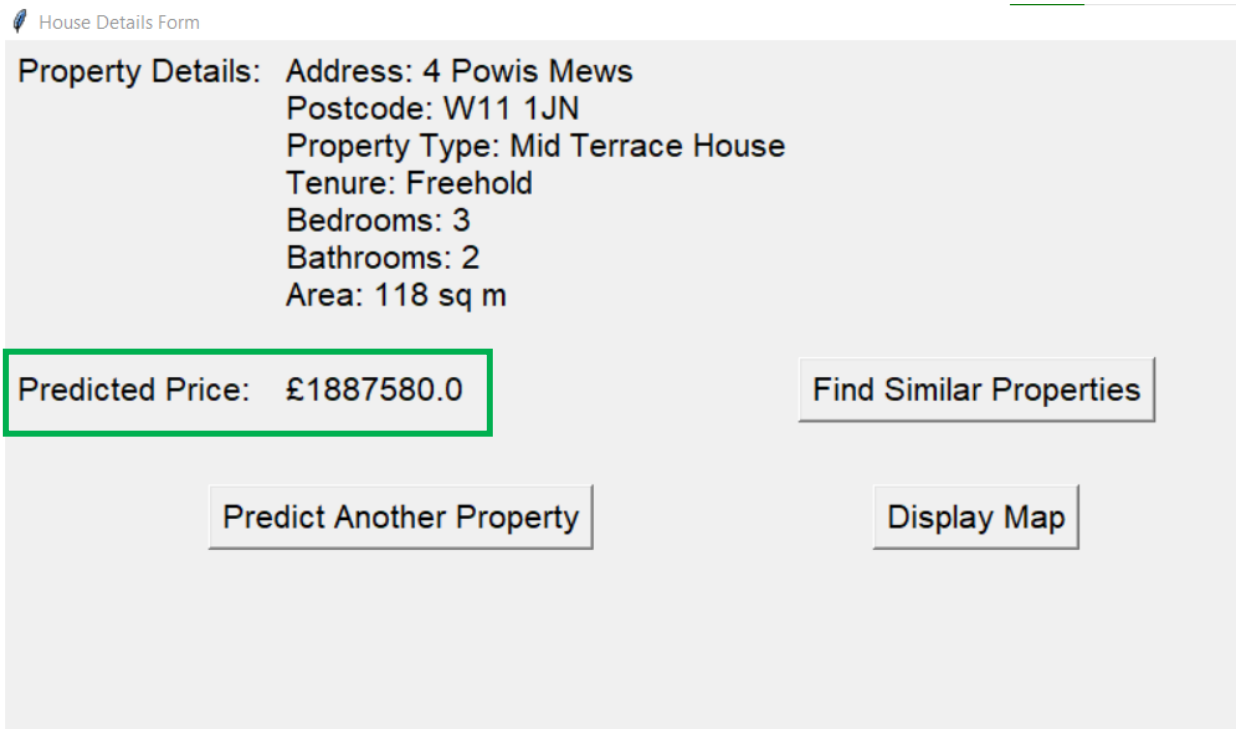
This has been copied again for reference:

Requirement 3: Predicted price

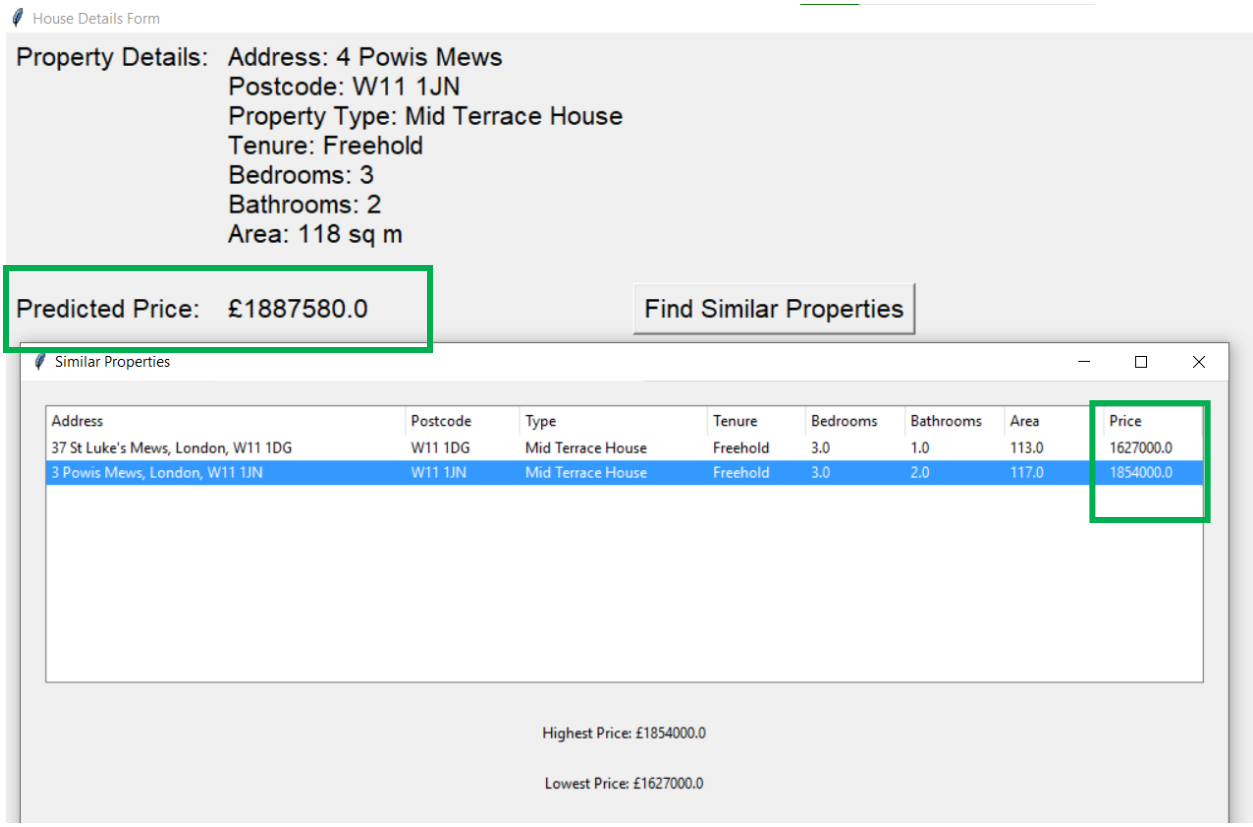
1. Calculates and displays a predicted price using a machine learning model
2. The predicted price should be as accurate as possible

Evidence

Success criteria 3.1: "Calculates and displays a predicted price using a machine learning model"



Success criteria 3.2: "The predicted price should be as accurate as possible



Explaination of how success criteria met:

3.1: "Calculates and displays a predicted price using a machine learning model"

This success criteria has been met as shown above, and in the *Post development 1* and *Post development 2* videos. The predicted price is calculated and clearly displayed for the user to see.

3.2: "The predicted price should be as accurate as possible"

This success criteria has only been **partially met**. In tests run, the program appears to generate a price in an error region of +/- £50,000, though occasionally, there is a larger discrepancy in price when predicting information already found within the dataset. This is because of a number of reasons, such as the following:

- An anomalous price/property within the dataset
- Not enough houses in a particular location/postcode
- Model trained on a smaller dataset leading to greater error

Summary

This shows that all success criteria for requirement 3 have been met or partially met as evidenced above. The predicted price has been displayed, and it is quite accurate as shown to be similar to other properties.

Requirement 4

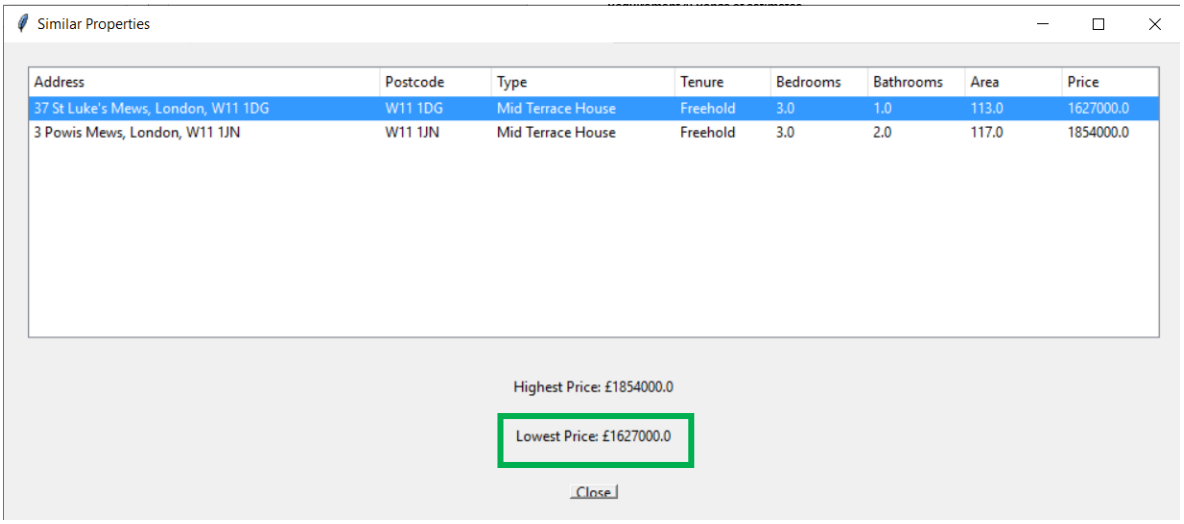
This requirement has been copied again below for reference:

Requirement 4: Range of estimates

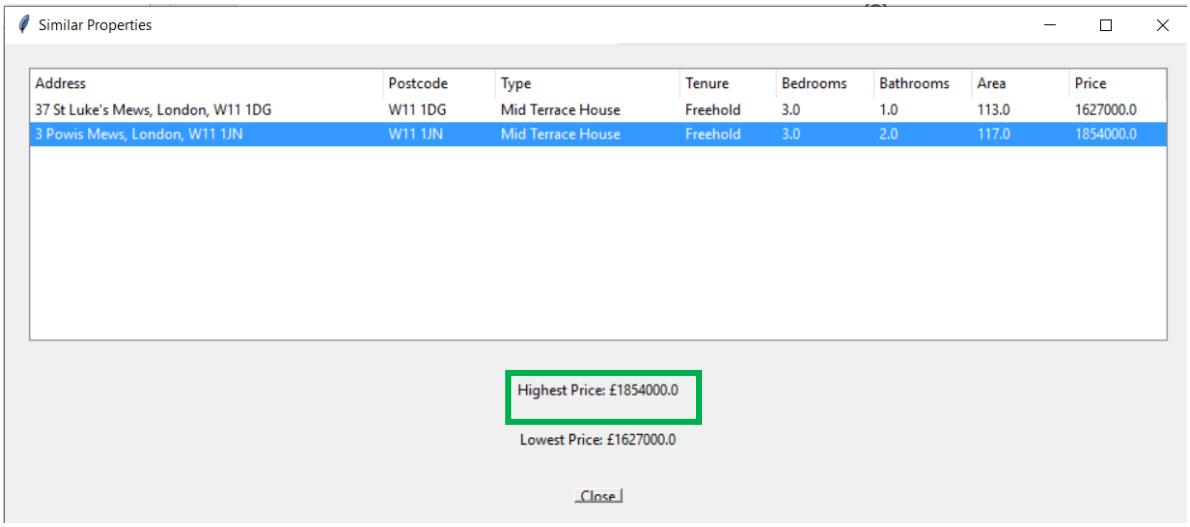
1. Displays lowest price of similar properties
2. Displays highest price of similar properties

Evidence:

Success criteria 4.1 – Displays lowest price of similar properties



Success criteria 4.2 – Displays highest price of similar properties



Explaination of how success criteria met:

4.1 – "Displays lowest price of similar properties"

This success criteria has been met as when displaying similar properties, the range of estimates function returns the lowest price of similar properties, and this is passed into the UI to be displayed as demonstrated in the screenshots above.

4.2 – "Displays highest price of similar properties"

Similar to success criteria 4.1, this has been met as the highest price of similar properties is returned into the UI and displayed along with the similar properties. This is evidenced above.

Summary

As demonstrated in the above evidence, all success criteria have been met for requirement 4

Requirement 5

Requirement 5 has been copied below for reference:

Requirement 5: Simple UI

1. UI is displayed when program starts
2. UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties
3. User can clearly see which buttons to press in the UI

Evidence

This requirement has been evidenced in its entirety in the Phase 3 testing section.

Explaination of how success criteria met:

5.1 – "UI is displayed when program starts"

This has been met as when the program is run, the UI opens a window and the submission for prediction interface is displayed. This is evidenced in *Video 3.1*.

5.2 – "UI has all the options the user needs, such as entering their data and obtaining the prediction or viewing other similar properties"

This has been met as the UI displays the buttons for entering the data, confirming the details and viewing other similar properties. When these buttons are clicked, they perform the specific intended function.

5.3 – "User can clearly see which buttons to press in the UI"

This has been **partially met** as although the text can be dynamically increased in size with the window, the text size cannot be controlled directly with a fixed window. This is something that could be implemented in future developments, with some sort of settings function that allows the user to manually change the size of the text. This will cater for everyone using the program, as the max size of the program will be reached when the window is the size of the screen.

Summary

This requirement has been fully met for two success criteria and partially met for the last success criteria. All success criteria have been evidenced and explained, with the partially met criteria explained above.

Requirement 6

Requirement 6 has been copied again here for reference:

Requirement 6: Similar properties

1. Program identifies similar properties
2. The properties share similar characteristics

Evidence

This requirement has been evidenced fully in the testing section of Phase 5: Presenting similar properties.

Explanation of how success criteria met:

6.1 – "Program identifies similar properties"

This success criteria has been met fully as the similar properties button opens the window with similar properties, all of which are in the same area. These properties are formatted in the form of a table and can be compared easily with other properties.

6.2 – "The properties share similar characteristics"

This success criteria has been fully met as the similar properties function searches through the entire dataset for properties which are in the same location, are of the same type and freehold and have similar numbers of bedrooms and bathrooms and area. The houses which match all of this are then outputted as similar properties. This means the properties must share similar characteristics.

Summary

This requirement has been evidenced fully, and therefore fully met.

Requirement 7

This requirement has been copied below for reference:

Requirement 7: Web scraping

1. Uses web scraping to extract information from property website
2. Extracts only necessary information
3. Stores necessary data into a suitable data structure

Evidence

This requirement has been fully met and evidenced in the Phase 1 Iterative testing method.

Explaination of how success criteria met:

7.1 – "Uses web scraping to extract information from property website"

This success criteria has been partially met. The program scrapes data from the website Rightmove, using the properties currently on sale. This allows the program to obtain data that is most relevant and representative of the market values. However, this solution fails to extract the postcode from the website, as this is loaded dynamically onto the page using javascript, rather than html. This makes it much harder to extract the information, so this program has not met the success criteria fully. This has been discussed earlier in the documentation in the Phase 1 Development.

7.2 – "Extracts only necessary information"

This success criteria has been met as the web scraping program scrapes through all the html, before extracting only the following information, which is the data fed into the machine learning model:

- Address
- Number of bedrooms and bathrooms
- Area
- Price

7.3 – "Stores necessary data into a suitable data structure"

This success criteria has been met as the program stores the data it has scraped into a panda data frame. This has been evidenced in Phase 1 development. This data structure is suitable as it can be permanently stored into a csv file which this program does as seen here:

```
df = pd.DataFrame.from_dict(data)
df.to_csv(r"london_house_price_data.csv", encoding="utf-8", header="true", index = False)
```

Summary

This requirement has been met either partially or fully and evidenced completely.

Requirement 8

The requirement has been copied again for reference:

Requirement 8: Machine learning model

1. Trains a machine learning model
2. Tests a machine learning model
3. Compares multiple machine learning models

Evidence

This requirement has been fully evidenced in the tests ran during Phase 2 iterative development.

Explanation of how success criteria met:

8.1 – "Trains a machine learning model"

This program meets this success criteria as the model is imported and trained using the data scraped from the website. The models used in this program are linear regression and random forest.

8.2 – "Tests a machine learning model"

This success criteria has been met as the machine learning model is tested using the testing libraries. Both models are tested using the testing dataset created from the original complete dataset.

8.3 – "Compares multiple machine learning models"

This success criteria has been met as the program uses test statistics such as mean squared error and r-squared to check how well each model does. Then the better, more accurate model is chosen and saved to be used for the user's prediction.

Summary

The requirement has been met fully and evidenced appropriately.

Further development to meet all success criteria

In further developments, the success criteria that were partially met, or not met at all can be fully met.

Success criteria 2.3 – "Map is navigable":

In future developments, the program can use an API, which it can directly connect to, and not just load an image. This means that the user can zoom in and out, and move from place to place. The current program uses some code to just display the obtained image, however, in further development, the code should be improved to have some sort of frame that allows for complete interaction with the map.

Success criteria 3.2 – "The predicted price should be as accurate as possible":

In future developments, the program can be trained on more houses, and as well as modifying the code to scrape more houses, the machine learning model can be modified to train on a larger proportion of this dataset. Furthermore, more models can be incorporated into the program, and the best model chosen from them. This could mean another model is more suitable for modelling the house prices, producing a better result.

Success criteria 5.3 – "User can clearly see which buttons to press in the UI":

In future developments, this program can be developed to include a settings option, which allows the user to manually increase the size of the text, and perhaps buttons to satisfy their needs. The colour could also perhaps be modified. This is discussed further in the usability section.

Success criteria 7.1 – "Uses web scraping to extract information from property website":

In future developments, additional web scraping functions could employed which would enable the scraping of data which is dynamically loaded. This can perhaps be done using a javascript web scraper, which would simulate a browser to load all the information, even those loaded dynamically. This would allow that data to be scraped properly, rather than ommitting it as I have done so far.

Usability features

Evidence

There are a number of additional usability features within my program. These include the following:

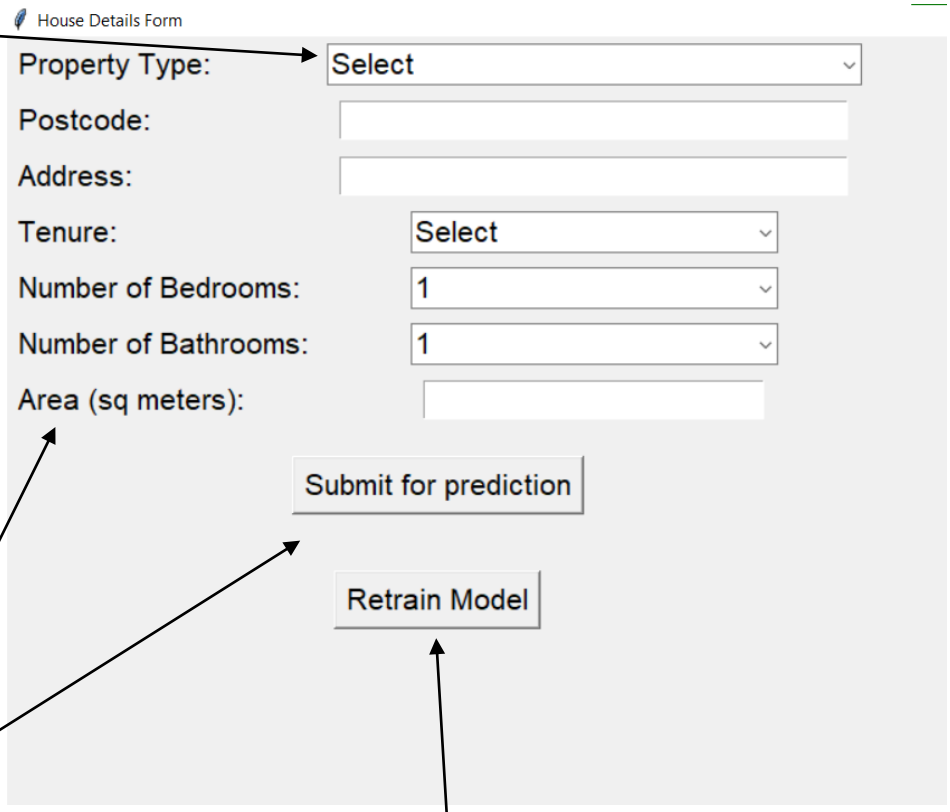
- Clear labelled buttons
- Drop down menus for inputs
- Dynamically increasing size based on the window
- Clear headings and titles of sections
- Formatting based on different sections
- Use of tables to display data for ease of understanding

These are **demonstrated** and **justified** below:

Drop down menus to show the user which option to select. This also means validation is not required for these particular inputs

Large clear font size to display to the user the inputs required. The font size is also adjustable as the window is enlarged and reduced. This allows those who have poorer eyesight to see.

Large buttons for the user to submit their information for prediction, or choose to retrain the model

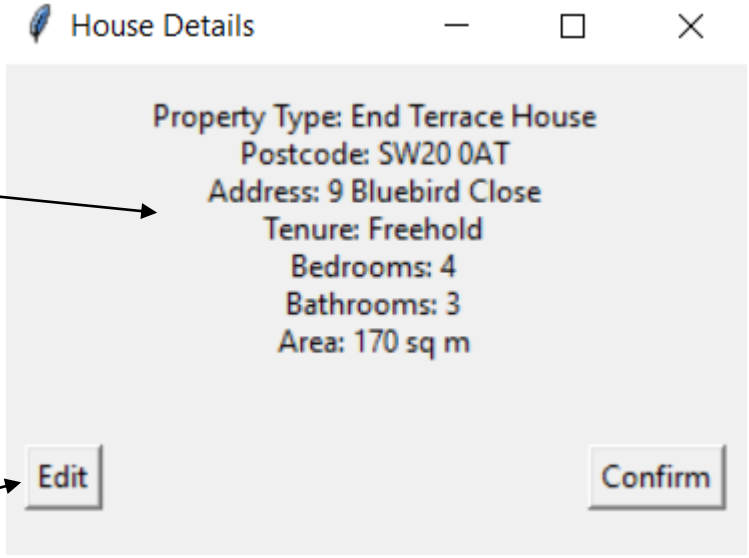


Dynamically sized text based on the window size – evidenced in *Video 3.3*. Again, this helps those unable to see clearly to increase text size

After the user has submitted their property for a prediction, the program also includes some usability features:

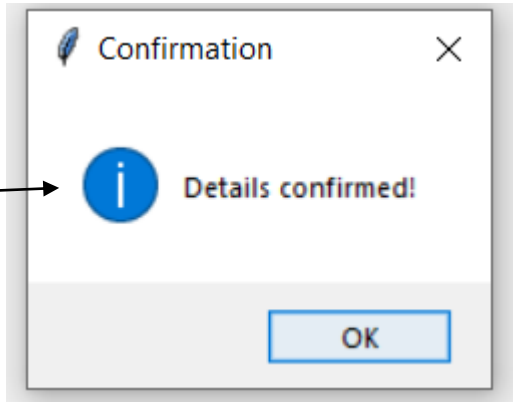
Displays user details again for confirmation message. This means the user can check their details to make sure they are correct.

Two clearly labelled buttons to either confirm the details or change them. This means the users can make errors and still have them fixed before entering for a prediction.

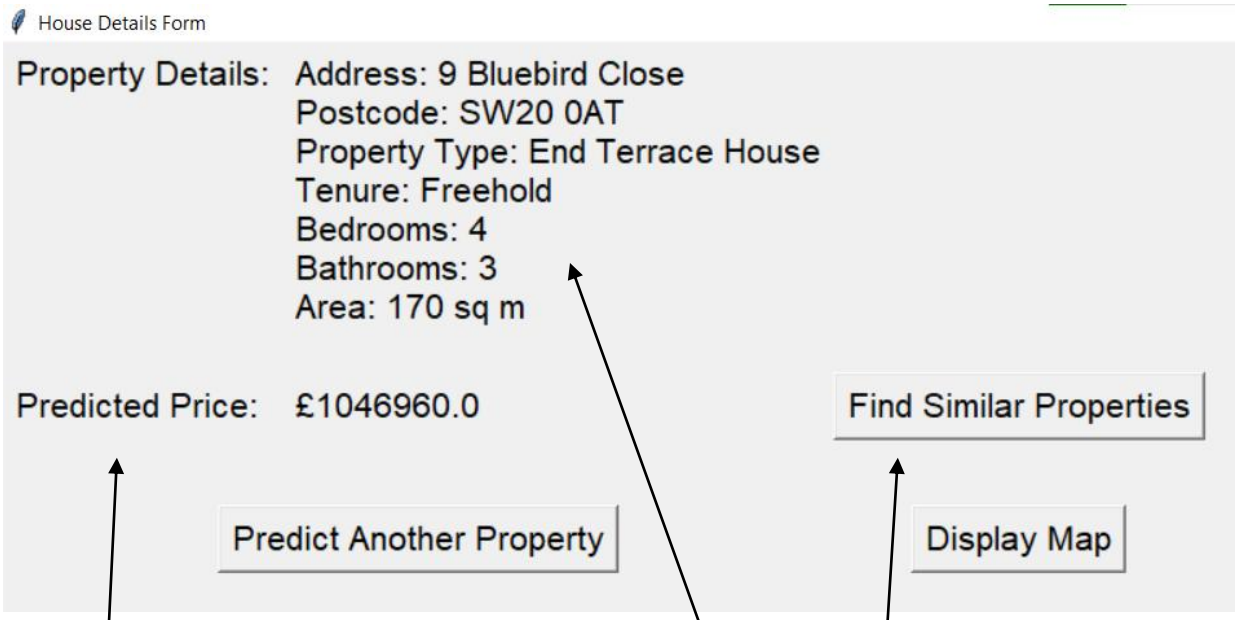


When displaying the predicted price, the program has the following usability features:

A confirmation message is displayed, telling the user that their details have been confirmed. This helps them to know with certainty that their entered details have gone through



When displaying the calculated predicted price, there are usability features again:

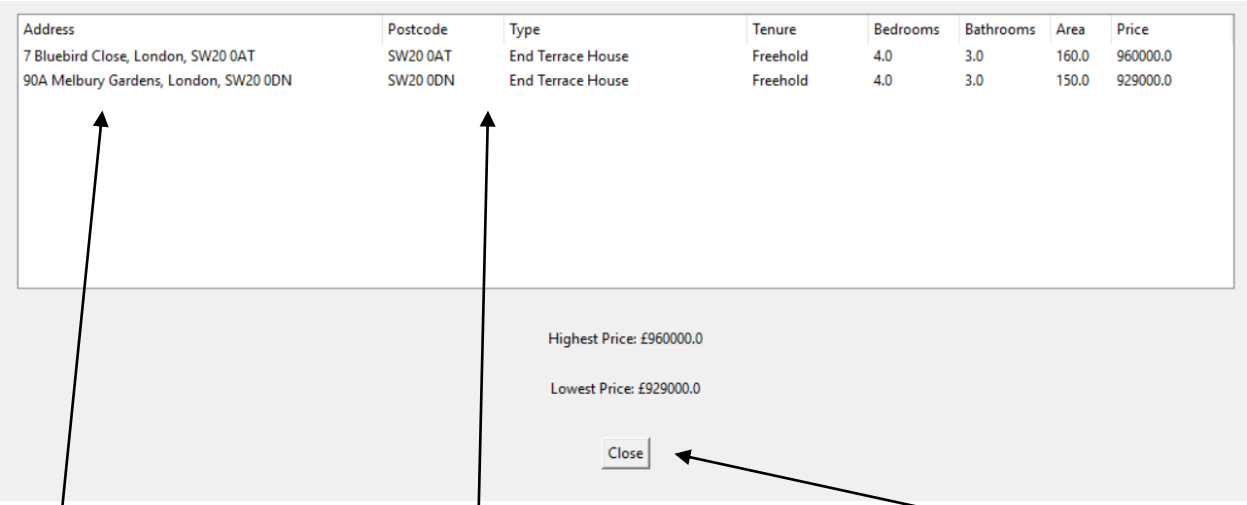


Clearly labelled predicted price as a heading to allow the user to quickly find the information they are looking for. The predicted price and property details have a slight gap to help with formatting on the page, making it easy to differentiate a section from another.

Property details are displayed yet again, to allow the user to see both the price and the property they submitted next to each other. This helps them to be sure that the price corresponds to the property that they wanted to predict.

Three clearly labelled buttons that allow the user to access certain functions within the program. These buttons are spaced apart and are clear in their purpose and use, making it easier for the user to choose what to do next.

Features when displaying similar properties can be seen below:



Multiple similar properties are displayed in this information table. This allows the user to directly compare their property with multiple other properties of similar nature.

Clearly divided segments with a label in the table to separate each column and row in the similar properties table. This means user can see clearly the different properties and the specific information they want.

Button to close the window when the user has identified all the information they wish to look at.

Evaluating success

The functionality requirements have been copied again here for reference:

Functionality / Usability

1. Runs without crashing
2. Allows user to input parameters
3. Validates all inputs

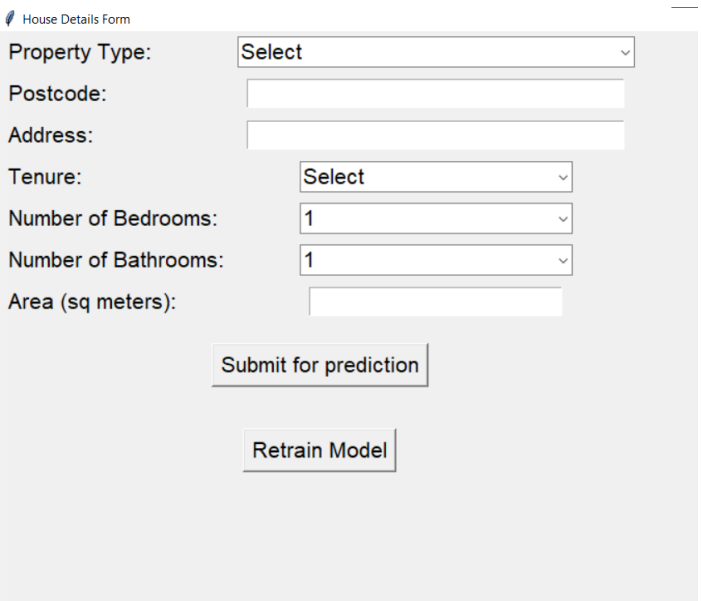
Evidence:

Functionality 1 – "Runs without crashing":

This can be evidenced in both the *post development testing videos*, as even when incorrect or invalid information is entered, the program simply displays and error message without completely crashing.

Functionality 2 – "Allows user to input parameters":

This can be seen in the submission page for prediction, as shown below. The user can enter their details about their property into the correct areas. This has also been evidenced in both *post development testing videos*.



Functionality 3 – "Validates all inputs"

This has been evidenced in the *validation 1* and *validation 2* videos.

Summary:

All functionality requirements have been fully met and evidenced appropriately.

Further development of usability features

Further development can be done to improve usability of the program:

Colours

The current colour scheme is very simple and plain. This was done to maintain professionalism, however in future developments, following stakeholder feedback, colours can be implemented to make the UI more appealing. Colours could also be added to highlight certain headings or titles, giving the user even more clarity on how to use the program.

Font size

As discussed previously in the Further development to meet success criteria section, there is currently no way to manually adjust the font size. This makes it harder for those who need extra large font to read the information on the screen. To combat this, a settings page can be created, where the user can manually adjust the font size if needed. This can be implemented quite easily.

Windows

Another potential area for further development is the use of multiple windows. Currently, a new window is created to display any new information such as the similar properties or the map. Potentially, further development can be done to create a UI, where there is only one window, and all the information can be displayed on the same window. This means that the layout would be more manageable and customizable, to fit a particular window.

Maintenance issues and limitations of the solution

There are a number of potential maintenance issues which are limitations of this solution. These include but are not limited to the following:

1. Processing limitations when training the model

The solution currently relies on using the scraped dataset to train a machine learning model. Training two separate models takes a lot of processing power and time, and as a result my solution only uses a randomly selected sample from the dataset to train my model. This randomly selected sample is only a small fraction of the entire dataset, meaning my model will lose accuracy when predicting a price, and will not accept certain postcodes from locations it does not recognise.

2. Restricted postcodes

This solution only allows for a prediction of a restricted area of location. This is because the postcode is one hot encoded into the model, meaning that there is a set of postcodes the model recognises, and if the postcode is not in this list, it will fail the prediction and tell the user their postcode is invalid.

3. Restricted web scraping

My web scraping function scrapes data from the website Rightmove. In its current iteration, the model only requests information about properties in specific London boroughs. This means that the house price prediction is restricted to the house prices in London and is reflective of those prices. In general, this can mean house prices are slightly more inflated compared to houses not located in London.

4. Reliance on website for web scraping

By its nature, web scraping is heavily reliant on the html of the website being the same. This means that if the website were to change, the web scraping function of the program would not work as intended and may return nothing when called. This will cause a big problem when training the model, as there will be no properties to train on.

5. Time taken to retrain the model

In order to retrain the model, the program needs to call the web scraping function and scrape data from all properties in the area, before feeding the data into the model for the model to be trained. This takes a very long time, and while the model is retraining, the user cannot use any other parts of the program.

Future maintenance:

This coded solution is modular in the fact that it uses multiple files, with functions that perform specific tasks. The code is also well annotated, with all functions and variables named appropriately and informatively. This makes it easy for additional developments in the future, where future developers should find it easy to optimise or modify certain aspects of the code without modifying the core functionality of the program. Careful steps have been deliberately carried out to make this possible.

Improvements to the solution

Improvements can be made to the solution based on the previously discussed limitations to the solution.

1. Processing limitations when training the model

For this particular limitation, there isn't any simple fixes to address this issue. Potentially, the code for the machine learning models can be optimised slightly, or parameters adjusted to produce an accurate model in less time. Furthermore, different models could be looked at which may produce more accurate results in a shorter space of time.

2. Restricted postcodes

This limitation to the solution is again hard to fix, as it is related directly to the dataset used to train the model. Potentially, the machine learning models can be trained with a larger dataset, which covers more land area and is more diverse in location. This can mean there are more postcodes that the model will recognise, hence improving the solution.

3. Restricted web scraping

An improvement to this solution would be to change the URLs that are requested to also include properties not within London. However, this will dramatically increase the time taken to scrape the data and also train the model. Hence, this is only a potential improvement to consider and must be evaluated carefully before implementing.

4. Reliance on website for web scraping

A potential improvement can be to scrape data from different sites, such as Zoopla as well as Rightmove. This means that hypothetically if the website were to change, data from other websites can still be used to train a model. This means that there is no "downtime" where the program will not be able to retrain the model when the website is changed. However, it is important to consider the data obtained from both websites – ensuring that they are compatible with each other, and do not duplicate, and all the data is in the same format.

5. Time taken to retrain the model

As mentioned in the Requirement 7 evaluation section, the time taken to scrape the data for retraining the model can be dramatically reduced if another web scraping technique was employed, such as the use of a virtual browser. This means that all information can be accessed on one link, rather than having to request multiple links for just one house. This may speed up the time taken to retrain the model, and improve the solution

Final code

webScraper.py

```python
#Imports
import requests #Used to connect to the website to scrape data from
from bs4 import BeautifulSoup #This is a web scraping library
import pandas as pd #This is a dataset analysis library

# Rightmove area codes for London boroughs
# A dictionary that stores all the boroughs of London and their corresponding regional codes

boroughs = {
    "City of London":"5E61224",
    "City of Westminster":"5E61233",
    "Kensington and Chelsea":"5E61229",
    "Hammersmith and Fulham":"5E61407",
    "Wandsworth":"5E93977",
    "Lambeth":"5E93971",
    "Southwark":"5E61518",
    "Tower Hamlets":"5E61417",
    "Hackney":"5E93953",
    "Islington":"5E93965",
    "Camden":"5E93941",
    "Brent":"5E93935",
    "Ealing":"5E93947",
    "Hounslow":"5E93962",
    "Richmond upon Thames":"5E61415",
    "Kingston upon Thames":"5E93968",
    "Merton":"5E61414",
    "Sutton": "5E93974",
    "Croydon": "5E93944",
    "Bromley": "5E93938",
    "Lewisham": "5E61413",
    "Greenwich": "5E61226",
    "Bexley": "5E93932",
    "Havering": "5E61228",
    "Barking and Dagenham": "5E61400",
    "Redbridge": "5E61537",
    "Newham": "5E61231",
    "Waltham Forest": "5E61232",
    "Haringey": "5E61227",
    "Enfield": "5E93950",
    "Barnet": "5E93929",
    "Harrow": "5E93956",
    "Hillingdon": "5E93959"

}

```

```python
#Setting up the preliminary lists to store all the data of the properties
all_properties_address = []
all_properties_price = []
all_properties_links = []
all_properties_type = []
all_properties_bedrooms = []
all_properties_bathrooms = []
all_properties_size = []

#This function will be used to locate and extract all the data from the webpage
def data_retrieval_development(properties_html):

    #This loop will go through each individual properties' html found in the full list
    for property_no in range(len(properties_html)):
        current_property = properties_html[property_no]



        #Obtaining the link to the webpage of the specific property

        #This finds and extracts the class where the link is held
        property_info = current_property.find("a", class_="propertyCard-link")

        #Concatenate the individual link to form a proper weblink
        link = "https://www.rightmove.co.uk" + property_info.attrs["href"]

        #Append this link to our list
        all_properties_links.append(link)



        #Obtaining the address of a specific property
        address = (
            property_info.find("address", class_="propertyCard-address")
            .get_text() #This will extract the text from the html
            .strip() #This ensures only the desired text is saved
        )
        all_properties_address.append(address)

        #Obtaining the price of a specific property
        price = (
            current_property.find("div", class_="propertyCard-priceValue")
            .get_text() #Extract the price
            .strip() #Ensure it is only the price
        )
        price = price[1:]
        all_properties_price.append(price)
```

```
91
92    # Set up a function and loop to iterate through all the boroughs
93  def property_scraping(max_search):
94      for borough in list(boroughs.values()):
95
96          # This will initialise the index to match the index on the rightmove website
97          index = 0
98
99          # This is to show what the program is currently trying to scrape
100         current_key = [key for key,value in boroughs.items() if value == borough][0]
101         print(current_key)
102
103         # Each borough will show a maximum of 42 pages hence this loops through all
104         for pages in range(41):
105
106             #This is needed to match the header of the browser when scraping
107             user_headers = {
108                 "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
                    like Gecko) Chrome/89.0.4389.114 Safari/537.36"
109             }
110
111             #The links on the website are slightly different on the first page compared to others
112             if index == 0:
113                 rightmove_link = f"https://www.rightmove.co.uk/property-for-sale/find.html?
                    locationIdentifier=REGION%{borough}&sortType=6&propertyTypes=&includeSSTC=false&
                    mustHave=&dontShow=&furnishTypes=&keywords="
114             elif index != 0:
115                 rightmove_link = f"https://www.rightmove.co.uk/property-for-sale/find.html?
                    locationIdentifier=REGION%{borough}&sortType=6&index={index}&propertyTypes=&
                    includeSSTC=false&mustHave=&dontShow=&furnishTypes=&keywords="
116
117             #The requests with the specific link and headers are sent to the website
118             request = request.get(rightmove_link, headers=user_headers)
119             request.html.render()
120
121             #This receives all the html of the web page using the BeautifulSoup library
122             page_html = BeautifulSoup(request.html.html, "html.parser")
123
124             #Having identified the class data is found in, this returns all relevant html
125             properties_html = page_html.find_all("div", class_="l-searchResult is-list")
126
```

```
127             #Find total number of listings displayed
128             total_no_listings = (
129                 page_html.find("span", {"class": "searchHeader-resultCount"})
130                 .get_text()
131             )
132             #Obtain just the integer value so can be used later
133             total_no_listings = int(total_no_listings.replace(",", ""))
134 
135             #Call the function to extract the data from the html
136             data_retrieval_development(properties_html)
137 
138             # For development purposes to show I am actually scraping
139             print(f"{pages + 1} pages scraped")
140             print(f"You have {total_no_listings - index} listings left")
141 
142             # Jump to the next page of properties on website
143             index = index + 24
144 
145             #To stop the program when there are no more listings
146             if index >= total_no_listings or index >= max_search:
147                 break
148 
149 
150 
151 #Function to extract the more detailed information from each specific property
152 def detailed_info_scrape():
153     #Iterate through all the properties found in the list of links
154     property_no = 0
155     while property_no < len(all_properties_links):
156 
157         property_link = all_properties_links[property_no]
158 
159         user_headers = {
160             "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
                Gecko) Chrome/89.0.4389.114 Safari/537.36"
161         }
162         request = requests.get(property_link, headers=user_headers)
163         request.raise_for_status()
164 
165         #This receives the html of the property web page using the BeautifulSoup library
166         property_html = BeautifulSoup(request.text, "html.parser")
167         properties_html = property_html.find_all("p", class_="_1hV1kqpVceE9m-QrX_hWDN")
168 
```

```
169
170         #This validates that all information is present otherwise I discard
171         if len(properties_html)!= 5:
172             all_properties_links.pop(property_no)
173             all_properties_price.pop(property_no)
174             all_properties_address.pop(property_no)
175
176         #This will extract the information and append to the relevant list
177         else:
178             all_properties_type.append(properties_html[0].get_text().strip())
179             all_properties_bedrooms.append(properties_html[1].get_text().strip())
180             all_properties_bathrooms.append(properties_html[2].get_text().strip())
181             size = properties_html[3].get_text().strip()
182             # Validate if the size is present or not
183             if size == "Ask agent":
184                 all_properties_size.append(-1)
185             else:
186                 all_properties_size.append(int(size[:-6]))
187             property_no+=1
188
189     #Function to create a dataset from the lists
190     def dataset_creation():
191         data = {
192             "Address": all_properties_address,
193             "Type": all_properties_type,
194             "Bedrooms": all_properties_bedrooms,
195             "Bathrooms": all_properties_bathrooms,
196             "Size": all_properties_size,
197             "Price": all_properties_price
198         }
199
200         # Write the data to a csv file
201         df = pd.DataFrame.from_dict(data)
202         df.to_csv(r"london_house_price_data.csv", encoding="utf-8", header="true", index = False)
203
204     #Main function from which all other functions are called
205     def webScraper():
206         property_scraping(1000)
207         detailed_info_scrape()
208         dataset_creation()
209
```

trainModel.py

```python
# Here I have imported libraries, which can then be used

import pandas as pd
import numpy as np
import math as math
import pickle
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score


#This function cleans the data
def clean_data(df):
    df = df.dropna()
    return df


#This function preprocesses the data using one hot encoding
def preprocess_data(df):
    df = pd.get_dummies(df, columns=['postcode','outcode','tenure','propertyType'])
    return df

def split_data(df):
    #Split the data into dependent and independent variables
    y = df['currentPrice']
    x = df.drop(['currentPrice', 'fullAddress'], axis=1)

    #Split the data into training and testing data
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
    return X_train, X_test, y_train, y_test

# Function to train and evaluate models
def train_and_evaluate_models(df):
    # Split the data into dependent and independent variables
    y = df['currentPrice']
    x = df.drop(['currentPrice', 'fullAddress'], axis=1)

    # Split the data into training and testing data
    X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)

```

```
    # Train and evaluate Linear Regression model
    lr = LinearRegression()
    lr.fit(X_train, y_train)
    y_lr_train_pred = lr.predict(X_train)
    y_lr_test_pred = lr.predict(X_test)

    # Calculate the test staistics
    lr_train_mse = mean_squared_error(y_train, y_lr_train_pred)
    lr_train_r2 = r2_score(y_train, y_lr_train_pred)
    lr_test_mse = mean_squared_error(y_test, y_lr_test_pred)
    lr_test_r2 = r2_score(y_test, y_lr_test_pred)
    lr_results = pd.DataFrame(['Linear regression', lr_train_mse, lr_train_r2, lr_test_mse,
    lr_test_r2]).transpose()
    lr_results.columns = ['Method', 'Training MSE', 'Training R2', 'Test MSE', 'Test R2']

    # Train and evaluate Random Forest model
    rf = RandomForestRegressor(random_state=100)
    rf.fit(X_train, y_train)
    y_rf_train_pred = rf.predict(X_train)
    y_rf_test_pred = rf.predict(X_test)

    # Calculate the test statistics
    rf_train_mse = mean_squared_error(y_train, y_rf_train_pred)
    rf_train_r2 = r2_score(y_train, y_rf_train_pred)
    rf_test_mse = mean_squared_error(y_test, y_rf_test_pred)
    rf_test_r2 = r2_score(y_test, y_rf_test_pred)
    rf_results = pd.DataFrame(['Random forest', rf_train_mse, rf_train_r2, rf_test_mse,
    rf_test_r2]).transpose()
    rf_results.columns = ['Method', 'Training MSE', 'Training R2', 'Test MSE', 'Test R2']

    # Save the better model
    if rf_test_r2 > lr_test_r2:
        with open('model.pkl', 'wb') as f:
            pickle.dump(rf, f)
    else:
        with open('model.pkl', 'wb') as f:
            pickle.dump(lr, f)

```

```
79
80  # Main function to execute the workflow
81  def main():
82      df = pd.read_csv('london_house_price_data.csv')
83
84      # Clean and sample the data
85      df = clean_data(df)
86      df = df.sample(frac=0.01)
87
88      # Prepare the data for model training
89      df.to_csv('sampled_data.csv', index=False)
90      df_dummies = preprocess_data(df)
91      df_dummies.to_csv('sampled_data_dummies.csv', index=False)
92
93      # Train the model
94      train_and_evaluate_models(df_dummies)
95
```

ui.py

```python
# Imports
import tkinter as tk
from tkinter import ttk, messagebox
import pandas as pd
import importlib.util
import threading

# Global list to store confirmed details
confirmed_details_list = []

# Import the predict_price function from phase 4 file
spec = importlib.util.spec_from_file_location("predictPrice", "predictPrice.py")
predictPrice = importlib.util.module_from_spec(spec)
spec.loader.exec_module(predictPrice)

# Import the similar properties function from phase 5 file
spec2 = importlib.util.spec_from_file_location("similarProperties", "similarProperties.py")
similarProperties = importlib.util.module_from_spec(spec2)
spec2.loader.exec_module(similarProperties)

# Import the range estimates function from phase 6 file
spec3 = importlib.util.spec_from_file_location("rangeEstimates", "rangeEstimates.py")
rangeEstimates = importlib.util.module_from_spec(spec3)
spec3.loader.exec_module(rangeEstimates)

# Import the map function from map.py
spec4 = importlib.util.spec_from_file_location("map", "map.py")
map = importlib.util.module_from_spec(spec4)
spec4.loader.exec_module(map)

# Import the web scraper function from webScraper.py
spec5 = importlib.util.spec_from_file_location("webScraper", "webScraper.py")
webScraper = importlib.util.module_from_spec(spec5)
spec5.loader.exec_module(webScraper)

# Import the model trainer function from trainModel.py
spec6 = importlib.util.spec_from_file_location("trainModel", "trainModel.py")
trainModel = importlib.util.module_from_spec(spec6)
spec6.loader.exec_module(trainModel)

# Function to get options for property types, postcodes, and tenures from the CSV file
def get_options():
    df = pd.read_csv('sampled_data.csv')
    df = df.dropna()

```

```
    # Get unique values for propertyType, postcode, and tenure
    property_types = df['propertyType'].unique()
    tenures = df['tenure'].unique()

    # Convert the numpy arrays to lists
    property_types = property_types.tolist()
    tenures = tenures.tolist()
    return property_types, tenures

# Function to create the main window
def create_main_window():
    root = tk.Tk()
    root.title("House Details Form")
    root.state("zoomed")  # Maximize the window
    root.configure(bg="#f0f0f0")  # Light grey background
    root.bind("<Configure>", lambda event: adjust_font_size(root))
    return root

# Function to adjust font size dynamically
def adjust_font_size(root):
    width = root.winfo_width()
    height = root.winfo_height()
    new_size = max(10, int(min(width, height) / 50))
    global font_style
    font_style = ("Helvetica", new_size)
    for widget in root.winfo_children():
        if isinstance(widget, (tk.Label, tk.Button, ttk.Combobox, tk.Entry)):
            widget.config(font=font_style)

# Function to create the widgets in the main window
def create_widgets(root, property_types, tenures):
    # Property Type
    tk.Label(root, text="Property Type:").grid(row=0, column=0, pady=5, padx=5, sticky="w")
    property_type_var = tk.StringVar(value="Select")
    property_type_menu = ttk.Combobox(root, textvariable=property_type_var,
    values=property_types, width=30)
    property_type_menu.grid(row=0, column=1, pady=5, padx=5)

    # Postcode
    tk.Label(root, text="Postcode:").grid(row=1, column=0, pady=5, padx=5, sticky="w")
    postcode_var = tk.StringVar()
    postcode_menu = tk.Entry(root, textvariable=postcode_var, width=30)
    postcode_menu.grid(row=1, column=1, pady=5, padx=5)

```

```
    # Address
    tk.Label(root, text="Address:").grid(row=2, column=0, pady=5, padx=5, sticky="w")
    address_var = tk.StringVar()
    address_entry = tk.Entry(root, textvariable=address_var, width=30)
    address_entry.grid(row=2, column=1, pady=5, padx=5)

    # Tenure
    tk.Label(root, text="Tenure:").grid(row=3, column=0, pady=5, padx=5, sticky="w")
    tenure_var = tk.StringVar(value="Select")
    tenure_menu = ttk.Combobox(root, textvariable=tenure_var, values=tenures)
    tenure_menu.grid(row=3, column=1, pady=5, padx=5)

    # Number of Bedrooms
    tk.Label(root, text="Number of Bedrooms:").grid(row=4, column=0, pady=5, padx=5, sticky="w")
    bedrooms_var = tk.IntVar(value=1)
    bedrooms_menu = ttk.Combobox(root, textvariable=bedrooms_var, values=list(range(1, 11)))
    bedrooms_menu.grid(row=4, column=1, pady=5, padx=5)

    # Number of Bathrooms
    tk.Label(root, text="Number of Bathrooms:").grid(row=5, column=0, pady=5, padx=5,
    sticky="w")
    bathrooms_var = tk.IntVar(value=1)
    bathrooms_menu = ttk.Combobox(root, textvariable=bathrooms_var, values=list(range(1, 11)))
    bathrooms_menu.grid(row=5, column=1, pady=5, padx=5)

    # Area
    tk.Label(root, text="Area (sq meters):").grid(row=6, column=0, pady=5, padx=5, sticky="w")
    area_var = tk.StringVar()
    area_entry = tk.Entry(root, textvariable=area_var)
    area_entry.grid(row=6, column=1, pady=5, padx=5)

    # Submit Button
    submit_button = tk.Button(root, text="Submit for prediction", command=lambda: submit_details
    (root, property_type_var, postcode_var, address_var, tenure_var, bedrooms_var,
    bathrooms_var, area_var))
    submit_button.grid(row=7, column=0, columnspan=2, pady=20)

    # Button to retrain the model
    retrain_button = tk.Button(root, text="Retrain Model", command=lambda: reset_model(root))
    retrain_button.grid(row=8, column=0, columnspan=2, pady=20)

```

```python
# Function to retrain the model
def reset_model(root):
    # Clear the main window
    for widget in root.winfo_children():
        widget.destroy()

    # Refresh the GUI so everything is deleted
    root.update()

    # Show loading screen on the main window
    loading_label = tk.Label(root, text="Retraining model, please wait...", font=("Helvetica", 16))
    loading_label.pack(pady=20, padx=20)
    root.update_idletasks()

    # Run the prediction in a separate thread
    threading.Thread(target=lambda: retrain_model(root)).start()

# Function to retrain the model
def retrain_model(root):
    # Call the web scraper function
    webScraper.webScraper()
    trainModel.main()

    # Message to show the model has been retrained
    messagebox.showinfo("Model Retrained", "Model has been retrained!")

    # Reseting the prediction
    reset(root)

# Function to handle the submission of details
def submit_details(root, property_type_var, postcode_var, address_var, tenure_var, bedrooms_var, bathrooms_var, area_var):
    # Get the entered details
    property_type = property_type_var.get()
    postcode = postcode_var.get()
    address = address_var.get()
    tenure = tenure_var.get()
    bedrooms = bedrooms_var.get()
    bathrooms = bathrooms_var.get()
    area = area_var.get()

```

```
    # Show the entered details in a custom pop-up window
    details_window = tk.Toplevel(root)
    details_window.title("House Details")
    details_window.geometry("300x200")

    # Display the entered details
    details = (
        f"Property Type: {property_type}\n"
        f"Postcode: {postcode}\n"
        f"Address: {address}\n"
        f"Tenure: {tenure}\n"
        f"Bedrooms: {bedrooms}\n"
        f"Bathrooms: {bathrooms}\n"
        f"Area: {area} sq m"
    )

    tk.Label(details_window, text=details).pack(pady=10)

    def edit_details():
        details_window.destroy()

    def confirm_details():
        user_details = [property_type, postcode, address, tenure, bedrooms, bathrooms, area]
        # Validate the inputs
        validation = predictPrice.validate_inputs(user_details)
        if validation == "valid":
            confirmed_details_list.append(user_details)
            messagebox.showinfo("Confirmation", "Details confirmed!")
            details_window.destroy()

            # Clear the main window
            for widget in root.winfo_children():
                widget.destroy()

            # Refresh the GUI so everything is deleted
            root.update()

            # Show loading screen on the main window
            loading_label = tk.Label(root, text="Calculating value, please wait...", font=
            ("Helvetica", 16))
            loading_label.pack(pady=20, padx=20)
            root.update_idletasks()

```

```
209                 # Run the prediction in a separate thread
210                 threading.Thread(target=lambda: run_prediction(root, user_details, loading_label)).
                    start()
211
212             # If details are not valid, return to original submission form
213             elif validation == "address":
214                 messagebox.showerror("Error", "Please enter your address!")
215                 details_window.destroy()
216             elif validation == "area":
217                 messagebox.showerror("Error", "Please enter the area!")
218                 details_window.destroy()
219             elif validation == "postcode":
220                 messagebox.showerror("Error", "Postcode not found! \n(Warning: Model only trained
                    on certain postcodes)")
221                 details_window.destroy()
222             elif validation == "tenure":
223                 messagebox.showerror("Error", "Please select a tenure!")
224                 details_window.destroy()
225             elif validation == "type":
226                 messagebox.showerror("Error", "Please select a property type!")
227                 details_window.destroy()
228             elif validation == "area_type":
229                 messagebox.showerror("Error", "Please enter a number for area!")
230                 details_window.destroy()
231
232         tk.Button(details_window, text="Edit", command=edit_details).pack(side=tk.LEFT, padx=10,
            pady=10)
233         tk.Button(details_window, text="Confirm", command=confirm_details).pack(side=tk.RIGHT,
            padx=10, pady=10)
234
235     # Function to run the prediction in a separate thread
236     def run_prediction(root, user_details, loading_label):
237         # Get the predicted price
238         prediction = get_predicted_price(user_details)
239
240         # Destroy loading screen
241         loading_label.destroy()
242
243         # Display the predicted price
244         display_prediction(root, prediction, user_details)
245
```

```python
# Function to get the predicted price from nea_phase_4.py
def get_predicted_price(user_details):
    return predictPrice.predict_price(user_details)

# Function to display the predicted price
def display_prediction(root, predicted_price, user_details):
    # Display the details about the users property
    tk.Label(root, text="Property Details:",).grid(row=0, column=0, pady=5, padx=5, sticky="nw")
    details = (
        f"Address: {user_details[2]}\n"
        f"Postcode: {user_details[1]}\n"
        f"Property Type: {user_details[0]}\n"
        f"Tenure: {user_details[3]}\n"
        f"Bedrooms: {user_details[4]}\n"
        f"Bathrooms: {user_details[5]}\n"
        f"Area: {user_details[6]} sq m"
    )

    tk.Label(root, text=details, anchor="w", justify="left").grid(row=0, column=1, pady=5,
    padx=5, sticky="w")


    # Display the predicted price
    tk.Label(root, text="Predicted Price:").grid(row=1, column=0, pady=5, padx=5, sticky="w")
    tk.Label(root, text=f"£{predicted_price}").grid(row=1, column=1, pady=5, padx=5, sticky="w")

    # Button to find similar properties
    similar_button = tk.Button(root, text="Find Similar Properties", command=lambda:
    find_similar_properties(root))
    similar_button.grid(row=1, column=2, columnspan=2, pady=20)

    # Button to predict another property
    another_button = tk.Button(root, text="Predict Another Property", command=lambda: reset
    (root))
    another_button.grid(row=2, column=0, columnspan=2, pady=20)

    # Button to display the map
    map_button = tk.Button(root, text="Display Map", command=lambda: map.create_window
    (user_details[1]))
    map_button.grid(row=2, column=2, columnspan=2, pady=20)

```

```
283  def reset(root):
284      # Clear the main window
285      for widget in root.winfo_children():
286          widget.destroy()
287
288      # Clear the confirmed details list
289      confirmed_details_list.clear()
290
291      # Refresh the GUI so everything is deleted
292      root.update()
293
294      # Get the property types and tenures
295      property_types, tenures = get_options()
296
297      # Create the widgets again
298      create_widgets(root, property_types, tenures)
299
300  # Function to find similar properties
301  def find_similar_properties(root):
302      # Get the user details
303      user_details = confirmed_details_list[0]
304
305      # Run the similar properties search in a separate thread
306      threading.Thread(target=lambda: run_similar_properties_search(root, user_details)).start()
307
308  # Function to run the similar properties search in a separate thread
309  def run_similar_properties_search(root, user_details):
310      # Find similar properties
311      similar_properties = similarProperties.find_similar(user_details)
312
313      # Display the similar properties
314      display_similar_properties(root, similar_properties)
315
316  # Function to display the similar properties
317  def display_similar_properties(root, similar_properties):
318      if similar_properties is False:
319          messagebox.showinfo("No Similar Properties", "No similar properties found!")
320      else:
321          # Display the similar properties
322          similar_properties_window = tk.Toplevel(root)
323          similar_properties_window.title("Similar Properties")
324          similar_properties_window.geometry("500x400")
325
```

```
        # Create a treeview to display the similar properties
        tree = ttk.Treeview(similar_properties_window)
        tree["columns"] = ("Address", "Postcode", "Type", "Tenure", "Bedrooms", "Bathrooms",
        "Area", "Price")
        tree.column("#0", width=0, stretch=tk.NO)
        tree.column("Address", anchor=tk.W, width=300)
        tree.column("Postcode", anchor=tk.W, width=100)
        tree.column("Type", anchor=tk.W, width=200)
        tree.column("Tenure", anchor=tk.W, width=100)
        tree.column("Bedrooms", anchor=tk.W, width=75)
        tree.column("Bathrooms", anchor=tk.W, width=75)
        tree.column("Area", anchor=tk.W, width=50)
        tree.column("Price", anchor=tk.W, width=100)

        tree.heading("#0", text="", anchor=tk.W)
        tree.heading("Address", text="Address", anchor=tk.W)
        tree.heading("Postcode", text="Postcode", anchor=tk.W)
        tree.heading("Type", text="Type", anchor=tk.W)
        tree.heading("Tenure", text="Tenure", anchor=tk.W)
        tree.heading("Bedrooms", text="Bedrooms", anchor=tk.W)
        tree.heading("Bathrooms", text="Bathrooms", anchor=tk.W)
        tree.heading("Area", text="Area", anchor=tk.W)
        tree.heading("Price", text="Price", anchor=tk.W)

        for index, row in similar_properties.iterrows():
            tree.insert("", tk.END, values=(row['fullAddress'], row['postcode'], row
            ['propertyType'],  row['tenure'], row['bedrooms'], row['bathrooms'], row
            ['floorAreaSqM'], row['currentPrice']))

        tree.pack(pady=20, padx=20)

        # Display the range of estimates
        highest, lowest = rangeEstimates.range_estimates(similar_properties)
        highest_label = tk.Label(similar_properties_window, text=f"Highest Price: £{highest}")
        highest_label.pack(pady=10)
        lowest_label = tk.Label(similar_properties_window, text=f"Lowest Price: £{lowest}")
        lowest_label.pack(pady=10)

        # Button to close the window
        close_button = tk.Button(similar_properties_window, text="Close",
        command=similar_properties_window.destroy)
        close_button.pack(pady=20)

```

```
# Main function to execute the workflow
def main():
    root = create_main_window()
    property_types, tenures = get_options()
    create_widgets(root, property_types, tenures)
    root.mainloop()

if __name__ == "__main__":
    main()

```

predictPrice.py

```python
# Imports
import pickle
import pandas as pd

# Function to format data for prediction
def validate_inputs(user_details):
    # Check if property type has been selected
    if user_details[0] == "Select":
        return "type"
    # Check if postcode is valid
    if postcode_checker(user_details[1]) == False:
        return "postcode"
    # Check if address has been entered
    if user_details[2] == "":
        return "address"
    # Check if tenure has been selected
    if user_details[3] == "Select":
        return "tenure"
    #Check if area is a number
    try:
        area = int(user_details[6])
        if area <= 0:
            return "area"
    except ValueError:
        return "area_type"
    #If all inputs are valid
    else:
        return "valid"

# Function to get the headers of the dataset
def get_headers():
    df = pd.read_csv('sampled_data_dummies.csv')
    headings = df.columns.values.tolist()
    return headings

# Function to check if the postcode is in the dataset
def postcode_checker(postcode):
    headings = get_headers()
    for heading in headings:
        if heading == "postcode_" + postcode:
            return True
    return False


```

```python
# Function to return a predicted price
def predict_price(user_details):
    # Open the machine learning model
    with open('model.pkl', 'rb') as file:
        # Call load method to deserialze
        model = pickle.load(file)

    # Format the data
    new_property = format_data(user_details)

    # Predict the price
    predicted_price = model.predict(pd.DataFrame(new_property, index=[0]))
    predicted_price = round(predicted_price[0], 2)

    return predicted_price

# Function to format the data for prediction
def format_data(user_details):
    # Get headings of dataset
    headings = get_headers()

    # Create a dictionary with all headings and default values
    new_property = {category: "0" for category in headings}
    del new_property['currentPrice']
    del new_property['fullAddress']

    # Get the outcode from the postcode
    outcode = user_details[1].split(" ")[0]

    # Update the dictionary with the user's inputs
    new_property.update(
        {
            "propertyType_" + user_details[0]: "1",
            "postcode_" + user_details[1]: "1",
            "tenure_" + user_details[3]: "1",
            "bedrooms": user_details[4],
            "bathrooms": user_details[5],
            "floorAreaSqM": user_details[6],
            "outcode_" + outcode: "1"

        }
    )

    return new_property

```

similarProperties.py

```python
import pandas as pd

# Function to remove duplicates from the dataset
def remove_duplicates(dataset):
    dataset = dataset.drop_duplicates(subset=['fullAddress'])
    return dataset

# Function to get all unique postcodes from the dataset
def get_postcodes():
    df = pd.read_csv('london_house_price_data.csv')
    df = df.dropna()
    postcodes = df['postcode'].unique()
    return postcodes

# Function to extract the outcode from a postcode
def get_outcode(postcode):
    return postcode.split(" ")[0]

# Function to extract the sector from a postcode
def get_sector(postcode):
    return postcode.split(" ")[0] + " " + postcode.split(" ")[1][0]

# Function to sort the dataset by postcode using merge sort
def merge_sort(dataset, factor):
    if len(dataset) > 1:
        mid = len(dataset) // 2
        left_half = dataset[:mid]
        right_half = dataset[mid:]

        # Recursive call on each half
        merge_sort(left_half, factor)
        merge_sort(right_half, factor)

        i = j = k = 0

        # Copy data to temp arrays left_half[] and right_half[]
        while i < len(left_half) and j < len(right_half):
            if left_half[i][factor] < right_half[j][factor]:
                dataset[k] = left_half[i]
                i += 1
            else:
                dataset[k] = right_half[j]
                j += 1
            k += 1

```

```
        while i < len(left_half):
            dataset[k] = left_half[i]
            i += 1
            k += 1

        while j < len(right_half):
            dataset[k] = right_half[j]
            j += 1
            k += 1
    # Return the sorted dataset
    return dataset

# Function to find postcode with the same sector
def binary_search(dataset, postcode):
    left = 0
    right = len(dataset) - 1
    # Binary search
    while left <= right:
        mid = left + (right - left) // 2
        # Check if postcode is in sector by using starts with
        if dataset[mid][1].startswith(postcode):
            return mid
        elif dataset[mid][1] < postcode:
            left = mid + 1
        else:
            right = mid - 1
    return False

# Function to find all properties with the same sector after binary search
def find_contiguous(dataset, postcode, start):
    left = start
    right = start
    while left >= 0 and dataset[left][1].startswith(postcode):
        left -= 1
    while right < len(dataset) and dataset[right][1].startswith(postcode):
        right += 1
    # Return the dataset with the sliced values
    return dataset[left + 1:right]

# Main function to find all properties with the same sector
def similar_dataset(postcode):
    # Read the dataset, removing duplicates and Nan values
    df = pd.read_csv('london_house_price_data.csv')
    df = df.dropna()
```

```
    df = remove_duplicates(df)

    # Convert dataframe to list and sort by postcode
    df_list = df.values.tolist()
    df_list = merge_sort(df_list, 1)

    # Use sector to find similarly located properties
    sector = get_sector(postcode)
    start_value = binary_search(df_list, sector)
    if start_value is False:
        return False
    df = find_contiguous(df_list, sector, start_value)

    # Convert list back to dataframe
    df = pd.DataFrame(df, columns=['fullAddress', 'postcode', 'outcode', 'bathrooms',
    'bedrooms', 'floorAreaSqM', 'tenure', 'propertyType', 'currentPrice'])
    return df

# Function to find properties with matching type
def matching_type(type, dataset):
    mod_dataset = dataset[dataset['propertyType'] == type]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset

# Function to find properties with matching tenure
def matching_tenure(tenure, dataset):
    mod_dataset = dataset[dataset['tenure'] == tenure]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset

# Function to find properties with matching bedrooms
def matching_bedrooms(bedrooms, dataset):
    mod_dataset = dataset[dataset['bedrooms'] == bedrooms]
    if dataset.shape[0] > 3:
        return mod_dataset
    # if no exact match, find properties with similar number of bedrooms
    return dataset[abs(dataset['bedrooms'] - bedrooms) <= 1]

```

```python
# Function to find properties with matching bathrooms
def matching_bathrooms(bathrooms, dataset):
    mod_dataset = dataset[dataset['bathrooms'] == bathrooms]
    if dataset.shape[0] > 3:
        return mod_dataset
    # if no exact match, find properties with similar number of bathrooms
    return dataset[abs(dataset['bathrooms'] - bathrooms) <= 1]

# Function to find properties with similar size
def similar_size(size, dataset):
    mod_dataset = dataset[(dataset['floorAreaSqM'] >= int(size) - 20) & (dataset
    ['floorAreaSqM'] <= int(size) + 20)]
    if not mod_dataset.empty:
        return mod_dataset
    return dataset

# Function to find similar properties based on user details
def find_similar(user_details):
    dataset = similar_dataset(user_details[1])
    if dataset is False:
        return False
    # Find matching properties based on user details
    else:
        dataset = matching_type(user_details[0], dataset)
        dataset = matching_tenure(user_details[3], dataset)
        dataset = matching_bedrooms(user_details[4], dataset)
        dataset = matching_bathrooms(user_details[5], dataset)
        dataset = similar_size(user_details[6], dataset)

        # Check if empty
        if dataset.empty:
            return False

    return dataset

```

rangeEstimates.py

```python
# This function finds the highest price in the list of similar properties
def highest_price(similar_properties):
    return similar_properties['currentPrice'].max()
# This function finds the lowest price in the list of similar properties
def lowest_price(similar_properties):
    return similar_properties['currentPrice'].min()

# This function returns the range of estimates for the property
def range_estimates(similar_properties):
    highest = highest_price(similar_properties)
    lowest = lowest_price(similar_properties)
    return highest, lowest
```

map.py

```python
import sys
import requests
from PyQt5.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QLabel, QPushButton
from PyQt5.QtGui import QPixmap
from geopy.geocoders import Nominatim
from io import BytesIO

def display_map(postcode, label):
    #Fetch and display the map for a given postcode.
    geolocator = Nominatim(user_agent="pyqt_map_app")
    location = geolocator.geocode(postcode)

    # Set the status message
    if location:
        latitude, longitude = location.latitude, location.longitude
        status_message = f"Location: {latitude}, {longitude}"

        # Fetch the static map with a pin at the location
        url = f"https://static-maps.yandex.ru/1.x/?ll={longitude},{latitude}&z=14&l=map&
        size=600,400&pt={longitude},{latitude},pm2rdm"
        response = requests.get(url)

        # Load the map image if the request is successful
        if response.status_code == 200:
            pixmap = QPixmap()
            pixmap.loadFromData(BytesIO(response.content).read())
            label.setPixmap(pixmap)
        else:
            status_message = "Error loading map image."
    else:
        status_message = "Location not found!"

    return status_message

def create_window(postcode):
    #Create the PyQt5 application window.
    app = QApplication(sys.argv)

    # Create the main window
    window = QMainWindow()
    window.setWindowTitle("Map")
    window.setGeometry(100, 100, 800, 600)

```

```
    # Central widget for the window
    central_widget = QWidget(window)
    layout = QVBoxLayout(central_widget)

    # Label displaying postcode
    postcode_label = QLabel(f"Displaying map for postcode: {postcode}")
    layout.addWidget(postcode_label)

    # Label for displaying the map
    map_label = QLabel()
    map_label.setFixedSize(600, 400)
    layout.addWidget(map_label)

    # Button to close the window
    close_button = QPushButton("Close Map")
    close_button.clicked.connect(window.close)
    layout.addWidget(close_button)

    # Set the central widget
    window.setCentralWidget(central_widget)

    # Display the map immediately
    status_message = display_map(postcode, map_label)

    # Set the status message
    window.statusBar().showMessage(status_message)

    # Show the window
    window.show()
    sys.exit(app.exec_())


if __name__ == "__main__":
    postcode = "SW3 1AJ"  # Example postcode
    create_window(postcode)

```

Bibliography

J, D., G, Z., S, W., & B, X. (2022). Forecast of Hourly Airport Visibility Based on Artifical Intelligence Methods. *Atmosphere*. Retrieved October 15, 2024

M, B. (2024, December 1). *Beautiful Soup: Build a Web Scraper With Python*. Retrieved December 18, 2024, from Real Python: https://realpython.com/beautiful-soup-web-scraper-python/

M, S. (2020, March 31). *5 Sorting algorithms every programmer should know*. Retrieved October 22, 2024, from Medium: https://manis1604.medium.com/5-sorting-algorithms-every-programmers-should-know-57f26b39847c